

Abstract of thesis entitled

**“User-Dissatisfaction-Based Static Bike
Repositioning: Formulations, Solution Methods, and
Extensions to Broken Bikes and On-Site Repair”**

Submitted by

Runqiu Hu

for the degree of Doctor of Philosophy

at The University of Hong Kong

in TODO Month Year

TODO: Insert the final abstract here. The abstract should be 200–500 words.

**User-Dissatisfaction-Based Static Bike Repositioning:
Formulations, Solution Methods, and Extensions to Broken Bikes
and On-Site Repair**

by

Runqiu Hu

B.Eng.; M.Eng.

Department of Civil Engineering

The University of Hong Kong

A thesis submitted in partial fulfilment of the requirements for
the Degree of Doctor of Philosophy
at The University of Hong Kong

TODO Month Year

Declaration

I declare that this thesis represents my own work, except where due acknowledgement is made, and that it has not been previously included in a thesis, dissertation or report submitted to this University or to any other institution for a degree, diploma or other qualifications.

Signed: _____

Date: _____

Acknowledgements

TABLE OF CONTENTS

Declaration	i
Acknowledgements	ii
TABLE OF CONTENTS	iii
CHAPTER 1 INTRODUCTION	1
1.1 Background of research	1
1.2 Motivations and objectives	1
1.3 Scope and outline of the thesis	1
CHAPTER 2 LITERATURE REVIEW	2
2.1 Overview of bike repositioning problems	2
2.1.1 Static and dynamic bike repositioning	2
2.1.2 Objectives and service-quality measures	2
2.1.3 Modeling approaches	2
2.1.4 Solution methods	2
2.2 User dissatisfaction in bike repositioning problems	2
2.2.1 User Dissatisfaction Function	2
2.2.2 Extended User Dissatisfaction Function	2
2.3 Bike repositioning problems with broken bikes	2
2.3.1 Off-site handling of broken bikes	2
2.3.2 On-site repair operations	2
2.4 Cluster-first, route-second methods for bike repositioning problems .	2
2.5 Summary of research gaps	2
CHAPTER 3 USER-DISSATISFACTION-AWARE CLUSTER-FIRST, ROUTE-SECOND DECOMPOSITION FOR THE STATIC BIKE REPOSITIONING PROBLEM	3
3.1 Problem formulation	3
3.1.1 Notation	4
3.1.2 Arc-indexed formulation	5
3.2 UDF-aware clustering model	8
3.2.1 Framework overview	8
3.2.2 Clustering model	9
Notation	9
MILP formulation	10

3.2.3	Routing phase	14
3.3	Activated Transfer Selection	14
3.4	Hybrid Genetic Search for UDF-aware Clustering	18
3.4.1	Solution representation	18
3.4.2	Algorithm overview	18
3.4.3	Solution construction	20
3.4.4	Crossover	21
3.4.5	Cluster size repair	22
3.4.6	Education	23
3.4.7	Population management	24
3.5	Computational experiments	26
3.5.1	Parameter tuning	27
3.5.2	Three-way benchmark comparison	28
3.5.3	Baseline comparison	31
3.6	Conclusions	43

CHAPTER 4 STATIC BIKE REPOSITIONING WITH BROKEN BIKES AND ON-SITE REPAIR 45

4.1	Problem formulation	45
4.1.1	Notations	49
4.1.2	Time-indexed formulation	51
4.2	Solution method	60
4.2.1	Solution representation	60
4.2.2	Overall algorithmic procedure	62
4.2.3	Population initialization	63
4.2.4	Station budget constrained (SBC) heuristic for quantity de- termination	64
	Station budget allocation	67
	Loading, unloading, and repairing quantity assignment . . .	69
	Station (un)loading budget and truck (un)loading quantity adjustments	70
	Amendments to the routes and assigned quantities for trucks and repairers	70
4.2.5	Feasibility check and solution evaluation	71
4.2.6	Selection, crossover, education, and repairing	73

4.2.7	Population management and search guidance	74
4.3	Computational experiments	75
4.3.1	Parameter tuning and setting for the algorithm	77
4.3.2	Comparison of the results between HGSADC-SBC and Gurobi	78
4.3.3	Comparison of results from HGSADC-SBC and Gurobi on larger networks	79
4.3.4	Cost-effectiveness of introducing a repairer under varying repair times and broken bike inventory levels	82
4.4	Conclusions	85

**CHAPTER 5 EXTENDED-USER-DISSATISFACTION-AWARE
CLUSTER-FIRST, ROUTE-SECOND DECOMPOSITION FOR THE STATIC BIKE REPOSITIONING
PROBLEM WITH BROKEN BIKES AND ON-SITE
REPAIR**

		87
5.1	Problem formulation	87
5.1.1	Problem setting	87
5.1.2	Notation	88
5.1.3	EUDF-based objective	91
5.1.4	Time-indexed mixed-integer formulation	92
5.2	EUDF-aware cluster-first, route-second framework	98
5.2.1	Station partition representation	99
5.3	Extended Activated Transfer Selection	99
5.4	Hybrid Genetic Search for Extended-UDF Clustering	103
5.4.1	Solution construction	104
5.4.2	Cluster-based crossover	104
5.4.3	Cluster-size repair and education	104
5.4.4	Population management and diversification	105
5.5	Cluster-level planning	106
5.6	Computational experiments	106
5.6.1	Parameter tuning	107
5.6.2	Benchmark comparison on small instances	108
5.6.3	Comparison on large instances	109
5.6.4	Comparison under varied instance configurations	111
	Vehicle-capacity configurations	111

Time-budget configurations	115
Target-cluster-size configurations	118
Broken-bike-rate configurations	122
5.7 Conclusions	129
CHAPTER 6 CONCLUSIONS AND FUTURE RESEARCH DIRECTIONS	131
6.1 Summary of main findings	131
6.2 Contributions of the thesis	132
6.3 Future research directions	132
CHAPTER A Linearization of the EUDF	134
CHAPTER B SBC Heuristic for the Quantity Assignment	136
CHAPTER C Assignment Algorithms for Station Repairing Budget and Station (Un)loading Budget	139
CHAPTER D Repairing, Loading, and Unloading Quantity Assignment Algorithms	142
CHAPTER E (Un)loading Budget Adjustment Algorithm for Trucks	147
Bibliography	148

CHAPTER 1

INTRODUCTION

1.1. BACKGROUND OF RESEARCH

1.2. MOTIVATIONS AND OBJECTIVES

1.3. SCOPE AND OUTLINE OF THE THESIS

CHAPTER 2

LITERATURE REVIEW

2.1. OVERVIEW OF BIKE REPOSITIONING PROBLEMS

2.1.1. Static and dynamic bike repositioning

2.1.2. Objectives and service-quality measures

2.1.3. Modeling approaches

2.1.4. Solution methods

2.2. USER DISSATISFACTION IN BIKE REPOSITIONING PROBLEMS

2.2.1. User Dissatisfaction Function

2.2.2. Extended User Dissatisfaction Function

2.3. BIKE REPOSITIONING PROBLEMS WITH BROKEN BIKES

2.3.1. Off-site handling of broken bikes

2.3.2. On-site repair operations

2.4. CLUSTER-FIRST, ROUTE-SECOND METHODS FOR BIKE REPOSITIONING PROBLEMS

2.5. SUMMARY OF RESEARCH GAPS

CHAPTER 3

USER-DISSATISFACTION-AWARE CLUSTER-FIRST, ROUTE-SECOND DECOMPOSITION FOR THE STATIC BIKE REPOSITIONING PROBLEM

This chapter develops a user-dissatisfaction-aware cluster-first, route-second decomposition framework for the multi-vehicle static bike repositioning problem. The aim is to improve the scalability of UDF-based repositioning by incorporating user dissatisfaction reduction directly into the clustering phase, rather than treating clustering as a purely distance- or demand-based preprocessing step.

The chapter first presents the benchmark multi-vehicle static BRP formulation with UDF as the service-quality objective. It then formulates the UDF-aware clustering model, introduces the Activated Transfer Selection procedure for evaluating candidate clusters, and develops a Hybrid Genetic Search for solving the clustering problem at city scale. Computational experiments are finally conducted to evaluate the proposed method against exact optimization and several baseline clustering methods.

3.1. PROBLEM FORMULATION

This section defines the multi-vehicle static BRP (MV-BRP) studied in this chapter and presents its mixed-integer linear programming formulation.

We consider a BSS consisting of a single depot and a set of stations. Each station is characterized by its capacity, its initial bike inventory, and a user dissatisfaction function. The user dissatisfaction function represents the expected shortage of bikes and docks incurred by station users as a function of the station's inventory after repositioning is carried out (Raviv and Kolka, 2013). Travel times between all pairs of nodes are known. The initial inventory at each station is assumed to be known, as modern BSSs provide the operator with real-time inventory data. The depot is assumed to hold a sufficient inventory of bikes and to have sufficient capacity to store returned bikes.

A fleet of homogeneous vehicles with limited capacity is used to redistribute bikes across the stations. A station whose initial inventory exceeds the level that

minimizes its UDF is called a surplus station, and a station whose initial inventory falls below that level is called a deficit station. Vehicles depart from the depot simultaneously, collect bikes from surplus stations and deliver bikes to deficit stations, and finally return to the depot within a given time budget. Loading and unloading each bike takes a fixed amount of time. Repositioning is performed overnight, when user activity is negligible. Station inventories change only through vehicle operations. We assume that each station may be visited by at most one vehicle. The inventory at each station after repositioning sets the starting condition for the following operating day. The objective is to determine the route of each vehicle and the number of bikes loaded and unloaded at each visited station to maximize the total reduction in user dissatisfaction across all stations.

We adopt the user dissatisfaction function introduced by Raviv and Kolka (2013). For each station i , $f_i(s)$ measures the expected number of failed rentals and returns over the planning horizon as a function of the station's inventory s after the repositioning, and is convex over the integer inventory levels $0, 1, \dots, c_i$. Convexity permits an exact piecewise-linear representation that can be embedded in a linear program. The UDF values at all inventory levels are precomputed and treated as input to the optimization model. We refer the reader to Raviv and Kolka (2013) for the computational procedure.

3.1.1. Notation

Table 3.1: Notation for the MV-BRP model.

Symbol	Description
<i>Sets</i>	
V	Set of stations
V_0	Set of all nodes, including the stations and the depot
K	Set of vehicles
<i>Parameters</i>	
c_i	Capacity of station i
s_i^0	Initial inventory at station i
Q	Vehicle capacity
T	Time budget per vehicle
t_{ij}	Travel time from node i to node j

Symbol	Description
t^{load}	Time to load or unload one bike
$a_{i\ell}$	Intercept coefficient in the linearization of f_i at inventory level ℓ
$b_{i\ell}$	Slope coefficient in the linearization of f_i at inventory level ℓ
<i>Decision variables</i>	
x_{ijk}	Binary variable that equals one if vehicle k travels directly from node i to node j , and zero otherwise
y_{ijk}	Number of bikes on vehicle k when it travels directly from node i to node j
y_{ik}^{load}	Number of bikes loaded onto vehicle k at node i
y_{ik}^{unload}	Number of bikes unloaded from vehicle k at node i
s_i	Inventory at station i after the repositioning
g_i	Linearized user dissatisfaction at station i after the repositioning
<i>Function</i>	
$f_i(s)$	User dissatisfaction function at station i

3.1.2. Arc-indexed formulation

For the sake of completeness, we present the multi-vehicle static BRP as a mixed-integer linear program, building on the arc-indexed formulation of Raviv et al. (2013). We specialize to a homogeneous fleet with shared capacity Q and equal loading and unloading times t^{load} , and restrict each station to be visited by at most one vehicle. The formulation is

$$\max \sum_{i \in V} (f_i(s_i^0) - g_i) \quad (3.1)$$

subject to

$$s_i = s_i^0 - \sum_{k \in K} (y_{ik}^{\text{load}} - y_{ik}^{\text{unload}}) \quad \forall i \in V \quad (3.2)$$

$$y_{ik}^{\text{load}} - y_{ik}^{\text{unload}} = \sum_{\substack{j \in V_0 \\ j \neq i}} y_{ijk} - \sum_{\substack{j \in V_0 \\ j \neq i}} y_{jik} \quad \forall i \in V_0, k \in K \quad (3.3)$$

$$y_{ijk} \leq Qx_{ijk} \quad \forall i, j \in V_0 (i \neq j), k \in K \quad (3.4)$$

$$\sum_{\substack{j \in V_0 \\ j \neq i}} x_{ijk} = \sum_{\substack{j \in V_0 \\ j \neq i}} x_{jik} \quad \forall i \in V_0, k \in K \quad (3.5)$$

$$\sum_{k \in K} \sum_{\substack{j \in V_0 \\ j \neq i}} x_{jik} \leq 1 \quad \forall i \in V \quad (3.6)$$

$$\sum_{k \in K} y_{ik}^{\text{load}} \leq s_i^0 \quad \forall i \in V \quad (3.7)$$

$$\sum_{k \in K} y_{ik}^{\text{unload}} \leq c_i - s_i^0 \quad \forall i \in V \quad (3.8)$$

$$\sum_{i \in V_0} (y_{ik}^{\text{load}} - y_{ik}^{\text{unload}}) = 0 \quad \forall k \in K \quad (3.9)$$

$$t^{\text{load}} \cdot \sum_{i \in V_0} (y_{ik}^{\text{load}} + y_{ik}^{\text{unload}}) + \sum_{\substack{i, j \in V_0 \\ i \neq j}} t_{ij} x_{ijk} \leq T \quad \forall k \in K \quad (3.10)$$

$$\sum_{j \in V} x_{0jk} = 1 \quad \forall k \in K \quad (3.11)$$

$$\sum_{j \in V} x_{j0k} = 1 \quad \forall k \in K \quad (3.12)$$

$$y_{ik}^{\text{load}} \leq s_i^0 \sum_{\substack{j \in V_0 \\ j \neq i}} x_{jik} \quad \forall i \in V, k \in K \quad (3.13)$$

$$y_{ik}^{\text{unload}} \leq (c_i - s_i^0) \sum_{\substack{j \in V_0 \\ j \neq i}} x_{jik} \quad \forall i \in V, k \in K \quad (3.14)$$

$$g_i \geq a_{i\ell} + b_{i\ell} s_i \quad \forall i \in V, \ell = 0, 1, \dots, c_i - 1 \quad (3.15)$$

$$\sum_{i \in S} \sum_{\substack{j \in S \\ j \neq i}} x_{ijk} \leq |S| - 1 \quad \forall S \subseteq V, |S| \geq 2, \forall k \in K \quad (3.16)$$

$$x_{ijk} \in \{0, 1\} \quad \forall i, j \in V_0 (i \neq j), k \in K \quad (3.17)$$

$$y_{ik}^{\text{load}} \geq 0, \text{ integer} \quad \forall i \in V_0, k \in K \quad (3.18)$$

$$y_{ik}^{\text{load}} \geq 0, \text{ integer} \quad \forall i \in V_0, k \in K \quad (3.19)$$

$$y_{ijk} \geq 0 \quad \forall i, j \in V_0 (i \neq j), k \in K \quad (3.20)$$

$$s_i \geq 0 \quad \forall i \in V \quad (3.21)$$

$$s_i \leq c_i \quad \forall i \in V \quad (3.22)$$

$$g_i \geq 0 \quad \forall i \in V \quad (3.23)$$

The objective function (3.1) maximizes the total reduction in user dissatisfaction across all stations: for each station i , the term $f_i(s_i^0) - g_i$ measures the improvement from the initial UDF level to the level after the repositioning. Constraints (3.2) define the post-repositioning inventory at each station as the initial inventory adjusted by all loading and unloading operations. Constraints (3.3) ensure conservation of

bikes on each vehicle: the net loading at a node equals the change in vehicle load. Constraints (3.4) limit the load on each vehicle to its capacity and set the load to zero on arcs not traversed. Constraints (3.5) are the vehicle flow conservation equations. Constraints (3.6) stipulate that each station can be visited by at most one vehicle across the fleet.

Constraints (3.7) and (3.8) bound the total loading and unloading at each station by the available inventory and remaining capacity, respectively. Constraints (3.9) require that all bikes collected by each vehicle are eventually delivered. Constraints (3.10) limit the total operating time of each vehicle, including travel, loading, and unloading, to the time budget. Constraints (3.11) and (3.12) require each vehicle to depart from and return to the depot exactly once. Constraints (3.13) and (3.14) tighten the loading and unloading bounds per vehicle by conditioning on whether that vehicle visits the station. These valid inequalities strengthen the LP relaxation (Raviv et al., 2013).

The UDF at each station is linearized following Raviv et al. (2013). Constraints (3.15) are the linear constraints that support the convex function f_i . For each station i and inventory level $\ell = 0, 1, \dots, c_i - 1$, the slope and intercept are $b_{i\ell} = f_i(\ell + 1) - f_i(\ell)$ and $a_{i\ell} = f_i(\ell) - b_{i\ell} \cdot \ell$, respectively. Because f_i is convex and defined over integer values, this linearization is exact (Raviv et al., 2013). Constraints (3.16) are the Dantzig-Fulkerson-Johnson (DFJ) subtour elimination constraints (Dantzig et al., 1954), which prevent any subset of stations from forming a disconnected tour. Constraints (3.17)–(3.23) define the variable domains and bounds.

3.2. UDF-AWARE CLUSTERING MODEL

3.2.1. Framework overview

This section presents the UDF-aware CFRS framework for the multi-vehicle BRP. The framework decomposes the problem into a clustering phase, which partitions the station set V into $|K|$ disjoint groups (one per vehicle) by maximizing the total estimated UDF reduction subject to a per-cluster time budget, and a routing phase, which solves a single-vehicle instance of the BRP within each cluster.

Figure 3.1 illustrates the framework on a small example with one depot and nine stations partitioned into $|K| = 3$ clusters. In the figure, square node 0 denotes

CFRS framework diagram placeholder.

Figure 3.1: Illustration of the CFRS framework on a BSS network with one depot, nine stations, and $|K| = 3$ vehicles (clusters).

the depot. Red and blue circles denote surplus stations ($s_i^0 > s_i^*$) and deficit stations ($s_i^0 < s_i^*$), respectively. Bars next to each station show the current inventory s_i^0 relative to the optimal level s_i^* . Phase 1 partitions the stations into clusters and determines approximate within-cluster transfers between surplus and deficit stations. Phase 2 solves a single-vehicle BRP within each cluster. The system-wide UDF reduction is the sum of the per-cluster reductions.

The clustering phase is formulated below as a mixed-integer linear program, followed by the cluster-level routing phase.

3.2.2. Clustering model

We formulate the clustering phase as a mixed-integer linear program that jointly assigns stations to clusters and determines approximate transfers between surplus and deficit stations within each cluster, subject to a per-cluster time budget and cluster size limits. The notation from Table 3.1 carries over. Table 3.2 lists the additional sets, parameters, and decision variables.

Notation

Table 3.2: Additional notation for the clustering model.

Symbol	Description
<i>Sets</i>	
V^+	Set of surplus stations, $V^+ = \{i \in V \mid s_i^0 > s_i^*\}$
V^-	Set of deficit stations, $V^- = \{i \in V \mid s_i^0 < s_i^*\}$
<i>Parameters</i>	
s_i^*	Inventory level that minimizes f_i at station i
U_{ij}	Maximum number of bikes transferable from station i to station j , $U_{ij} = \min \{s_i^0 - s_i^*, s_j^* - s_j^0\}$ if $i \in V^+$ and $j \in V^-$, 0 otherwise
$N_{\min}$	Minimum number of stations per cluster

Symbol	Description
$N_{\max}$	Maximum number of stations per cluster
<i>Decision variables</i>	
z_{ik}	Binary variable that equals one if station i is assigned to cluster k , and zero otherwise
w_{ijk}	Number of bikes transferred from station i to station j within cluster k
δ_{ik}	Binary variable that equals one if station i participates in at least one transfer within cluster k , and zero otherwise
τ_k^{star}	Maximum spanning star component of the estimated travel time for cluster k
τ_k^{depot}	Depot round-trip component of the estimated travel time for cluster k
$\tilde{s}_i$	Estimated inventory at station i after the bike transfers
$\tilde{g}_i$	Estimated linearized UDF at station i after the bike transfers

MILP formulation

The formulation is

$$\max \sum_{i \in V} \left(f_i(s_i^0) - \tilde{g}_i \right) \quad (3.24)$$

subject to

$$\sum_{k \in K} z_{ik} = 1 \quad \forall i \in V \quad (3.25)$$

$$w_{ijk} \leq U_{ij} z_{ik} \quad \forall i \in V^+, j \in V^-, k \in K \quad (3.26)$$

$$w_{ijk} \leq U_{ij} z_{jk} \quad \forall i \in V^+, j \in V^-, k \in K \quad (3.27)$$

$$\sum_{j \in V^-} \sum_{k \in K} w_{ijk} \leq s_i^0 - s_i^* \quad \forall i \in V^+ \quad (3.28)$$

$$\sum_{i \in V^+} \sum_{k \in K} w_{ijk} \leq s_j^* - s_j^0 \quad \forall j \in V^- \quad (3.29)$$

$$\delta_{ik} \leq z_{ik} \quad \forall i \in V, k \in K \quad (3.30)$$

$$\sum_{j \in V^-} w_{ijk} \leq (s_i^0 - s_i^*) \delta_{ik} \quad \forall i \in V^+, k \in K \quad (3.31)$$

$$\sum_{j \in V^-} w_{ijk} \geq \delta_{ik} \quad \forall i \in V^+, k \in K \quad (3.32)$$

$$\sum_{i \in V^+} w_{ijk} \leq (s_j^* - s_j^0) \delta_{jk} \quad \forall j \in V^-, k \in K \quad (3.33)$$

$$\sum_{i \in V^+} w_{ijk} \geq \delta_{jk} \quad \forall j \in V^-, k \in K \quad (3.34)$$

$$\tau_k^{\text{star}} \geq \sum_{i \in V} \max \{t_{ij}, t_{ji}\} \cdot (\delta_{ik} + \delta_{jk} - 1) \quad \forall j \in V, k \in K \quad (3.35)$$

$$\tau_k^{\text{depot}} \geq 2 \max \{t_{0j}, t_{j0}\} \cdot \delta_{jk} \quad \forall j \in V, k \in K \quad (3.36)$$

$$\tau_k^{\text{star}} + \tau_k^{\text{depot}} + 2t^{\text{load}} \sum_{i \in V^+} \sum_{j \in V^-} w_{ijk} \leq T \quad \forall k \in K \quad (3.37)$$

$$\sum_{i \in V} z_{ik} \geq N_{\min} \quad \forall k \in K \quad (3.38)$$

$$\sum_{i \in V} z_{ik} \leq N_{\max} \quad \forall k \in K \quad (3.39)$$

$$\tilde{s}_i = s_i^0 - \sum_{j \in V^-} \sum_{k \in K} w_{ijk} \quad \forall i \in V^+ \quad (3.40)$$

$$\tilde{s}_j = s_j^0 + \sum_{i \in V^+} \sum_{k \in K} w_{ijk} \quad \forall j \in V^- \quad (3.41)$$

$$\tilde{s}_i = s_i^0 \quad \forall i \in V \setminus (V^+ \cup V^-) \quad (3.42)$$

$$\tilde{g}_i \geq a_{i\ell} + b_{i\ell} \tilde{s}_i \quad \forall i \in V, \ell = 0, 1, \dots, c_i - 1 \quad (3.43)$$

$$z_{ik} \in \{0, 1\} \quad \forall i \in V, k \in K \quad (3.44)$$

$$\delta_{ik} \in \{0, 1\} \quad \forall i \in V, k \in K \quad (3.45)$$

$$w_{ijk} \geq 0, \text{ integer} \quad \forall i \in V^+, j \in V^-, k \in K \quad (3.46)$$

$$\tau_k^{\text{star}} \geq 0 \quad \forall k \in K \quad (3.47)$$

$$\tau_k^{\text{depot}} \geq 0 \quad \forall k \in K \quad (3.48)$$

$$\tilde{s}_i \geq 0 \quad \forall i \in V \quad (3.49)$$

$$\tilde{s}_i \leq c_i \quad \forall i \in V \quad (3.50)$$

$$\tilde{g}_i \geq 0 \quad \forall i \in V \quad (3.51)$$

The objective function (3.24) has the same form as (3.1), with $\tilde{g}_i$ replacing g_i to indicate that the UDF is evaluated at the estimated inventory after the transfers rather than at the actual repositioning outcome. Constraints (3.25) ensure that each station is assigned to exactly one cluster. Constraints (3.26) and (3.27) ensure that a transfer between two stations can take place only if both are assigned to the same cluster.

Constraints (3.28) and (3.29) bound the total transfers out of each surplus station and into each deficit station by the available surplus and deficit, respectively. Constraints (3.30)–(3.34) tie the activation variables to the assignment and transfer variables, so that $\delta_{ik} = 1$ if and only if station i is assigned to cluster k and participates in at least one transfer within it. Balanced stations ($s_i^0 = s_i^*$) are never activated, as they have neither surplus to offer nor deficit to fill.

Constraints (3.35)–(3.37) approximate the within-cluster routing time as the sum of a maximum spanning star (MSS) component and a depot round-trip component, following Schuijbroek et al. (2017). Specifically, it is defined as the maximum over all candidate centers of the sum of distances from the center to all other active stations in the cluster, which is an upper bound on the shortest Hamiltonian path through the cluster and admits a compact linear representation. Constraints (3.35) encode this maximum through the weight $(\delta_{ik} + \delta_{jk} - 1)$: when j is an active center ($\delta_{jk} = 1$), it reduces to δ_{ik} and the right-hand side becomes the sum of distances from j to all other active stations in cluster k . Otherwise, every term is non-positive and the inequality is deactivated. Constraints (3.36) account for the depot round-trip through the farthest active station. Constraints (3.37) enforce the time budget, where the factor 2 on t^{load} reflects the loading time at the source and the unloading time at the destination of each transferred bike.

When the travel time matrix is asymmetric, constraints (3.35) and (3.36) use the symmetrized value $\max \{t_{ij}, t_{ji}\}$ on each station pair rather than a single directed value. Since the within-cluster route is not yet determined at the clustering stage, the orientation in which any edge would be traversed is unknown. Taking the maximum of the two directions yields a symmetric surrogate that upper-bounds the true routing cost in either orientation, so a cluster satisfying the time budget under this surrogate remains feasible once the actual route is computed.

Constraints (3.38) and (3.39) bound the cluster sizes between $N_{\min}$ and $N_{\max}$.

Constraints (3.40)–(3.42) define the estimated inventory at each station as the initial inventory adjusted by the within-cluster transfers. Constraints (3.43) linearize the estimated UDF using the same piecewise-linear technique as constraints (3.15) in Section 3.1.2. The representation is exact because f_i is convex over integer inventory levels. Constraints (3.44)–(3.51) define the variable domains and bounds.

Compared with the savings heuristic of Forma et al. (2015), which also incorporates UDF into clustering but evaluates each cluster’s UDF reduction by optimally redistributing its total bike count under the assumption of costless within-cluster transfers, our formulation differs in two structural aspects. First, transfer quantities w_{ijk} between individual surplus–deficit pairs are determined explicitly, bounded by each station’s available surplus or deficit, so that UDF is evaluated at station-level rather than cluster-level inventories. Second, these transfers are subject to the per-cluster time budget through the MSS approximation in constraints (3.35)–(3.37), so that geographic proximity and routing feasibility enter the clustering decision directly rather than through separate proximity constraints.

The MSS is known to consistently overestimate the actual routing cost (Schuijbroek et al., 2017), so the time budget constraints (3.37) are enforced against an upper bound rather than the exact shortest Hamiltonian path. This keeps the clustering model linear and free of exact routing variables, at the cost of excluding some clusters whose exact single-vehicle route would in fact fit within the time budget. Nonetheless, the clustering MILP remains intractable for city-scale instances, motivating the metaheuristic developed in Section 3.4.

3.2.3. Routing phase

Given the partition produced by the clustering phase, each cluster is solved as an independent single-vehicle instance of the BRP formulated in Section 3.1.2, restricted to the stations assigned to that cluster together with the depot. Only the station-to-cluster assignment is passed to the routing phase. The approximate transfers w_{ijk} from the clustering phase are discarded, and each subproblem is solved from scratch on its own UDF curves and travel time submatrix.

3.3. ACTIVATED TRANSFER SELECTION

To evaluate a solution $\phi = (C_1, \dots, C_{|K|})$, the algorithm computes the total estimated UDF reduction across all clusters. The UDF reduction at each station is determined

by its inventory after the repositioning, which in turn depends on the bike transfers carried out within its cluster. Exact computation of the optimal within-cluster transfers and the corresponding route requires solving a single-vehicle BRP per cluster, which is impractical inside a population-based metaheuristic that evaluates many clusters per generation.

Therefore, we propose the Activated Transfer Selection (ATS) procedure, which estimates a time-feasible transfer set for each cluster. ATS scores each candidate transfer by its expected UDF improvement per unit handling time, selects transfers in decreasing score order, and terminates when the time budget is exhausted.

The ATS procedure operates on the following derived quantities, defined per cluster C_k unless otherwise stated:

- (1) Surplus and deficit quantities. For each station $i \in V^+$, the surplus quantity is $r_i^+ = s_i^0 - s_i^*$. For each station $j \in V^-$, the deficit quantity is $r_j^- = s_j^* - s_j^0$.
- (2) Edges. A directed pair (i, j) with $i \in V^+$ and $j \in V^-$ is called an edge. The maximum transferable quantity on edge (i, j) is $\min(r_i^+, r_j^-)$.
- (3) Transfers and transfer sets. A transfer is a triple $e = (i, j, q)$, where (i, j) is an edge and $q \geq 1$ is the number of bikes moved from i to j . A transfer set $\mathcal{T}_k$ collects the transfers selected for execution within cluster C_k .
- (4) Marginal UDF reductions. For a surplus station i , the m -th marginal UDF reduction is $\Delta_i^m = f_i(s_i^0 - m + 1) - f_i(s_i^0 - m)$ for $m = 1, \dots, r_i^+$, representing the decrease in UDF when the m -th bike is removed. For a deficit station j , the m -th marginal UDF reduction is $\Delta_j^m = f_j(s_j^0 + m - 1) - f_j(s_j^0 + m)$ for $m = 1, \dots, r_j^-$. Because f_i is convex, the marginals are decreasing: $\Delta_i^1 \geq \Delta_i^2 \geq \dots \geq 0$.
- (5) Edge scores. The score of edge (i, j) is $\sigma_{ij} = (\Delta_i^1 + \Delta_j^1)/(2t^{\text{load}})$, where the denominator accounts for one loading and one unloading operation. The score measures the UDF reduction per unit handling time for the first bike transferred on the edge.
- (6) Activated stations. The set of stations participating in at least one accepted transfer within cluster C_k is denoted $\mathcal{A} \subseteq C_k$. Each member of $\mathcal{A}$ must be visited by the vehicle, incurring routing time.

- (7) Routing time estimate. The routing time for a set of activated stations $\mathcal{A}$ is estimated as $\tau(\mathcal{A}) = MSS(\mathcal{A}) + 2\max_{i \in \mathcal{A}} \max(t_{0i}, t_{i0})$, where $MSS(\mathcal{A}) = \max_{i \in \mathcal{A}} \sum_{j \in \mathcal{A}, j \neq i} \max(t_{ij}, t_{ji})$ is the maximum spanning star and the second term accounts for the depot round trip. This is the same estimate used in the clustering model.
- (8) MSS time feasibility. A transfer set $\mathcal{T}_k$ is MSS time-feasible if $\tau(\mathcal{A}) + 2t^{\text{load}} \sum_{(i,j,q) \in \mathcal{T}_k} q \leq T$, where the handling term reflects that each transferred bike is loaded at its source and unloaded at its destination. Because the MSS provides an upper bound on the optimal tour length through $\mathcal{A}$, any transfer set that satisfies this condition remains feasible when the actual route is computed.

We now present the ATS procedure in detail. The procedure takes a cluster C_k as input and returns an MSS time-feasible transfer set $\mathcal{T}_k$. Each cluster is processed independently.

Step 0 (Initialization). Generate all edges within C_k and sort them by σ_{ij} in decreasing order. Set $\mathcal{A} \leftarrow \emptyset$ and $\mathcal{T}_k \leftarrow \emptyset$. Initialize the residual surplus $\tilde{r}_i^+ \leftarrow r_i^+$ for all $i \in V^+ \cap C_k$ and the residual deficit $\tilde{r}_j^- \leftarrow r_j^-$ for all $j \in V^- \cap C_k$.

Step 1 (First-marginal selection). Process the sorted edges sequentially. For each edge (i, j) , skip if $i \in \mathcal{A}$ or $j \in \mathcal{A}$. Otherwise, tentatively add i and j to $\mathcal{A}$, recompute $\tau(\mathcal{A})$, and compute the feasible quantity

$$q = \min \left\{ \tilde{r}_i^+, \tilde{r}_j^-, \left\lfloor \frac{T - \tau(\mathcal{A}) - 2t^{\text{load}} \sum_{(i',j',q') \in \mathcal{T}_k} q'}{2t^{\text{load}}} \right\rfloor \right\},$$

If $q \geq 1$, accept transfer (i, j, q) into $\mathcal{T}_k$ and update $\tilde{r}_i^+ \leftarrow \tilde{r}_i^+ - q$ and $\tilde{r}_j^- \leftarrow \tilde{r}_j^- - q$. If $q < 1$, remove i and j from $\mathcal{A}$ and proceed to the next edge. Skipping edges whose stations are already active enforces mutual exclusivity and keeps the precomputed first-marginal scores valid throughout this step.

Step 2 (Residual capacity matching). This step fills in additional transfers that reuse stations already activated in Step 1, exploiting any remaining time budget. A station may appear in multiple transfers during this step. Each additional bike on a station is scored by its own marginal UDF reduction, so that the diminishing returns on a repeatedly used station are reflected.

At each iteration, Step 2 selects the surplus-deficit pair (i, j) within C_k with the highest score

$$\tilde{\sigma}_{ij} = \frac{\Delta_i^{m_i+1} + \Delta_j^{m_j+1}}{2 t^{\text{load}}},$$

where m_i and m_j are the numbers of bikes already transferred from i and into j by previously accepted transfers in this cluster. Only pairs with $\tilde{\sigma}_{ij} > 0$ are considered.

For the selected pair, any inactive station is tentatively activated. $\tau(\mathcal{A})$ is recomputed, and the feasible quantity q is determined as in Step 1. If $q \geq 1$, the transfer is accepted and all residual quantities are updated. Step 2 terminates when no surplus-deficit pair with a positive score remains, or when the remaining time budget can no longer accommodate any transfer.

After applying ATS to every cluster, the objective value of solution ϕ is

$$F(\phi) = \sum_{i \in V} [f_i(s_i^0) - f_i(s_i^0 + v_i)],$$

where

$$v_i = \begin{cases} -\sum_{j:(i,j,q) \in \mathcal{T}} q, & \text{if } i \in V^+, \\ \sum_{j:(j,i,q) \in \mathcal{T}} q, & \text{if } i \in V^-, \\ 0, & \text{otherwise,} \end{cases}$$

is the net inventory change at station i and $\mathcal{T} = \bigcup_{k=1}^{|K|} \mathcal{T}_k$ is the set of all accepted transfers. Figure 3.2 illustrates the procedure on a four-station toy cluster.

ATS procedure diagram placeholder.

Figure 3.2: Illustration of the ATS procedure on a toy cluster with surplus stations 1 and 2 and deficit stations 3 and 4. (a) Candidate edges are ranked by the first-marginal score σ_{ij} . (b) Step 1 scans the edges in decreasing σ_{ij} order and accepts transfers (1, 3, 2) and (2, 4, 2), activating all four stations and consuming 70% of the time budget T . (c) Step 2 revisits the previously skipped pair (1, 4) and appends a transfer of $q = 1$, raising time budget usage to 80%. The accepted-transfer set $\mathcal{T}_k$ shown below each panel updates accordingly.

3.4. HYBRID GENETIC SEARCH FOR UDF-AWARE CLUSTERING

This section presents the Hybrid Genetic Search for UDF-aware Clustering (HGS-UC), a metaheuristic for the clustering model formulated in Section 3.2. HGS-UC adapts the Hybrid Genetic Search framework of Vidal et al. (2012), refined in the open-source implementation of Vidal (2022).

3.4.1. Solution representation

A clustering solution is represented by $\phi = (C_1, C_2, \dots, C_{|K|})$, where K is the homogeneous vehicle set and C_k denotes the set of stations assigned to cluster k . The clusters form a partition of V , that is $\bigcup_{k=1}^{|K|} C_k = V$ and $C_k \cap C_{k'} = \emptyset, \forall k \neq k'$. The depot is not assigned to any cluster and is not represented in the solution. Since each cluster is served by exactly one vehicle, the terms cluster and vehicle are used interchangeably throughout this section.

For example, given $|V| = 9$ stations and $|K| = 3$ vehicles, one feasible solution is $\phi = (\{2, 5, 7\}, \{1, 4, 9\}, \{3, 6, 8\})$, in which stations 2, 5, and 7 are assigned to vehicle 1, stations 1, 4, and 9 to vehicle 2, and stations 3, 6, and 8 to vehicle 3.

3.4.2. Algorithm overview

The algorithmic procedure of HGS-UC is given in Algorithm 1. The algorithm evolves a population P of clustering solutions. The objective value of a solution ϕ is denoted as $F(\phi)$ ¹.

The algorithm first generates an initial population P (Line 1) and identifies the best solution ϕ^* (Line 2). It then records the current best objective value as F_{prev} and initializes the non-improvement counters $iter_{\text{NI}}$ and $iter_{\text{div}}$ to zero (Line 3). The population then iteratively evolves through successive operations (Lines 4–25). In each generation, two parents are selected by binary tournament based on their biased fitness values (Line 5) and recombined with swapped parent roles to produce two offspring via crossover (Lines 6–11), each undergoing cluster size repair and education before entering the population. Survivor selection is applied if the population exceeds the maximum size (Lines 12–14). After updating ϕ (Line 15), the counters $iter_{\text{NI}}$ and $iter_{\text{div}}$ are reset to 0 if $F(\phi^*)$ improves over F_{prev} , and

¹Throughout this section, F denotes the objective value of the optimization problem, while f retains its meaning as the UDF defined in Sections 3.1 and 3.2.

are incremented by 1 otherwise (Lines 16–20). A diversification phase is activated every It_{div} iterations without improvement (Lines 21–24), retaining a small set of solutions and regenerating the remainder through the initialization procedure. The algorithm terminates and returns the best-known solution ϕ^* and its objective value $F(\phi^*)$ (Line 26) when the number of consecutive non-improving iterations $iter_{NI}$ reaches the limit It_{NI} or the elapsed time reaches T_{max} .

Algorithm 1: HGS-UC

```

1  $P \leftarrow \text{InitPopulation};$ 
2  $\phi^* \leftarrow \arg \max_{\phi \in P} F(\phi);$ 
3 Set  $F_{prev} \leftarrow F(\phi^*)$ ,  $iter_{NI} \leftarrow 0$ , and  $iter_{div} \leftarrow 0$ ;
4 while  $iter_{NI} < It_{NI}$  and elapsed time  $< T_{max}$  do
5   Select  $\phi_1$  and  $\phi_2$  from  $P$  by binary tournament based on biased fitness;
6   for  $(\phi_A, \phi_B) \in \{(\phi_1, \phi_2), (\phi_2, \phi_1)\}$  do
7      $\phi \leftarrow \text{Crossover}(\phi_A, \phi_B);$ 
8      $\text{ClusterSizeRepair}(\phi);$ 
9      $\text{Educate}(\phi);$ 
10    Add  $\phi$  to  $P$ ;
11   end
12   if the size of  $P$  exceeds the maximum population size then
13      $\text{SelectSurvivors}(P);$ 
14   end
15   Update  $\phi^*$  if a better solution exists in  $P$ ;
16   if  $F(\phi^*) > F_{prev}$  then
17     Set  $F_{prev} \leftarrow F(\phi^*)$ ,  $iter_{NI} \leftarrow 0$ , and  $iter_{div} \leftarrow 0$ ;
18   end
19   else
20     Set  $iter_{NI} \leftarrow iter_{NI} + 1$  and  $iter_{div} \leftarrow iter_{div} + 1$ ;
21   end
22   if  $iter_{div} \geq It_{div}$  then
23      $\text{Diversify}(P);$ 
24     Set  $iter_{div} \leftarrow 0$ ;
25   end
26 end
27 return  $\phi^*, F(\phi^*)$ 

```

Unlike the original HGS framework, which maintains separate subpopulations of feasible and infeasible solutions, HGS-UC operates on a single feasible population. Feasibility is enforced by the cluster size repair (Line 8) invoked before each individual enters the population.

The ATS-based solution evaluation procedure is described in Section 3.3. Solution construction, crossover, cluster size repair, education, and population management are described below.

3.4.3. Solution construction

The algorithm constructs new solutions when building the initial population (Algorithm 1, Line 1) and when regenerating individuals during diversification (Line 22). Each solution $\phi = (C_1, \dots, C_{|K|})$ is constructed from scratch by applying K-medoids clustering on a composite dissimilarity measure, which combines two normalized components: geographic proximity, measured by the travel time t_{ij} , and UDF reduction potential, measured by the bilateral UDF reduction B_{ij} .

Specifically, the bilateral UDF reduction of stations i and j is defined as

$$B_{ij} = \begin{cases} \max(f_i(s_i^0) + f_j(s_j^0) - f_i(s_i^0 - q_{ij}) - f_j(s_j^0 + q_{ij}), 0) & \text{if } i \in V^+, j \in V^- \\ 0 & \text{otherwise} \end{cases},$$

where $q_{ij} = \min(r_i^+, r_j^-)$. The dissimilarity of stations i and j is then defined as

$$D_{ij} = \alpha \hat{t}_{ij} + (1 - \alpha)(1 - \hat{B}_{ij}),$$

where $\hat{t}_{ij}$ and $\hat{B}_{ij}$ denote min-max normalizations over all off-diagonal station pairs, and $\alpha \in [0, 1]$ controls the relative weight. A low D_{ij} indicates that i and j are geographically close and offer substantial mutual UDF reduction, making them desirable cluster partners.

The construction procedure applies K-medoids clustering with K-means++ seeding (Arthur and Vassilvitskii, 2007), adapted to the K-medoids setting by sampling proportionally to the dissimilarity rather than its square. The procedure is as follows:

Step 0 (Seeding). Select the first medoid m_1 uniformly at random from V . For $k = 2, \dots, |K|$, let $M_{k-1} = \{m_1, \dots, m_{k-1}\}$ and $d_i = \min_{m \in M_{k-1}} D_{i,m}$ for each $i \in V \setminus M_{k-1}$. Sample $m_k = i$ from $V \setminus M_{k-1}$ with probability $\frac{d_i}{\sum_{j \in V \setminus M_{k-1}} d_j}$.

Step 1 (Assignment). Assign each station $i \in V$ to the cluster C_k whose medoid minimizes D_{i,m_k} .

Step 2 (Medoid update). Recompute each medoid as $m_k = \arg \min_{i \in C_k} \sum_{j \in C_k} D_{ij}$.

Step 3 (Convergence check). If the set of medoids has changed, return to Step 1. Otherwise, terminate and return $\phi = (C_1, \dots, C_{|K|})$.

Diversity across constructed solutions is enabled through the stochastic seeding in Step 0. Each invocation samples different initial medoids and produces a distinct partition.

3.4.4. Crossover

The crossover operator recombines two parent solutions to produce an offspring that inherits intra-cluster structure from one parent and inter-cluster boundaries from the other. Two offspring are produced per generation by applying the operator with swapped parent roles (Algorithm 1, Line 6). The procedure of crossover is given in Algorithm 2.

Algorithm 2: Crossover(ϕ_A, ϕ_B)

```

1  $\mathcal{I} \leftarrow$  a set of  $\lfloor |K|/2 \rfloor$  indices sampled uniformly from  $K$ ;
2  $\mathcal{AS} \leftarrow \emptyset$ ; // record stations assigned to the offspring so far
3 for  $k \in \mathcal{I}$  do
4    $C_k \leftarrow C_k(\phi_A), m_k \leftarrow m_k(\phi_A), \mathcal{AS} \leftarrow \mathcal{AS} \cup C_k$ ;
5 end
6 for each  $i \in V \setminus \mathcal{AS}$  do
7    $k' \leftarrow$  cluster of  $i$  in  $\phi_B$ ;
8   if  $k' \notin \mathcal{I}$  then
9     Assign  $i$  to cluster  $k'$ ;
10  end
11  else
12    Assign  $i$  to cluster  $\arg \min_{k \in \mathcal{I}} D_{i,m_k}$ ;
13  end
14 end
15 Reseed any empty cluster from the largest cluster;
16 Recompute all medoids;
17 return  $(C_1, \dots, C_{|K|})$ 

```

Given parents ϕ_A and ϕ_B , the operator first samples $\lfloor |K|/2 \rfloor$ cluster indices and copies the corresponding clusters from ϕ_A into the offspring, preserving member

stations and medoids (Lines 1–5). Each remaining station $i \in V \setminus \mathcal{AS}$ is then allocated using ϕ_B : if ϕ_B places i in a cluster k' not among the inherited indices, i is assigned to k' . Otherwise, i is assigned to the inherited cluster whose medoid has the smallest dissimilarity D_{i,m_k} (Lines 6–13). Any cluster that remains empty after reassignment is reseeded from the largest cluster, and all medoids are recomputed (Lines 14–15). Figure 3.3 walks through a nine-station example with $|K| = 4$ that triggers both the direct copy and the fallback assignment.

Inheriting complete clusters preserves the intra-cluster groupings refined by the donating parent’s local search history, and the reassignment phase recombines inter-cluster boundaries.

Crossover operator diagram placeholder.

Figure 3.3: Illustration of the crossover operator on a nine-station example with $|K| = 4$. Parent ϕ_A partitions the stations into $C_1^A, \dots, C_4^A$, with medoids $\{2, 4, 6, 8\}$; parent ϕ_B partitions them into $C_1^B, \dots, C_4^B$. The sampled index set is $\mathcal{I} = \{1, 3\}$, so the offspring inherits C_1^A and C_3^A together with their medoids $m_1 = 2$ and $m_3 = 6$. Each remaining station is then allocated from ϕ_B : stations 5 and 9 lie in clusters whose index is outside $\mathcal{I}$ and are copied directly into C_2 and C_4 (Line 9 of Algorithm 2); stations 4 and 8 lie in clusters whose index coincides with an inherited one and therefore fall back to the inherited cluster whose medoid minimizes D_{i,m_k} (Line 11), under the assumption that $D_{4,2} < D_{4,6}$ and $D_{8,6} < D_{8,2}$.

3.4.5. Cluster size repair

Solution construction (Section 3.4.3) and crossover (Section 3.4.4) can both produce partitions in which one or more clusters violate the size bounds $N_{\min}$ and $N_{\max}$. A two-phase repair procedure restores feasibility before education (Algorithm 1, Line 8).

In Phase 1, the procedure corrects oversized clusters by repeatedly relocating the station most dissimilar to its cluster’s medoid to the nearest cluster whose size is below $N_{\max}$. In Phase 2, it corrects undersized clusters by pulling the station nearest (by dissimilarity) to the undersized cluster’s medoid from a cluster whose size exceeds $N_{\min}$. Medoids are recomputed after both phases.

3.4.6. Education

Every new individual undergoes local search education before entering the population (Algorithm 1, Line 9). The procedure applies six operators in a randomly shuffled order and returns as soon as any operator produces an improving move.

Two mechanisms guide candidate selection across all operators. The first is a complementary fit score that estimates a station's compatibility with a target cluster before committing to an ATS evaluation. For a station i and a target cluster C_k , we define the complementary fit score

$$FS(i, C_k) = \frac{1}{|\bar{C}_k(i)|} \sum_{j \in \bar{C}_k(i)} (1 - D_{ij}),$$

where $\bar{C}_k(i)$ denotes the set of stations in C_k whose surplus/deficit status is complementary to that of i : surplus stations are scored against deficit members, and vice versa. A higher value indicates stronger UDF complementarity between station i and the members of C_k . Balanced stations receive a score of zero and are excluded from candidate generation.

The second mechanism is a granular neighbourhood restriction adapted from Vidal (2022). For each station, a fixed number of nearest neighbours by dissimilarity D_{ij} are precomputed once before the algorithm begins. LS-1 and LS-2 restrict their search to clusters containing at least one of these nearest neighbours, reducing the set of target clusters from K to a small subset.

Within each operator, candidates are evaluated sequentially by computing the ATS objective for the affected clusters before and after the move, and the first candidate yielding an improvement is accepted. Cluster evaluations are cached by station membership, allowing tentatively rejected moves to still populate the cache for subsequent lookups. The six operators are described below.

LS-1 (Relocate station). Move a single station i from C_u to C_v , producing $C_{u'} = C_u \setminus \{i\}$ and $C_{v'} = C_v \cup \{i\}$. Granular search restricts candidates to clusters containing at least one of i 's nearest neighbours. A candidate is admitted when $FS(i, C_v) > FS(i, C_u)$, $|C_u| > N_{\min}$, and $|C_v| < N_{\max}$.

LS-2 (Swap stations). Exchange station $i \in C_u$ with station $j \in C_v$, producing $C_{u'} = (C_u \setminus \{i\}) \cup \{j\}$ and $C_{v'} = (C_v \setminus \{j\}) \cup \{i\}$. Granular search applies. A candidate is admitted when $FS(i, C_v) > FS(i, C_u)$ and $FS(j, C_u) > FS(j, C_v)$.

LS-3 (Merge and split). Select two clusters C_u and C_v at random, merge their stations into a single pool, and repartition via farthest-pair seeding followed by fit-score-based assignment. This operator restructures two clusters entirely, enabling large-scale partition changes that incremental operators cannot reach. The size bound also applies when admitting a candidate.

LS-4 (Relocate pair). Move station $i \in C_u$ together with its nearest cluster neighbour $j = \arg \min_{j' \in C_u \setminus \{i\}} D_{ij'}$ to C_v , producing $C_{u'} = C_u \setminus \{i, j\}$ and $C_{v'} = C_v \cup \{i, j\}$. A candidate is admitted when $FS(i, C_v) + FS(j, C_v) > FS(i, C_u) + FS(j, C_u)$, $|C_{u'}| - 2 \geq N_{\min}$, and $|C_{v'}| + 2 \leq N_{\max}$.

LS-5 (Swap pair for single). Exchange a pair (i, j) from C_v with a single station k from C_u , producing $C_{u'} = (C_u \setminus \{k\}) \cup \{i, j\}$ and $C_{v'} = (C_v \setminus \{i, j\}) \cup \{k\}$. A candidate is admitted when $FS(k, C_v) > FS(k, C_u)$ and $FS(i, C_u) + FS(j, C_u) > FS(i, C_v) + FS(j, C_v)$, with $|C_{u'}| + 1 \leq N_{\max}$ and $|C_{v'}| - 1 \geq N_{\min}$.

LS-6 (Swap pair for pair). Exchange pair (i_u, j_u) from C_u with pair (i_v, j_v) from C_v , producing $C_{u'} = (C_u \setminus \{i_u, j_u\}) \cup \{i_v, j_v\}$ and $C_{v'} = (C_v \setminus \{i_v, j_v\}) \cup \{i_u, j_u\}$. A candidate is admitted when $FS(i_u, C_v) + FS(j_u, C_v) > FS(i_u, C_u) + FS(j_u, C_u)$ and the symmetric condition holds for (i_v, j_v) .

When the fit-score step produces no candidate at all, each operator falls back to evaluating a single random move. This fallback prevents the search from stalling in configurations where the complementary fit criterion excludes all surplus-deficit pairings.

3.4.7. Population management

HGS-UC maintains a single population of feasible solutions whose size ranges between μ (minimum) and $\mu + \lambda$ (maximum), where μ is the base population size and λ is the number of offspring slots. Selection and survival decisions are driven by a biased fitness measure that balances solution quality and population diversity.

Each individual ϕ is assigned a fitness rank $r^{\text{fit}}(\phi)$, determined by objective value in descending order, and a diversity rank $r^{\text{div}}(\phi)$, determined by diversity

contribution in descending order. The biased fitness of individual ϕ is given by

$$BF(\phi) = r^{\text{fit}}(\phi) + \left(1 - \frac{n_{\text{elite}}}{\mu}\right) r^{\text{div}}(\phi),$$

where lower values are better.

The diversity contribution of an individual is the average distance to its n_{close} nearest neighbours in the population. For two solutions ϕ_a and ϕ_b , let $\chi_\phi(i, j) = 1$ if stations i and j belong to the same cluster in partition ϕ and 0 otherwise. The distance between ϕ_a and ϕ_b is the fraction of station pairs on which the two partitions disagree,

$$d(\phi_a, \phi_b) = \binom{|V|}{2}^{-1} \sum_{\{i,j\} \subseteq V} |\chi_{\phi_a}(i, j) - \chi_{\phi_b}(i, j)|,$$

Exact evaluation is $O(|V|^2)$ per solution pair, which is computationally expensive given the frequency of distance queries during selection and survival. The implementation uses a sampled estimator $\hat{d}(\phi_a, \phi_b)$ that draws N_{sample} station pairs uniformly at random and returns the proportion of broken pairs among them. Each draw is an independent Bernoulli trial with success probability $d(\phi_a, \phi_b)$. By Hoeffding's inequality (Hoeffding, 1963), we have

$$\Pr[|\hat{d} - d| \geq \epsilon] \leq 2e^{-2N_{\text{sample}}\epsilon^2} \quad \forall \epsilon > 0.$$

Consequently, with $N_{\text{sample}} = 500$ and $\epsilon = 0.1$ used in the implementation, the probability that $\hat{d}$ deviates from $d(\phi_a, \phi_b)$ by more than 0.1 is bounded by $2e^{-10} < 10^{-4}$. This precision is adequate for the ranking-based use of d in biased fitness.

Parents are selected by binary tournament on biased fitness. Two individuals are drawn at random, and the one with lower biased fitness is chosen. Each generation draws two parents independently. When offspring insertion brings the population size above $\mu + \lambda$, survivor selection removes individuals with the highest biased fitness until the size returns to μ . No explicit clone removal is applied.

The diversity component of biased fitness implicitly penalizes near-duplicate solutions, since individuals surrounded by similar neighbours receive low diversity contributions. Directed diversification is triggered after It_{div} consecutive generations without improvement (Algorithm 1, Lines 21–24). The $\max(1, \lfloor \mu/3 \rfloor)$ best individuals in the current population are retained by objective value. The remaining population slots are regenerated through the construction procedure of Section 3.4.3 until the population reaches 4μ individuals. The diversification counter $iter_{\text{div}}$ is then reset to zero. The global non-improvement counter $iter_{\text{NI}}$ is reset only if diversification yields a new best solution. Otherwise, it continues to accumulate.

3.5. COMPUTATIONAL EXPERIMENTS

The HGS-UC algorithm is implemented in single-threaded C++ and compiled with GCC. The clustering MILP, baseline clustering methods, and routing-phase BRP solver are implemented in Python using Gurobi 13.0.1 as the MIP solver. All computational experiments were carried out on a desktop PC equipped with a 13-th Gen Intel Core i9-13900 CPU (24 cores, 32 threads) and 64 GB of RAM. Every Gurobi solve, including the MV-BRP, the clustering MILP, and the per-cluster single-vehicle BRPs, is executed with a single thread to ensure reproducible and comparable timing results across methods. For decomposition-based methods (Gurobi-UC, HGS-UC, and all baseline clustering methods in Section 3.5.3), the per-cluster single-vehicle BRPs produced by the clustering phase are solved in parallel, each as an independent single-threaded Gurobi process. Accordingly, the reported CPU time of a decomposition-based method is the sum of (i) the clustering time and (ii) the wall-clock time of the longest single-cluster BRP solve within that run, reflecting the end-to-end time an operator would observe. For stochastic methods, we report the mean over 20 independent seeds. The HGS-UC clustering time is the mean over 20 seeds, and the per-run BRP solve time is the maximum across all clusters within that run, averaged over the 20 seeds.

All instances were generated from the New York Citi Bike system data. Citi Bike NYC November 2024 trip records² were aggregated into five-minute pick-up and drop-off rates for each station, and the queueing-theoretic model of Raviv and Kolka (2013) was applied to compute the UDF at every station. After filtering stations with incomplete or anomalous activity records, 1,000 stations were retained

²<https://s3.amazonaws.com/tripdata/index.html>

as the master pool. To examine how the CFRS framework scales with network size, instances of varying sizes $|V| \in \{50, 100, 200, 400, 800, 1000\}$ were constructed by sampling stations uniformly at random without replacement from the master pool. The duration matrices of the networks were calculated by the OSRM project³, which provides the duration of the fastest route between all pairs of supplied coordinates. The resulting matrices are asymmetric.

The number of vehicles is $|K| = \lfloor |V|/25 \rfloor$, yielding approximately 25 stations per cluster, which is in line with the per-vehicle workload in Schuijbroek et al. (2017). The cluster size bounds are set to $N_{\min} = 10$ and $N_{\max} = 40$, chosen so that the single-vehicle BRP within each cluster remains tractable for Gurobi at the routing phase. The default vehicle capacity is $Q = 20$ and the default repositioning time budget is $T = 120$ min, unless stated otherwise.

Three sets of experiments were conducted: parameter tuning of HGS-UC on a canonical instance, a comparison between the CFRS decomposition and the MV-BRP formulation solved directly by Gurobi on small-to-medium instances ($|V| \in \{50, 100\}$), and a comparison between HGS-UC and five baseline clustering methods on large-scale instances ($|V| \in \{200, 400, 800, 1000\}$) under varying time budgets and vehicle capacities.

3.5.1. Parameter tuning

The six key HGS-UC parameters were tuned sequentially on a canonical instance with $|V| = 200$ and $|K| = 8$. For each parameter, 20 independent seeds were run per candidate value. The selection criterion was the mean UDF reduction, with mean CPU time as a tiebreaker when the difference in UDF reduction was marginal. Table 3.3 defines the parameters and lists the selected values. Figure 3.4 shows the mean UDF reduction and CPU time for each candidate value.

Parameter tuning result plot placeholder.

Figure 3.4: Parameter tuning results ($|V| = 200$, $|K| = 8$, 20 seeds per candidate value). Solid lines (left axis): mean UDF reduction. Dashed lines (right axis): mean CPU time.

The maximum number of iterations without improvement, It_{NI} , exhibits a clear trade-off between solution quality and computation time. The mean UDF reduction

³<https://project-osrm.org/>

Table 3.3: HGS-UC parameter setting.

Parameter	Description	Value
It_{NI}	Maximum consecutive iterations without improvement	1000
μ	Minimum subpopulation size	10
λ	Number of offspring per generation	30
n_{elite}	Number of elite individuals	6
n_{close}	Number of close individuals for diversity measurement	5
α	Dissimilarity trade-off weight	0.5

increases from 268.06 at $It_{NI} = 100$ to 287.60 at $It_{NI} = 1000$, after which the objective declines slightly while CPU time continues to grow. The dissimilarity trade-off parameter α shows that a balanced setting ($\alpha = 0.5$) outperforms extreme values, confirming that both the travel-time proximity and UDF-improvement potential components contribute to clustering quality. The remaining four parameters ($\mu, \lambda, n_{elite}, n_{close}$) show relatively small differences across candidates. The selected values are listed in Table 3.3.

3.5.2. Three-way benchmark comparison

Three approaches were compared on instances with $|V| = 50$ ($|K| = 2$) and $|V| = 100$ ($|K| = 4$):

- (i) MV-BRP solved directly by Gurobi with a 2-hour time limit;
- (ii) Gurobi-UC, which solves the exact MILP clustering model (Section 3.2) at the clustering phase and then solves the per-cluster BRP at the routing phase; and
- (iii) HGS-UC, which replaces the exact clustering with the heuristic described in Section 3.4, followed by the same routing-phase solver.

Ten instances were generated for each network size. Both $T = 120$ min and $T = 180$ min were evaluated. HGS-UC was run 20 times per instance with independent seeds. Tables 3.4 and 3.5 present the results. A negative gap indicates that the decomposition method surpasses the MV-BRP incumbent.

Table 3.4: Three-way benchmark comparison results, $|V| = 50$ and $|K| = 2$.

T (min)	Instance	MV-BRP				Gurobi-UC			HGS-UC			
		UB	LB	Gap (%)	CPU (s)	Obj.	Gap* (%)	CPU (s)	Best	Avg.	Gap** (%)	CPU (s)
120	1	70.76	70.76	0.00	30.60	64.86	8.34	4.80	70.76	70.36	0.57	65.40
	2	66.55	66.55	0.00	608.50	65.88	1.01	2.30	66.55	66.42	0.20	84.50
	3	73.01	73.01	0.00	223.00	72.62	0.53	3.50	72.84	72.62	0.53	76.50
	4	52.61	52.61	0.00	117.90	52.11	0.95	3.20	52.26	52.11	0.95	85.70
	5	60.84	60.84	0.00	2489.40	59.57	2.09	7.00	60.82	60.50	0.56	24.80
	6	37.22	37.22	0.00	648.40	36.10	3.01	2.00	37.09	36.90	0.86	10.90
	7	57.05	57.05	0.00	355.20	56.15	1.58	2.20	56.88	56.74	0.54	8.80
	8	43.11	43.11	0.00	3075.70	42.30	1.88	3.60	43.11	43.05	0.14	11.30
	9	48.01	48.01	0.00	636.40	47.21	1.67	3.10	48.01	47.82	0.40	9.00
	10	49.13	49.13	0.00	199.40	48.56	1.16	2.90	49.08	48.82	0.63	9.00
180	Avg			0.00	838.45		2.22	3.46			0.54	38.59
	1	74.82	74.82	0.00	497.00	73.62	1.60	9.00	74.82	74.44	0.51	53.30
	2	69.17	69.17	0.00	862.10	68.20	1.40	3.70	68.89	68.79	0.55	131.60
	3	76.56	76.56	0.00	1313.90	75.82	0.97	7.20	76.28	75.97	0.77	58.80
	4	63.21	63.12	0.14	7200.00	60.67	3.88	3.20	63.12	63.03	0.14	63.90
	5	74.93	74.69	0.31	7200.00	73.39	1.74	9.20	74.60	74.33	0.48	18.10
	6	38.52	38.52	0.00	970.10	38.20	0.83	5.60	38.52	38.31	0.55	59.50
	7	60.49	60.49	0.00	2156.80	59.84	1.07	7.00	60.49	60.40	0.15	60.30
	8	46.76	46.66	0.21	7200.00	46.44	0.47	5.30	46.66	46.39	0.58	55.00
	9	51.41	51.41	0.00	3446.20	51.22	0.36	6.10	51.41	51.22	0.37	22.00
	10	50.91	50.91	0.00	852.10	50.51	0.78	11.40	50.76	50.56	0.69	35.40
Avg				0.07	3169.82		1.31	6.77			0.48	55.79

*Gap = (MV-BRP LB – Gurobi-UC Obj.) / MV-BRP LB $\times$ 100.

**Gap = (MV-BRP LB – HGS-UC Avg.) / MV-BRP LB $\times$ 100.

Table 3.5: Three-way benchmark comparison results, $|V| = 100$ and $|K| = 4$.

T (min)	Instance	MV-BRP				Gurobi-UC			HGS-UC			
		UB	LB	Gap (%)	CPU (s)	Obj.	Gap* (%)	CPU (s)	Best	Avg.	Gap** (%)	CPU (s)
120	1	91.11	86.24	5.35	7200.00	86.50	-0.30	1328.80	87.08	86.89	-0.75	36.70
	2	107.55	98.38	8.53	7200.00	96.54	1.87	2202.30	100.73	100.37	-2.02	53.90
	3	89.37	83.57	6.50	7200.00	82.82	0.90	7210.50	83.18	83.08	0.59	253.20
	4	137.02	122.08	10.90	7200.00	124.65	-2.10	7202.60	125.56	125.29	-2.63	51.30
	5	173.62	157.64	9.20	7200.00	152.99	2.95	7203.00	160.92	160.46	-1.79	42.50
	6	139.68	128.78	7.81	7200.00	127.91	0.67	7203.10	129.64	129.23	-0.35	86.30
	7	75.72	71.98	4.94	7200.00	71.28	0.97	7204.00	72.16	72.05	-0.10	74.80
	8	157.71	138.75	12.02	7200.00	140.18	-1.03	7206.90	141.65	141.33	-1.86	51.90
	9	131.62	127.57	3.07	7200.00	123.82	2.94	5198.60	127.62	127.13	0.34	38.40
	10	112.77	104.60	7.24	7200.00	101.88	2.61	7204.70	105.88	105.62	-0.98	52.40
180	Avg			7.56	7200.00		0.95	5916.45			-0.95	74.14
	1	92.20	90.34	2.02	7200.00	90.86	-0.58	7232.70	91.85	91.66	-1.46	264.10
	2	110.68	108.70	1.79	7200.00	108.64	0.06	7228.00	109.63	109.25	-0.51	77.00
	3	89.40	87.77	1.82	7200.00	88.06	-0.33	7222.30	89.32	88.95	-1.34	108.10
	4	143.41	136.73	4.66	7200.00	140.57	-2.81	7224.60	141.52	141.24	-3.30	158.80
	5	194.97	186.42	4.38	7200.00	182.76	1.96	7222.20	188.13	187.71	-0.69	228.90
	6	158.83	151.09	4.88	7200.00	150.13	0.63	7215.10	154.45	154.09	-1.99	126.10
	7	76.28	74.74	2.02	7200.00	75.08	-0.44	7232.40	76.28	75.87	-1.51	160.20
	8	179.13	170.69	4.71	7200.00	165.87	2.83	7229.00	170.93	170.77	-0.05	148.40
	9	135.84	131.91	2.89	7200.00	133.65	-1.31	7205.10	135.32	134.87	-2.24	68.30
	10	118.78	114.97	3.21	7200.00	116.00	-0.90	7231.00	117.53	117.20	-1.94	81.40
Avg				3.24	7200.00		-0.09	7224.24			-1.50	142.13

*Gap = (MV-BRP LB – Gurobi-UC Obj.) / MV-BRP LB $\times$ 100.

**Gap = (MV-BRP LB – HGS-UC Avg.) / MV-BRP LB $\times$ 100.

For the MV-BRP, UB and LB denote the upper bound (relaxation bound) and lower bound (best feasible solution) returned by Gurobi, respectively, and the gap is the Gurobi optimality gap $(UB - LB) / LB \times 100$. For the decomposition methods, Best and Avg. denote the maximum and mean UDF reduction across seeds (for the deterministic Gurobi-UC, Best and Avg. coincide with its single objective value). The gap is computed as $(MV\text{-}BRP\text{ LB} - \text{Method Obj.}) / MV\text{-}BRP\text{ LB} \times 100$, where Method Obj. is the Gurobi-UC objective or HGS-UC average, as indicated in the table footnotes. A negative value indicates that the method surpasses the MV-BRP incumbent.

At $|V| = 50$, the MV-BRP solved all instances to optimality within the 2-hour time limit at $T = 120$ min and most instances at $T = 180$ min (average Gurobi gap 0.07%). Gurobi-UC solved the clustering MILP within seconds and achieved an average gap of 2.22% at $T = 120$ min and 1.31% at $T = 180$ min. HGS-UC achieved an average gap of 0.54% and 0.48%, respectively, with the best seed matching the MV-BRP incumbent solution on several instances. At $|V| = 100$, the MV-BRP hit the 2-hour time limit on all 10 instances at both time budgets, with an average Gurobi gap of 7.56% at $T = 120$ min and 3.24% at $T = 180$ min. HGS-UC surpassed the MV-BRP incumbent on 8 out of 10 instances at $T = 120$ min (average gap -0.95%, average CPU time 74.1 s) and on all 10 instances at $T = 180$ min (average gap -1.50%, average CPU time 142.1 s). Gurobi-UC also produced competitive solutions at this scale (average gap 0.95% and -0.09% at the two time budgets), but its total CPU time approached or exceeded the MV-BRP time limit because the clustering MILP itself becomes intractable at $|V| = 100$. These results confirm that the CFRS decomposition is a viable strategy for the multi-vehicle BRP, and that HGS-UC provides both competitive solution quality and a substantial reduction in computation time.

3.5.3. Baseline comparison

For each of $|V| \in \{200, 400, 800, 1000\}$, one instance was generated and each stochastic method was run 20 times with independent seeds. Five baseline clustering methods were compared against HGS-UC:

- K-means: Multidimensional scaling is applied to the travel-time matrix, followed by standard K-means clustering in the resulting coordinate space.

- KNN: A k -nearest-neighbour graph is constructed from travel times and spectral clustering is applied.
- Schuijbroek et al. (2017): Stations are clustered to minimize the total service-level violation, accounting for inventory imbalance at each station.
- Forma et al. (2015): A savings-based heuristic partitions stations into vehicle districts by merging station pairs that share high UDF-reduction potential.
- Lv et al. (2020): A self-sufficiency-based clustering method that constructs clusters where internal supply and demand are balanced. The perturbation parameter θ was not specified in the original paper. We tested $\theta \in \{0, 5, 10, 20\}$.

All methods use the same routing-phase BRP solver and the same instance data. The cluster size is $|K| = \lfloor |V| / 25 \rfloor$ for all methods. Table 3.6 reports the average UDF reduction (Avg.), the best single run (Best), the CPU time, and the percentage gap relative to HGS-UC, defined as $(\text{Method Avg.} - \text{HGS-UC Avg.}) / \text{HGS-UC Avg.} \times 100$. A negative gap indicates that HGS-UC outperforms the baseline. The reported performance therefore reflects the cluster compositions produced by each method under a common routing benchmark.

Table 3.6: Baseline comparison results ($T = 120$ min, $Q = 20$, 20 seeds per stochastic method).

$ V $	$ K $	Method	Avg.	Best	CPU (s)	Gap* (%)	p-value**
200	8	HGS-UC	286.23	289.62	324.33	–	–
		Schuijbroek et al.	244.06	244.06	65.82	-14.73	1.90E-23
		Forma et al.	232.42	232.42	5.86	-18.80	1.89E-25
		Lv et al. ($\theta = 0$)	207.56	209.93	31.64	-27.48	1.12E-25
		Lv et al. ($\theta = 5$)	205.81	222.26	40.77	-28.09	1.40E-16
		Lv et al. ($\theta = 10$)	198.44	219.63	47.15	-30.67	7.33E-15
		Lv et al. ($\theta = 20$)	200.87	220.47	54.68	-29.82	2.33E-13
		K-means	212.81	215.53	11.25	-25.65	1.97E-24
		KNN	211.51	224.26	29.63	-26.10	1.36E-26
400	16	HGS-UC	494.29	508.01	773.81	–	–
		Schuijbroek et al.	406.77	406.77	89.71	-17.71	6.66E-14
		Forma et al.	381.82	381.82	19.73	-22.75	1.39E-24
		Lv et al. ($\theta = 0$)	293.26	333.00	46.46	-40.67	2.68E-19
		Lv et al. ($\theta = 5$)	307.72	356.14	64.54	-37.74	1.86E-18
		Lv et al. ($\theta = 10$)	301.83	339.41	72.67	-38.94	1.88E-18
		Lv et al. ($\theta = 20$)	297.06	333.04	87.92	-39.90	5.14E-21

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times 100$.

**p-values obtained from one-sided Welch's t-test.

$ V $	$ K $	Method	Avg.	Best	CPU (s)	Gap* (%)	p-value**
		K-means	315.75	354.08	14.05	-36.12	2.68E-18
		KNN	318.89	319.09	29.37	-35.48	2.23E-28
800	32	HGS-UC	1068.71	1085.21	2227.90	–	–
		Schuijbroek et al.	884.36	884.36	313.25	-17.25	1.62E-28
		Forma et al.	865.84	865.84	20.09	-18.98	5.51E-28
		Lv et al. ($\theta = 0$)	709.11	767.39	80.64	-33.65	1.11E-19
		Lv et al. ($\theta = 5$)	736.28	773.94	113.47	-31.10	9.70E-22
		Lv et al. ($\theta = 10$)	711.75	773.15	132.54	-33.40	1.34E-19
		Lv et al. ($\theta = 20$)	700.01	752.04	162.33	-34.50	1.19E-22
		K-means	697.73	733.97	27.93	-34.71	3.74E-23
		KNN	700.35	735.59	29.21	-34.47	4.01E-27
1000	40	HGS-UC	1429.16	1453.92	3228.15	–	–
		Schuijbroek et al.	1160.66	1160.66	554.44	-18.79	1.92E-26
		Forma et al.	961.75	961.75	27.19	-32.71	5.18E-31
		Lv et al. ($\theta = 0$)	893.02	954.43	98.18	-37.51	2.01E-24
		Lv et al. ($\theta = 5$)	913.83	986.05	135.78	-36.06	7.98E-23
		Lv et al. ($\theta = 10$)	893.76	961.88	159.03	-37.46	4.55E-26
		Lv et al. ($\theta = 20$)	890.52	952.22	201.56	-37.69	5.05E-24

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times$ 100.

**p-values obtained from one-sided Welch's t-test.

$ V $	$ K $	Method	Avg.	Best	CPU (s)	Gap* (%)	p-value**
		K-means	874.83	941.21	41.82	-38.79	5.80E-23
		KNN	847.97	873.94	28.62	-40.67	7.13E-31

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times$ 100.

**p-values obtained from one-sided Welch's t-test.

HGS-UC achieves the highest average UDF reduction at every instance size. The gap ranges from -14.73% (Schuijbroek et al., $|V| = 200$) to -40.67% (KNN, $|V| = 1000$). All differences are statistically significant (one-sided Welch’s t-test, $p < 10^{-12}$ in all cases). The HGS-UC clustering requires more CPU time than the baseline methods because it solves an optimization-based clustering problem rather than applying a closed-form partitioning rule. The total CPU time of the CFRS pipeline (clustering plus routing) is approximately 54 minutes, which fits within the overnight repositioning window.

Tables 3.7 and 3.8 report the gap of each baseline method relative to HGS-UC under varying time budgets ($T \in \{60, 120, 180, 240\}$ min with $Q = 20$) and vehicle capacities ($Q \in \{10, 20, 30, 40\}$ with $T = 120$ min), respectively. Methods whose clustering depends on T or Q (HGS-UC, Schuijbroek et al., and Lv et al. for T ; Schuijbroek et al. and Lv et al. for Q) were re-clustered at each parameter value. HGS-UC clustering is independent of Q and was not re-clustered for the capacity experiments.

Table 3.7: Baseline comparison results under varying time budgets ($Q = 20$, 20 seeds per stochastic method).

Method	$ V $	$T = 60$		$T = 120$		$T = 180$		$T = 240$	
		Gap* (%)	p-value**	Gap (%)	p-value	Gap (%)	p-value	Gap (%)	p-value
Schuijbroek et al.	200	-21.45	1.73E-24	-14.73	1.90E-23	-12.82	3.73E-30	-11.37	1.01E-42
	400	-18.44	4.48E-07	-17.71	6.66E-14	-14.82	1.11E-18	-13.19	1.69E-25
	800	-18.78	1.13E-19	-17.25	1.62E-28	-16.53	6.33E-27	-13.20	7.47E-31
	1000	-19.96	1.26E-18	-18.79	1.92E-26	-18.32	1.97E-30	-16.23	9.15E-39
Forma et al.	200	-31.16	1.46E-27	-18.80	1.89E-25	-7.50	9.68E-26	-2.27	1.98E-29
	400	-30.66	2.90E-24	-22.75	1.39E-24	-13.84	4.24E-32	-6.24	2.25E-29
	800	-19.16	7.71E-20	-18.98	5.51E-28	-11.60	5.25E-24	-5.00	7.65E-23
	1000	-34.25	2.07E-25	-32.71	5.18E-31	-23.27	2.11E-32	-14.33	9.81E-38
Lv et al. ($\theta = 0$)	200	-33.50	2.23E-24	-27.48	1.12E-25	-19.99	1.99E-24	-12.27	8.09E-20
	400	-45.74	2.19E-20	-40.67	2.68E-19	-31.28	1.40E-16	-21.73	6.86E-13
	800	-36.63	4.73E-20	-33.65	1.11E-19	-26.29	6.70E-17	-18.23	5.15E-15
	1000	-41.70	5.03E-24	-37.51	2.01E-24	-28.83	1.90E-20	-18.89	1.64E-16
Lv et al. ($\theta = 5$)	200	-34.47	9.78E-19	-28.09	1.40E-16	-18.98	8.04E-13	-9.66	3.24E-07
	400	-43.17	9.41E-20	-37.74	1.86E-18	-28.51	2.16E-15	-19.26	4.83E-12

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times 100$.

**p-values obtained from one-sided Welch's t-test.

Method	V	$T = 60$		$T = 120$		$T = 180$		$T = 240$	
		Gap* (%)	p-value**	Gap (%)	p-value	Gap (%)	p-value	Gap (%)	p-value
	800	-34.96	4.07E-21	-31.10	9.70E-22	-23.75	2.18E-20	-15.61	4.90E-16
	1000	-40.39	9.69E-24	-36.06	7.98E-23	-27.53	2.05E-20	-18.01	1.04E-16
Lv et al. ($\theta = 10$)	200	-35.87	4.55E-18	-30.67	7.33E-15	-22.37	5.74E-12	-13.89	2.59E-08
	400	-43.69	6.84E-22	-38.94	1.88E-18	-30.23	7.27E-16	-21.53	2.93E-12
	800	-37.52	1.20E-19	-33.40	1.34E-19	-26.02	2.91E-17	-17.33	3.58E-14
	1000	-41.87	2.65E-26	-37.46	4.55E-26	-28.86	1.48E-20	-18.89	2.58E-16
Lv et al. ($\theta = 20$)	200	-35.83	8.55E-17	-29.82	2.33E-13	-20.87	2.01E-09	-12.09	1.31E-05
	400	-45.89	1.13E-22	-39.90	5.14E-21	-29.81	2.77E-17	-19.96	6.83E-13
	800	-38.41	1.04E-20	-34.50	1.19E-22	-26.75	1.47E-22	-17.69	2.87E-18
	1000	-42.07	2.81E-24	-37.69	5.05E-24	-29.07	1.64E-19	-18.99	3.69E-15
K-means	200	-33.61	1.04E-26	-25.65	1.97E-24	-14.37	3.27E-23	-4.24	3.00E-22
	400	-42.77	6.01E-21	-36.12	2.68E-18	-27.32	3.98E-16	-18.26	3.48E-13
	800	-38.19	1.31E-21	-34.71	3.74E-23	-27.56	4.00E-21	-20.02	3.80E-19
	1000	-42.48	2.08E-21	-38.79	5.80E-23	-30.95	7.38E-21	-21.90	5.53E-17
KNN	200	-36.56	3.87E-28	-26.10	1.36E-26	-14.10	1.19E-24	-4.18	5.40E-22
	400	-40.60	1.37E-26	-35.48	2.23E-28	-28.29	3.32E-38	-20.72	2.61E-39

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times$ 100.

**p-values obtained from one-sided Welch's t-test.

Method	V	$T = 60$		$T = 120$		$T = 180$		$T = 240$	
		Gap* (%)	p-value**	Gap (%)	p-value	Gap (%)	p-value	Gap (%)	p-value
	800	-39.91	1.59E-23	-34.47	4.01E-27	-26.21	5.75E-24	-17.67	2.57E-21
	1000	-45.05	6.91E-26	-40.67	7.13E-31	-33.07	2.91E-26	-24.71	1.17E-24

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times$ 100.

**p-values obtained from one-sided Welch's t-test.

Table 3.8: Baseline comparison results under varying vehicle capacities ($T = 120$ min, 20 seeds per stochastic method).

Method	$ V $	$Q = 10$		$Q = 20$		$Q = 30$		$Q = 40$	
		Gap* (%)	p-value**	Gap (%)	p-value	Gap (%)	p-value	Gap (%)	p-value
Schuijbroek et al.	200	-9.60	6.28E-17	-14.73	1.90E-23	-17.31	1.82E-17	-15.02	7.59E-21
	400	-14.09	3.32E-24	-17.71	6.66E-14	-15.44	1.41E-20	-8.73	1.54E-16
	800	-22.40	2.06E-27	-17.25	1.62E-28	-17.65	7.29E-24	-16.88	2.94E-22
	1000	-18.38	5.04E-26	-18.79	1.92E-26	-12.85	2.04E-21	-14.23	5.69E-21
Forma et al.	200	-13.82	6.95E-20	-18.80	1.89E-25	-20.22	1.62E-24	-20.29	2.61E-23
	400	-19.80	5.25E-27	-22.75	1.39E-24	-23.22	6.35E-24	-23.32	1.48E-24
	800	-18.68	3.19E-26	-18.98	5.51E-28	-18.54	2.86E-24	-18.74	4.07E-23
	1000	-30.80	2.81E-30	-32.71	5.18E-31	-33.25	3.12E-29	-32.75	7.98E-28
Lv et al. ($\theta = 0$)	200	-13.34	3.45E-15	-27.48	1.12E-25	-30.62	6.74E-15	-29.73	5.09E-16
	400	-35.97	3.70E-20	-40.67	2.68E-19	-33.91	2.11E-16	-25.91	3.34E-17
	800	-28.36	2.25E-18	-33.65	1.11E-19	-30.53	1.99E-21	-25.61	1.63E-15
	1000	-33.82	3.04E-20	-37.51	2.01E-24	-34.52	7.18E-22	-25.28	5.23E-17
Lv et al. ($\theta = 5$)	200	-19.21	1.38E-10	-28.09	1.40E-16	-32.15	6.60E-15	-26.24	1.49E-12
	400	-35.49	2.10E-20	-37.74	1.86E-18	-34.54	7.84E-18	-25.80	3.59E-15

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times 100$.

**p-values obtained from one-sided Welch's t-test.

Method	V	$Q = 10$		$Q = 20$		$Q = 30$		$Q = 40$	
		Gap* (%)	p-value**	Gap (%)	p-value	Gap (%)	p-value	Gap (%)	p-value
	800	-26.82	1.07E-18	-31.10	9.70E-22	-27.59	4.97E-22	-24.37	1.20E-16
	1000	-32.81	3.75E-20	-36.06	7.98E-23	-33.36	1.54E-20	-23.18	2.29E-15
Lv et al. ($\theta = 10$)	200	-17.43	8.83E-12	-30.67	7.33E-15	-32.91	2.62E-15	-26.30	3.17E-11
	400	-34.73	9.00E-18	-38.94	1.88E-18	-35.12	4.02E-18	-25.31	1.78E-16
	800	-26.96	2.06E-20	-33.40	1.34E-19	-29.87	1.20E-18	-24.23	1.87E-17
	1000	-33.85	9.47E-21	-37.46	4.55E-26	-32.31	1.29E-21	-21.56	3.70E-16
Lv et al. ($\theta = 20$)	200	-16.00	4.65E-10	-29.82	2.33E-13	-31.86	5.84E-18	-30.41	9.94E-14
	400	-37.56	7.26E-18	-39.90	5.14E-21	-35.19	2.03E-17	-24.66	2.29E-13
	800	-28.42	5.34E-17	-34.50	1.19E-22	-30.11	2.78E-21	-20.30	9.81E-16
	1000	-33.43	3.15E-20	-37.69	5.05E-24	-33.78	7.85E-22	-23.34	5.93E-19
K-means	200	-13.02	3.01E-16	-25.65	1.97E-24	-28.16	8.81E-26	-29.16	1.37E-25
	400	-28.21	1.53E-16	-36.12	2.68E-18	-37.56	1.35E-18	-37.78	6.47E-19
	800	-27.16	1.25E-20	-34.71	3.74E-23	-36.14	3.13E-23	-36.66	2.63E-23
	1000	-29.79	4.96E-22	-38.79	5.80E-23	-40.46	6.75E-23	-40.95	8.24E-23
KNN	200	-13.93	1.44E-19	-26.10	1.36E-26	-28.62	2.55E-27	-29.60	4.78E-27
	400	-28.77	2.71E-30	-35.48	2.23E-28	-36.76	8.12E-28	-36.92	1.87E-28

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times$ 100.

**p-values obtained from one-sided Welch's t-test.

Method	V	$Q = 10$		$Q = 20$		$Q = 30$		$Q = 40$	
		Gap* (%)	p-value**	Gap (%)	p-value	Gap (%)	p-value	Gap (%)	p-value
	800	-26.53	2.71E-24	-34.47	4.01E-27	-35.82	1.18E-25	-36.40	3.88E-25
	1000	-31.87	2.66E-27	-40.67	7.13E-31	-42.29	1.23E-30	-42.74	1.70E-30

*Gap = (Method Avg. – HGS-UC Avg.) / HGS-UC Avg. $\times$ 100.

**p-values obtained from one-sided Welch's t-test.

HGS-UC outperforms all baselines at every time-budget level, every capacity level, and every instance size. All gaps are statistically significant ($p < 10^{-4}$).

3.6. CONCLUSIONS

This chapter addresses the multi-vehicle static BRP with UDF as the service measure and proposes a CFRS framework in which the clustering phase is formulated as an MILP that jointly captures UDF reduction and vehicle routing time feasibility. To the best of our knowledge, no prior CFRS framework for the BRP formulates clustering as an optimization model that simultaneously evaluates UDF reduction at station-level inventories and enforces a per-cluster routing time budget. Because this MILP is intractable for city-scale instances, we develop HGS-UC, a hybrid genetic search tailored to the clustering model. HGS-UC represents a solution as a partition of the station set, constructs initial solutions by K-medoids clustering on a dissimilarity measure that combines travel-time proximity and bilateral UDF reduction potential, and refines them through a local search with six operators guided by a complementary fit score and a granular neighborhood restriction. Solution evaluation uses the ATS procedure, which estimates a time-feasible transfer set for each cluster by selecting surplus-deficit transfers in decreasing order of UDF reduction per unit handling time. ATS replaces an exact single-vehicle BRP solve that would be prohibitively expensive at every evaluation, making population-based search tractable on city-scale instances.

Computational experiments on instances generated from New York Citi Bike data demonstrate the framework’s performance across three instance scales. First, on the 50-station instances, HGS-UC obtains solutions whose mean is within 0.54% of the MV-BRP incumbent in under a minute of CPU time, with the best seed matching the incumbent on several instances. Second, on the 100-station instances, where Gurobi reaches the two-hour limit on every instance with average optimality gaps of 7.56% and 3.24% at the 120-minute and 180-minute budgets, respectively, HGS-UC’s mean solution exceeds the Gurobi incumbent on 8 of 10 instances at the 120-minute repositioning budget and on all 10 instances at the 180-minute budget, in 1 to 3 minutes of CPU time. Third, on instances of 200 to 1,000 stations — a scale not previously reported in the CFRS literature for the BRP — HGS-UC delivers higher UDF reduction than five baseline clustering methods, namely the methods of Schuijbroek et al. (2017), Forma et al. (2015), and Lv et al. (2020), together with

K-means and KNN, at every combination of network size, time budget, and vehicle capacity tested, with all gaps statistically significant ($p < 10^{-4}$). At the default time budget and vehicle capacity, the average UDF reduction gains range from 14.73% to 40.67%, with consistent improvements across variations in both parameters. On the largest 1,000-station instances, the complete CFRS pipeline (clustering plus per-cluster routing) finishes in under one hour, well within the overnight repositioning window.

CHAPTER 4

STATIC BIKE REPOSITIONING WITH BROKEN BIKES AND ON-SITE REPAIR

This chapter studies a static bike repositioning problem with broken bikes and on-site repair. In addition to relocating usable and broken bikes by trucks, the problem considers repairers who visit stations and repair broken bikes directly in the system. The resulting planning problem involves two interacting routing decisions and station-level quantity decisions for repairing, loading, and unloading bikes.

The chapter first presents the mathematical formulation of the problem. It then introduces a hybrid solution algorithm that combines Hybrid Genetic Search with Adaptive Diversity Control and a station-budget-constrained heuristic for determining repairing, loading, and unloading quantities. Computational experiments are conducted to evaluate the performance of the proposed method and to examine the operational value of introducing on-site repairers.

4.1. PROBLEM FORMULATION

In this chapter, we consider a BSS with one depot and a finite number of stations. Every station has a fixed capacity, an initial inventory of usable bikes, and an initial inventory of broken bikes. The numbers of usable bikes and empty docks at each station are assumed to be known as the current technology of BSSs provides the system operator with this information in real-time. Moreover, the number of broken bikes at each station is assumed to be known in this study, as we assume that *users report the existence of broken bikes once they know in the daytime*, and we only consider a nighttime bike repositioning problem.

A fleet of homogeneous repositioning trucks with the same capacity depart from the depot, then travel among selected stations in the system to deliver usable bikes to visited stations or pick up the usable bikes from visited stations, collect broken bikes at visited stations, and finally return to the depot to drop off usable and broken bikes within a given time budget. Throughout this process, the trucks can return to the depot to drop off usable and broken bikes when needed, but the number of bikes carried on each truck cannot exceed the truck capacity. Meanwhile, a set of repairers also depart from the depot, head to stations with broken bikes to conduct repairs directly on-site, and return to the depot within the same time budget. Loading, unloading, and repairing each bike take time, with a constant duration for

each operation type. For each repositioning truck, every station and the depot can be visited multiple times. For each repairer, each station can be visited at most once. A station can be visited by at most one repairer. We assume the depot holds an infinite inventory of usable bikes and can store an infinite number of usable and broken bikes. Moreover, we assume that the repairers only repair broken bikes at stations but do not repair broken bikes at the depot. Broken bikes at the depot are repaired by the repairing staff there but these bikes have no urgency to be repaired within the repositioning period as we assume we have a sufficient number of usable bikes in the system. Furthermore, we assume that the repairing staff at the depot and the onsite repairers are hired at fixed daily salary rates. Therefore, the total repair staffing cost is constant daily, which is independent of the number of broken bikes repaired onsite or sent back to the depot within the repositioning period. In addition, we assume that the repositioning process involves no rental or returning behaviors from the users, and the final inventories after the repositioning set the starting inventories for the following day. Additionally, we assume the repairers travel on electric tricycles that generate no emissions during the repositioning process, and the travel speed of the electric tricycle is proportional to that of a truck. The objective of the BRP is to minimize the cost consisting of the following components:

- The sum of user dissatisfaction at all stations on the following day and;
- The CO₂ emissions of the repositioning trucks during the rebalancing process.

Note that the daily repair staffing cost is not included in our optimization objective as it does not affect the optimal repositioning decisions.

Figure 4.1 illustrates an example of the problem. The BSS consists of one depot (represented by a red star and labeled with 0) and ten stations (labeled from 1 to 10). The tables in Figure 4.1 (a)-(d) show the number of usable and broken bikes loaded onto a truck and unloaded from a truck at each node, and the number of bikes repaired by repairers at each node. Two trucks (Truck 1 in Figure 4.1 (a) and Truck 2 in Figure 4.1 (b)) with the identical capacity of 25 bikes and two repairers (Repairer 1 in Figure 4.1 (c) and Repairer 2 in Figure 4.1 (d)) depart from the depot to visit these stations. Specifically, Truck 1 follows the route 0-4-6-9-0-2-0, with one intermediate return to the depot to drop off 18 broken bikes and pick up three usable bikes. Truck 2 follows the route 0-5-4-0-7-5-9-0, with one intermediate return to the depot to drop off eight usable bikes and 13 broken bikes. Repairer 1 follows

the route 0–6–8–0, performing on-site repairs before returning to the depot, while Repairer 2 follows the route 0–1–3–0. Node 5 is visited by Truck 2 twice because the total number of broken and surplus usable bikes to be loaded at the station (26) is more than the truck capacity (25). To reduce vehicle emissions, it is wise to drop off all broken bikes when Truck 2 visits the depot during the repositioning process. The travel times between nodes are labeled on the arcs in the figures, and the travel times between nodes are not assumed to be symmetric (e.g., the back-and-forth travel times for Truck 1 between nodes 0 to 2 are different). Note that Station 10 is not visited by both trucks and repairers.

Problem illustration placeholder.

Figure 4.1: Problem illustration

Figure 4.2 shows the repositioning and repair process of Truck 1 and Repairer 1 from the example in Figure 4.1 from a temporal perspective. The repositioning time budget is 7200 s as marked on the time axis. The figure shows the routes taken, inventory variations at the stations, and temporal aspects of the operations (e.g., loading and repairing times, arrival times). Loading or unloading one bike takes 60 s while repairing one broken bike takes 300 s. Note that $(p, b) \rightarrow (p', b')$ at each station represents the variations of usable bike inventory from p to p' and broken bike inventory from b to b' after the operations of trucks or repairers. $\langle A, B \rangle$ on the axis represents the truck inventory of A usable bikes and B broken bikes when it travels between the nodes. The round-cornered shapes show the variation of the user dissatisfaction under the inventories before and after the operations. Specifically, Truck 1 follows the route 0–4–6–9–0–2–0 as indicated above the time axis. It first visits Station 4 to collect two usable bikes and one broken bike, then travels to Station 6 to collect five usable and 17 broken bikes, reaching its full capacity of 25 bikes. Next, the truck travels to Station 9 to unload the seven usable bikes. Following that, it returns to the depot to unload all broken bikes, loads three usable bikes, and proceeds to Station 2 to deliver the three usable bikes and collect nine broken bikes. Finally, it returns to the depot, unloading all broken bikes and completing its operations in 7123 s, calculated by adding up the arrival time at the depot (6583 s) and the operational time for unloading the nine broken bikes (540 s). Meanwhile, Repairer 1 performs on-site repairs along the route 0–6–8–0, repairing four broken bikes at Station 6 and 12 at Station 8. It concludes its operations in 7180 s. Note that Station 6 is visited by both Truck 1 and Repairer 1, and all 21 broken bikes at the

station are handled, with four repaired by the repairer before the truck arrives at the station and 17 loaded by the truck. The four repaired bikes, together with one usable bike originally at Station 6 and two usable bikes at Station 4 are used to partially address the deficit of bikes at Station 9. The remaining deficit of eight usable bikes at Station 9 is satisfied by the usable bikes collected by Truck 2 from other stations and delivered to the station.

Truck and repairer timeline placeholder.

Figure 4.2: The repositioning and repair process of Truck 1 and Repairer 1.

The first component of the objective function of the problem involves the calculation of user dissatisfaction throughout the day following the rebalancing process based on the starting inventories resulting from that process. The method proposed by Kaspi et al. (2017) is used for this calculation, which is briefly introduced here. Given station i with C_i docks and a finite period for the following day, at the beginning of that day, there are p_i usable bikes and b_i broken bikes present at the station. Throughout this timeframe, users intending to rent or return a bike arrive at the station based on a stochastic process. When a user's demand for a bike or an available dock is met, the bike is rented or returned, subsequently updating the station's inventory level. Conversely, if the demand cannot be fulfilled, it is assumed that the user promptly leaves the station, corresponding to a failure in renting or returning a bike. The user dissatisfaction at a station is calculated as the weighted expected number of occurrences where users are unable to rent or return bikes during the specified period, provided that the usable and broken bike inventories at the beginning of the period are p_i and b_i , respectively. It is assumed that changes in inventory levels are solely caused by users' rental or return actions, with no external interventions such as truck repositioning or repairs. An Extended User Dissatisfaction Function (EUDF), denoted as $F_i(p_i, b_i)$, is defined to represent the dissatisfaction at station i given the inventories p_i and b_i . The domain of this bivariate function is $\Theta_i = \{(p_i, b_i) \mid p_i \geq 0, b_i \geq 0, p_i + b_i \leq C_i\}$. Kaspi et al. (2017) proposed a method for determining a convex polyhedral function that has nearly identical values to the EUDF values in its range, enabling the integration of the EUDF into the subsequent linear optimization model. Specifically, for each $(p_i, b_i) \in \Theta_i$, a plane $\alpha_{p_i, b_i}^i \cdot p_i + \beta_{p_i, b_i}^i \cdot b_i + \gamma_{p_i, b_i}^i$ that passes as close as possible to $F_i(p_i, b_i)$ can be fitted, where $\alpha_{p_i, b_i}^i, \beta_{p_i, b_i}^i$, and γ_{p_i, b_i}^i are the coefficients of the plane that can be computed for every $(p_i, b_i) \in \Theta_i$. The computation details of the

coefficients of the fitted plane are given in Appendix A. Due to limited space, we omit the lengthy details on generating the station EUDF values. Interested readers can consult Kaspi et al. (2017) for generating them.

The second component requires the calculation of the CO₂ emissions of a repositioning truck as it moves. Wang and Szeto (2021) used the following emission model to calculate the amount of CO₂ emissions e produced by a truck when it carries bikes over a distance:

$$e = CER \cdot \left(\rho_0 + \frac{\rho^* - \rho_0}{Q} \psi \right) \cdot d \quad (4.1)$$

where CER is the CO₂ emission rate, ρ^* and ρ_0 are the fuel consumption rates when the vehicle is empty and fully loaded, respectively, Q is the vehicle capacity, ψ is the number of bikes carried by the truck, and d is the distance traveled by the truck. Given the similarities between their and our research context, we chose to adopt this emission model for the calculation in the second component.

As aforementioned, each station can be visited by each truck an arbitrary number of times. Hence, the classic arc-indexed formulation is not suitable for our problem. As proposed by Raviv et al. (2013), the time-indexed formulation requires the whole time budget T to be discretized into multiple periods with equal length, which allows us to keep track of each visit to a station and the inventory of bikes on each truck and at each station during every period, thereby enabling the formulation (1) for multiple visits to a station and the depot by vehicles and repairers and (2) to decide the pickup quantities of usable bikes by trucks and the repair quantities of broken bikes by repairs based on the actual inventory status. Therefore, we formulate our problem based on the time-indexed formulation.

4.1.1. Notations

Table 4.1: Notation used in the problem for

Sets	
S	The set of stations, indexed by $i = 1, \dots, S $
S_0	The set of nodes, including the stations and the depot (denoted by 0)
K	The set of rebalancing trucks
R	The set of repairers

Parameters

p_i^0	The initial inventory of usable bikes at node $i \in S_0$
b_i^0	The initial quantity of broken bikes at node $i \in S_0$
ρ_0	The empty-load fuel consumption rate of each truck (liters/km)
ρ^*	The full-load fuel consumption rate of each truck (liters/km)
CER	The CO ₂ emission rate (kg/liter)
Q_k	The capacity of truck k
C_i	The capacity of node $i \in S_0$
τ	The length of a discretized period (second)
t^{load}	Time for loading/unloading a bike ($t^{\text{load}} < \tau$)
t^{repair}	Time for repairing a bike on-site ($t^{\text{repair}} < \tau$)
T	Rebalancing time budget (second)
t_{ij}^v	Travel time for a truck from node i to node j
t_{ij}^r	Travel time for a repairer from node i to node j
t'_{ij}	The number of periods required by a truck to travel from node i to node j , which is calculated as $t'_{ij} = \lceil t_{ij}^v / \tau \rceil$
t''_{ij}	The number of periods required by a repairer to travel from node i to node j , which is calculated as $t''_{ij} = \lceil t_{ij}^r / \tau \rceil$
d_{ij}	The travel distance from node i to node j (km)
α_{p_i, b_i}^i	The first coefficient of the fitted plane to approximate the EUDF at station i , given the usable and broken bikes at station i
β_{p_i, b_i}^i	The second coefficient of the fitted plane to approximate the EUDF at station i , given the usable and broken bikes at station i
γ_{p_i, b_i}^i	The third coefficient of the fitted plane to approximate the EUDF at station i , given the usable and broken bikes at station i
T'	The number of periods in the rebalancing time budget such that $T' = \lfloor T / \tau \rfloor$
c^p	The penalty cost for a user unable to rent/return a bike (CNY)
c^e	The unit carbon trading cost (CNY/kg)

Decision variables

$x_{i,j,t,k}^v$	If truck k travels directly from node i to node j during period t , $x_{i,j,t,k}^v = 1$, 0 otherwise
$x_{i,j,t,m}^r$	If repairer m travels directly from node i to node j during period t , $x_{i,j,t,m}^r = 1$, 0 otherwise
$p_{i,t}$	The number of usable bikes at node i at the end of period t
$b_{i,t}$	The number of broken bikes at node i at the end of period t
$y_{i,t,k}^{u+}$	The number of usable bikes delivered to node i by truck k during period t
$y_{i,t,k}^{u-}$	The number of usable bikes collected from node i by truck k during period t
$y_{i,t,k}^{b+}$	The number of broken bikes delivered to node i by truck k during period t
$y_{i,t,k}^{b-}$	The number of broken bikes collected from node i by truck k during period t

$g_{i,t,m}$	The number of broken bikes repaired by repairer m at node i during period t
$\psi_{i,j,t,k}^u$	The number of usable bikes on truck k when it departs from node i to node j during period t
$\psi_{i,j,t,k}^b$	The number of broken bikes on truck k when it departs from node i to node j during period t
$e_{i,j,t,k}$	The CO ₂ emissions generated when truck k travels from node i to node j during period t
∂_k^v	The period in which truck k returns to the depot and remains stationary until the rebalancing process
∂_m^r	The period in which repairer m returns to the depot and remains stationary until the rebalancing process
$\varsigma_{t,k}^v$	If truck k remains stationary at the depot from period t to the end of the repositioning process
$\varsigma_{t,m}^r$	If repairer m remains stationary at the depot from period t to the end of the repositioning process
$\widehat{F}_i$	The auxiliary variable for the linearization of the EUDF of station i
Function	
$F_i(p_i, b_i)$	The EUDF defined over the usable bike inventory level p_i and broken bike inventory level b_i

4.1.2. Time-indexed formulation

The time-indexed formulation of the BRP separating the routes of trucks and repairers is as follows.

$$\begin{aligned}
& \text{minimize } c^p \sum_{i \in S} F_i(p_{i,T'}, b_{i,T'}) + c^e \sum_{i \in S_0} \sum_{j \in S_0} \sum_{t=t'_{i,j}+1}^{T'} \sum_{k \in K} e_{i,j,t-t'_{i,j},k} + \\
& \theta \cdot \left(\sum_{i \in S_0} \sum_{j \in S_0} \sum_{t=t'_{i,j}+1}^{T'} \sum_{k \in K} \left(t_{i,j}^v \cdot x_{i,j,t-t'_{i,j},k}^v \right) + \sum_{i \in S_0} \sum_{t=t'_{i,j}+1}^{T'} \sum_{k \in K} t^{\text{load}} \cdot \left(y_{i,t,k}^{u+} + y_{i,t,k}^{u-} + y_{i,t,k}^{b+} + y_{i,t,k}^{b-} \right) \right. \\
& \left. \sum_{i \in S_0} \sum_{j \in S_0} \sum_{t=t''_{i,j}+1}^{T'} \sum_{m \in R} \left(t_{i,j}^r \cdot x_{i,j,t-t''_{i,j},m}^r \right) + \sum_{i \in S_0} \sum_{t=t''_{i,j}+1}^{T'} \sum_{m \in R} t^{\text{repair}} \cdot g_{i,t,m} \right)
\end{aligned} \tag{4.2}$$

Subject to

$$p_{i,1} = p_i^0 - \sum_{k \in K} y_{i,1,k}^{u-} + \sum_{k \in K} y_{i,1,k}^{u+} + \sum_{m \in R} g_{i,1,m}, \forall i \in S_0, \tag{4.3}$$

$$p_{i,t} = p_{i,t-1} - \sum_{k \in K} y_{i,t,k}^{u-} + \sum_{k \in K} y_{i,t,k}^{u+} + \sum_{m \in R} g_{i,t,m}, \forall i \in S_0, t = 2, \dots, T', \tag{4.4}$$

$$b_{i,1} = b_i^0 - \sum_{k \in K} y_{i,1,k}^{b^-} + \sum_{k \in K} y_{i,1,k}^{b^+} - \sum_{m \in R} g_{i,1,m}, \forall i \in S_0, \quad (4.5)$$

$$b_{i,t} = b_{i,t-1} - \sum_{k \in K} y_{i,t,k}^{b^-} + \sum_{k \in K} y_{i,t,k}^{b^+} - \sum_{m \in R} g_{i,t,m}, \forall i \in S_0, t = 2, \dots, T', \quad (4.6)$$

$$p_{i,t} + b_{i,t} \leq C_i, \forall i \in S, t = 1, \dots, T', \quad (4.7)$$

$$\sum_{j \in S_0} \psi_{0,j,1,k}^u = y_{0,1,k}^{u^-}, \forall k \in K, \quad (4.8)$$

$$\sum_{j \in S_0} \psi_{i,j,t,k}^u = \sum_{j \in S_0, t-t'_{j,i} > 0} \psi_{j,i,t-t'_{j,i},k}^u - y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-}, \forall i \in S_0, t = 2, \dots, T', k \in K, \quad (4.9)$$

$$\psi_{i,j,T',k}^u = 0, \forall i \in S_0, j \in S_0, k \in K, \quad (4.10)$$

$$\psi_{i,j,t,k}^u \leq Q_k x_{i,j,t,k}^v, \forall i \in S_0, j \in S_0, t = 1, \dots, T', k \in K, \quad (4.11)$$

$$\psi_{i,j,1,k}^b = 0, \forall i \in S_0, j \in S_0, k \in K, \quad (4.12)$$

$$\sum_{j \in S_0} \psi_{i,j,t,k}^b = \sum_{j \in S_0, t-t'_{j,i} > 0} \psi_{j,i,t-t'_{j,i},k}^b - y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-}, \forall i \in S_0, t = 2, \dots, T', k \in K, \quad (4.13)$$

$$\psi_{i,j,T',k}^b = 0, \forall i \in S_0, j \in S_0, k \in K, \quad (4.14)$$

$$\psi_{i,j,t,k}^b \leq Q_k x_{i,j,t,k}^v, \forall i \in S_0, j \in S_0, t = 1, \dots, T', k \in K, \quad (4.15)$$

$$\psi_{i,j,t,k}^u + \psi_{i,j,t,k}^b \leq Q_k, \forall i \in S_0, j \in S_0, t = 1, \dots, T', k \in K, \quad (4.16)$$

$$y_{i,t,k}^{u+} \leq \min \{C_i, Q_k\} \cdot \sum_{j \in S_0} x_{i,j,t,k}^v, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.17)$$

$$y_{i,t,k}^{u-} \leq \min \{C_i, Q_k\} \cdot \sum_{j \in S_0} x_{i,j,t,k}^v, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.18)$$

$$y_{0,t,k}^{b+} \leq Q_k \cdot \sum_{j \in S_0} x_{0,j,t,k}^v, \forall t = 1, \dots, T', k \in K, \quad (4.19)$$

$$y_{i,t,k}^{b+} = 0, \forall i \in S, t = 1, \dots, T', k \in K, \quad (4.20)$$

$$y_{0,t,k}^{b-} = 0, \forall t = 1, \dots, T', k \in K, \quad (4.21)$$

$$y_{i,t,k}^{b-} \leq \min \{b_i^0, Q_k\} \cdot \sum_{j \in S_0} x_{i,j,t,k}^v, \forall i \in S, t = 1, \dots, T', k \in K, \quad (4.22)$$

$$g_{0,t,m} = 0, \forall t = 1, \dots, T', m \in R, \quad (4.23)$$

$$g_{i,t,m} \leq b_i^0 \cdot \sum_{j \in S_0} x_{i,j,t,m}^r, \forall i \in S, t = 1, \dots, T', m \in R, \quad (4.24)$$

$$\sum_{t=1}^{T'} \sum_{k \in K} y_{i,t,k}^{b-} + \sum_{t=1}^{T'} \sum_{m \in R} g_{i,t,m} \leq b_i^0, \forall i \in S, \quad (4.25)$$

$$\sum_{j \in S_0} x_{0,j,1,k}^v = 1, \forall k \in K, \quad (4.26)$$

$$\sum_{j \in S} x_{0,j,1,m}^r = 1, \forall m \in R, \quad (4.27)$$

$$\sum_{i \in S_0} x_{i,0,T'-t'_{i,0},k}^v = 1, \forall k \in K, \quad (4.28)$$

$$\sum_{i \in S_0} x_{i,0,T'-t''_{i,0},m}^r = 1, \forall m \in R, \quad (4.29)$$

$$\sum_{j \in S_0, t-t'_{ji} > 0} x_{j,i,t-t'_{ji},k}^v = \sum_{u \in S_0} x_{i,u,t,k}^v, \forall i \in S_0, t = 2, \dots, T', k \in K, \quad (4.30)$$

$$\sum_{j \in S_0, t-t''_{ji} > 0} x_{j,i,t-t''_{ji},m}^r = \sum_{u \in S_0} x_{i,u,t,m}^r, \forall i \in S_0, t = 2, \dots, T', m \in R, \quad (4.31)$$

$$s_{t,k}^v \geq (t - \partial_k^v + 1) / T', \forall t = 1, \dots, T', k \in K, \quad (4.32)$$

$$s_{t,k}^v \leq (t - \partial_k^v + 0.5) / T' + 1, \forall t = 1, \dots, T', k \in K, \quad (4.33)$$

$$s_{t,m}^r \geq (t - \partial_m^r + 1) / T', \forall t = 1, \dots, T', m \in R, \quad (4.34)$$

$$s_{t,m}^r \leq (t - \partial_m^r + 0.5) / T' + 1, \forall t = 1, \dots, T', m \in R, \quad (4.35)$$

$$\sum_{i,j \in S_0} \sum_{t \leq w, t > t'_{ij}} t_{i,j}^v \cdot x_{i,j,t-t'_{ij},k}^v + \sum_{i \in S_0} \sum_{t \leq w} t^{\text{load}} \left(y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-} + y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-} \right) \leq w \cdot \tau, \forall w = 1, \dots, T', k \in K, \quad (4.36)$$

$$\sum_{i,j \in S_0} \sum_{t \leq w, t > 0} t_{i,j}^v \cdot x_{i,j,t,k}^v + \sum_{i \in S_0} \sum_{t \leq w} t^{\text{load}} \left(y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-} + y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-} \right) \geq (w-2) \cdot \tau - \varsigma_{w,k}^v T, \forall w = 1, \dots, T', k \in K. \quad (4.37)$$

$$\sum_{i,j \in S_0} \sum_{t \leq w, t > t_{ij}''} t_{i,j}^r \cdot x_{i,j,t-t_{ij}'',m}^r + \sum_{i \in S} \sum_{t \leq w} t^{\text{repair}} g_{i,t,m} \leq w \cdot \tau, \forall w = 1, \dots, T', m \in R, \quad (4.38)$$

$$\sum_{i,j \in S_0} \sum_{t \leq w, t > 0} t_{i,j}^r \cdot x_{i,j,t,m}^r + \sum_{i \in S} \sum_{t \leq w} t^{\text{repair}} g_{i,t,m} \geq (w-2) \cdot \tau - \varsigma_{w,m}^r T, \forall w = 1, \dots, T', m \in R, \quad (4.39)$$

$$x_{0,0,t,k}^v \leq 1 - \left(\varsigma_{t+1,k}^v - \varsigma_{t,k}^v \right), \forall t = 1, \dots, T' - 1, k \in K, \quad (4.40)$$

$$x_{0,0,t,m}^r \leq 1 - \left(\varsigma_{t+1,m}^r - \varsigma_{t,m}^r \right), \forall t = 1, \dots, T' - 1, m \in R, \quad (4.41)$$

$$x_{i,i,t,k}^v \leq \left(y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-} + y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-} \right) + \left\lfloor \frac{\tau}{t^{\text{load}}} \right\rfloor \varsigma_{t,k}^v, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.42)$$

$$x_{i,i,t,m}^r \leq g_{i,t,m} + \left\lfloor \frac{\tau}{t^{\text{repair}}} \right\rfloor \varsigma_{t,m}^r, \forall i \in S_0, t = 1, \dots, T', m \in R, \quad (4.43)$$

$$\sum_{i \in S_0} \sum_{j \in S_0} x_{i,j,t,k}^v \leq 1, \forall t = 1, \dots, T', k \in K, \quad (4.44)$$

$$\sum_{i \in S_0} \sum_{j \in S_0} x_{i,j,t,m}^r \leq 1, \forall t = 1, \dots, T', m \in R, \quad (4.45)$$

$$\sum_{j \in S_0, j \neq i} \sum_{t=t''_{j,i}+1}^{T'} \sum_{m \in R} x_{j,i,t-t''_{j,i},m}^r \leq 1, \forall i \in S_0, \quad (4.46)$$

$$e_{i,j,t,k} \geq CER \cdot \left[\rho_0 \cdot x_{i,j,t,k}^v + \frac{\rho^* - \rho_0}{Q_k} \cdot \left(\psi_{i,j,t,k}^u + \psi_{i,j,t,k}^b \right) \right] \cdot d_{ij}, \quad \forall i, j \in S_0, t = 1, \dots, T', k \in K, \quad (4.47)$$

$$x_{i,j,t,k}^v \in \{0, 1\}, \forall i, j \in S_0, t = 1, \dots, T', k \in K, \quad (4.48)$$

$$x_{i,j,t,m}^r \in \{0, 1\}, \forall i, j \in S_0, t = 1, \dots, T', m \in R, \quad (4.49)$$

$$s_{t,k}^v \in \{0, 1\}, \forall t = 1, \dots, T', k \in K, \quad (4.50)$$

$$s_{t,m}^r \in \{0, 1\}, \forall t = 1, \dots, T', m \in R, \quad (4.51)$$

$$p_{i,t} \geq 0, \forall i \in S_0, t = 1, \dots, T', \quad (4.52)$$

$$b_{i,t} \geq 0, \forall i \in S_0, t = 1, \dots, T', \quad (4.53)$$

$$y_{i,t,k}^{u^+} \geq 0, \text{integer}, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.54)$$

$$y_{i,t,k}^{u^-} \geq 0, \text{integer}, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.55)$$

$$y_{i,t,k}^{b^+} \geq 0, \text{integer}, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.56)$$

$$y_{i,t,k}^{b^-} \geq 0, \text{integer}, \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (4.57)$$

$$g_{i,t,m} \geq 0, \text{ integer}, \forall i \in S_0, t = 1, \dots, T', m \in R, \quad (4.58)$$

$$\partial_k^v \geq 0, \text{ integer}, \forall k \in K, \quad (4.59)$$

$$\partial_m^r \geq 0, \text{ integer}, \forall m \in R, \quad (4.60)$$

$$\partial_k^v \leq T', \text{ integer}, \forall k \in K, \quad (4.61)$$

$$\partial_m^r \leq T', \text{ integer}, \forall m \in R, \quad (4.62)$$

$$\psi_{i,j,t,k}^u \geq 0, \forall i, \quad j \in S_0, t = 1, \dots, T', k \in K, \quad (4.63)$$

$$\psi_{i,j,t,k}^b \geq 0, \forall i, \quad j \in S_0, t = 1, \dots, T', k \in K, \quad (4.64)$$

$$e_{i,j,t,k} \geq 0, \forall i, \quad j \in S_0, t = 1, \dots, T', k \in K. \quad (4.65)$$

The objective (4.2) is to minimize the sum of the penalty cost of the total user dissatisfaction at all stations and the cost of the total CO₂ emissions of all the trucks. Note that we add a third term to penalize the sum of the total travel time of trucks and repairers, bike loading/unloading times for trucks, and repairing times for the repairers. The inclusion of the third term helps discourage non-intuitive or redundant operations that, while feasible, may not be necessary given the available resources and time. For example, as shown in Figure 4.3 (a), the first case illustrates a truck loading eight usable bikes at a station in period t_0 and then unloading two bikes at the same station in the next period $t_0 + 1$. This solution is feasible and could be optimal when the third term in the objective was not included, but such an operation is redundant as it can be simplified by loading only six bikes at once. The second case

in Figure 4.3 (b) shows a repairer repairing one broken bike at one station, traveling to another station to repair one broken bike, and then returning to the original station to repair one additional bike. Again, while feasible, this solution is non-intuitive because the unnecessary return visit to the same station can be avoided. Overall, the third term in the objective penalizes such operations, ensuring a more straightforward and efficient rebalancing process. To avoid introducing unnecessary bias when the experiments involve the scenario without repairers, we set the coefficient θ for this penalty term to be $1e-8$. This ensures that the third term does not affect solution optimality. The largest possible value for this term is $(|K|+|R|)T$, and under our experimental setting, the maximum value of the term is 43,200 ($|K| = 2$, $|R| = 2$, and $T = 10800$), which is small enough to avoid any noticeable impact on results that are rounded to two decimal places.

Redundant operation examples placeholder.

Figure 4.3: Examples of using the third component in the objective to avoid redundant operations.

Constraints (4.3), (4.4), (4.5), (4.6) keep track of the numbers of usable and broken bikes at each node at the end of each period. Constraints (4.7) stipulate that the total number of usable and broken bikes at each station is no more than its capacity. Constraints (4.8) specify the initial load of usable bikes on each truck during the first period. Constraints (4.9) update the load of usable bikes on each truck at each node during each period. Constraints (4.10) require that each truck holds no usable bikes at the end of the last period. Constraints (4.11) ensure that the quantity of usable bikes carried by each truck in any period does not exceed its capacity. Similar constraints for broken bikes are provided in constraints (4.12), (4.13), (4.14), (4.15). Constraints (4.16) ensure that the total number of usable and broken bikes on each truck during each period is no more than its capacity.

Constraints (4.17), (4.18) make sure that the total quantity of usable bikes delivered to or collected from a visited station in a period is less than or equal to the minimum of the station capacity and the truck capacity. Constraints (4.19) limit that the number of broken bikes that can be deposited at the depot in a period is less than or equal to the truck capacity. Constraints (4.20) ensure no broken bikes are delivered to any station. Constraints (4.21) enforce that no broken bikes are collected from the depot. Constraints (4.22) make sure that the total quantity of

broken bikes collected from a visited station in a period is less than or equal to the minimum of the initial broken bike inventory at the station and the truck capacity. Constraints (4.23) guarantee that no broken bike is repaired by the on-site repairers at the depot. Constraints (4.24) limit that the number of broken bikes repaired at any station is less than or equal to the initial broken bike inventory at the station. Constraints (4.25) ensure that the total number of broken bikes collected by trucks and repaired by repairers at each station does not exceed the initial broken bike inventory.

Constraints (4.26), (4.27), (4.28), (4.29) require that both trucks and repairers depart from the depot initially and return to the depot eventually. Constraints (4.30), (4.31) are the flow conservation conditions for trucks and repairers, respectively. Constraints (4.32), (4.33), (4.34), (4.35) relate $\varsigma_{t,k}^v$ (the variable used to define whether a truck or a repairer stays still at the depot from period t to the end of the rebalancing process) to ∂_k^v (the period of the truck's or repairer's final return to the depot).

Constraints (4.36), (4.37), (4.38), (4.39) restrict the upper and lower bounds of the operational time, and are modified from the study of Liu et al. (2018). Specifically, constraints (4.36) imply that up to each period, the sum of route travel time and the service time at each node cannot exceed the time from the start of repositioning to the end of that period, and constraints (4.37) imply that up to each period, the sum of route travel time and the service time at each node cannot be lower than the time from the start of repositioning to the end of two periods before that period. Similar constraints are also imposed on the repairers in constraints (4.38), (4.39). Note that constraints (4.37), (4.39) are only activated before the period that the truck and the repairer return to the depot for the last time, respectively.

Constraints (4.40), (4.41) ensure the truck or the repairer stays at the depot after the final return. Constraints (4.42), (4.43) limit that before the final return to the depot, if the truck or repairer stays at the same node for another period, it must conduct at least one loading, unloading, or bike repair here. Constraints (4.44) enforce that a truck can travel between at most one pair of nodes within a period. A similar set of constraints for repairers is given in constraints (4.45). Constraints (4.46) ensure that each station can be visited at most once by at most one repairer in the whole repositioning horizon. Constraints (4.47) define the carbon emissions of each truck produced in each period when traveling between any two nodes. Constraints (4.48), (4.49), (4.50), (4.51), (4.52), (4.53),

(4.54), (4.55), (4.56), (4.57), (4.58), (4.59), (4.60), (4.61), (4.62), (4.63), (4.64), (4.65) are the domain constraints.

Following Kaspi (2015), the EUDF is linearized by introducing a new variable $\hat{F}_i$ for each station $i \in S$, and the following linear constraints are added to the formulation.

$$\hat{F}_i \geq \alpha_{p_i, b_i}^i \cdot p_{i, T'} + \beta_{p_i, b_i}^i \cdot b_{i, T'} + \gamma_{p_i, b_i}^i, \forall (p_i, b_i) \in \{(p_i, b_i) \in \mathbb{Z}^2 \mid p_i \geq 0, b_i \geq 0, p_i + b_i \leq C_i\}, \forall i \in S. \quad (4.66)$$

The resultant objective is

$$\begin{aligned} & \text{minimize } c^p \sum_{i \in S} \hat{F}_i + c^e \sum_{i \in S_0} \sum_{j \in S_0} \sum_{t=t'_{i,j}+1}^{T'} \sum_{k \in K} e_{i,j,t-t'_{i,j},k} + \\ & \theta \cdot \left(\sum_{i \in S_0} \sum_{j \in S_0} \sum_{t=t'_{i,j}+1}^{T'} \sum_{k \in K} \left(t_{i,j}^v \cdot x_{i,j,t-t'_{i,j},k}^v \right) + \sum_{i \in S_0} \sum_{t=t'_{i,j}+1}^{T'} \sum_{k \in K} t^{\text{load}} \cdot \left(y_{i,t,k}^u + y_{i,t,k}^{u-} + y_{i,t,k}^{b+} + y_{i,t,k}^{b-} \right) \right. \\ & \left. + \sum_{i \in S_0} \sum_{j \in S_0} \sum_{t=t''_{i,j}+1}^{T'} \sum_{m \in R} \left(t_{i,j}^r \cdot x_{i,j,t-t''_{i,j},m}^r \right) + \sum_{i \in S_0} \sum_{t=t''_{i,j}+1}^{T'} \sum_{m \in R} t^{\text{repair}} \cdot g_{i,t,m} \right) \end{aligned} \quad (4.67)$$

4.2. SOLUTION METHOD

This study developed a hybrid algorithm (HGSADC-SBC) consisting of Hybrid Genetic Search with Adaptive Diversity Control (HGSADC) and a Station Budget Constrained (SBC) heuristic to solve the proposed model. The HGSADC is the backbone of the algorithm, and the SBC heuristic is encapsulated to determine the quantities of loading, unloading, and repairing of bikes for a given route.

4.2.1. Solution representation

Each solution is represented by $\phi = (S_R, S_K, \Omega_R, \Omega_K)$, where $S_R = (r_1, r_2, \dots, r_{|R|})$ denotes the list of repairer routes r_i ($i \in R$), and $S_K = (r'_1, r'_2, \dots, r'_{|K|})$ denotes the list of truck routes r'_k ($k \in K$). The repairing quantities are represented by $\Omega_R = (G_1, G_2, \dots, G_{|R|})$, with G_i ($i \in R$) specifying the repairing quantities associated with the repairer route r_i ($i \in R$). Similarly, the (un)loading quantities are represented by $\Omega_K = (Y_1, Y_2, \dots, Y_{|K|})$,

where Y_k ($k \in K$) specifies the (un)loading quantities associated with the truck route r'_k ($k \in K$).

The route of each repairer is represented as $r_i = (s_{i_0}, s_{i_1}, s_{i_2}, \dots, s_{i_{|r_i|-2}}, s_{i_{|r_i|-1}})$, $i \in R$, and the route of each truck is represented as $r'_k = (s'_{k_0}, s'_{k_1}, s'_{k_2}, \dots, s'_{k_{|r'_k|-2}}, s'_{k_{|r'_k|-1}})$, $k \in K$, where $s_{i_j} \in S_0$ and $s'_{k_l} \in S_0$ ($0 \leq j \leq |r_i| - 1$, $0 \leq l \leq |r'_k| - 1$), and the subscripts j and l are used to define the order of nodes being visited. Every route comprises a permutation of visited nodes, with the first and last being the depot, labeled with 0, i.e., $s_{i_0} = s_{i_{|r_i|-1}} = 0$ ($i \in R$) and $s'_{k_0} = s'_{k_{|r'_k|-1}} = 0$ ($k \in K$). For any of the two routes in S_R , no common stations are allowed between the initial and ending depot.

For repairer i ($i \in R$), the repairing quantities associated with route r_i are represented as $G_i = (g_{s_{i_0}, i}, g_{s_{i_1}, i}, \dots, g_{s_{i_{|r_i|-1}}, i})$, where $g_{s_{i_j}, i}$ denotes the number of broken bikes repaired at node s_{i_j} in r_i ($0 \leq j \leq |r_i| - 1$). Similarly, for truck k ($k \in K$), the (un)loading quantities associated with route r'_k are denoted by $Y_k = ((y_{0,k}^{u+}, y_{0,k}^{u-}, y_{0,k}^{b+}, y_{0,k}^{b-}), (y_{1,k}^{u+}, y_{1,k}^{u-}, y_{1,k}^{b+}, y_{1,k}^{b-}), \dots, (y_{|r'_k|-1,k}^{u+}, y_{|r'_k|-1,k}^{u-}, y_{|r'_k|-1,k}^{b+}, y_{|r'_k|-1,k}^{b-}))$, where $y_{l,k}^{u+}$ and $y_{l,k}^{u-}$ denote the number of usable bikes unloaded and loaded at node s_{k_l} respectively, and $y_{l,k}^{b+}$ and $y_{l,k}^{b-}$ represent the number of broken bikes unloaded and loaded at s_{k_l} ($0 \leq l \leq |r'_k| - 1$) respectively.

The solution that corresponds to the one shown in Figure 4.1 in a network with ten stations and one depot with two repairers and two trucks can be represented as

$$\phi = (S_R, S_K, \Omega_R, \Omega_K) = ((r_1, r_2), (r'_1, r'_2), (G_1, G_2), (Y_1, Y_2))$$

where:

- $r_1 = (0, 6, 8, 0)$ and $G_1 = (0, 4, 12, 0)$;
- $r_2 = (0, 1, 3, 0)$ and $G_2 = (0, 3, 2, 0)$;
- $r'_1 = (0, 4, 6, 9, 0, 2, 0)$ and $Y_1 = ((0, 0, 0, 0), (0, 2, 0, 1), (0, 5, 0, 17), (7, 0, 0, 0), (0, 3, 18, 0), (3, 0, 0, 9), (0, 0, 9, 0))$;
- $r'_2 = (0, 5, 4, 0, 7, 5, 9, 0)$ and $Y_2 = ((0, 0, 0, 0), (0, 12, 0, 10), (0, 0, 0, 3), (8, 0, 13, 0), (0, 3, 0, 2), (0, 1, 0, 3), (8, 0, 0, 0), (0, 0, 5, 0))$.

Note that the zeros at the beginning and end of each route are included solely for representation purposes. In the implementation, these zeros are excluded during operations at each step, with only the sequence of the visited nodes in between being retained to enhance computational efficiency.

4.2.2. Overall algorithmic procedure

The algorithmic procedure for HGSADC-SBC is given in Table 4.2. The algorithm starts with the initialization of a population Pop consisting of a feasible subpopulation Pop_f and an infeasible subpopulation Pop_i (Line 1). Then, the population of individuals iteratively evolves through successive operations (Line 3 to Line 29). The best fitness value and its corresponding solution up to each iteration are kept with $current_best_value$ and $current_best_sol$, respectively. The algorithm terminates and returns the best known solution and fitness value when the number of iterations without improvement (denoted as $iter_no_imp$) reaches the limit It_{NI} , or the running time (denoted as cpu_time) reaches the limit T_{max} . A diversification process is activated every It_{div} iterations without improvement to maintain a high level of diversity among individuals (Line 26 to Line 28).

Table 4.2: Algorithmic procedure

Algorithm 1: HGSADC-SBC

- 1 Initialization: Generate a population Pop consisting of a feasible subpopulation Pop_f and an infeasible subpopulation Pop_i
- 2 Set $iter_no_imp = 0$ and $current_best_value = +\infty$
- 3 **while** $iter_no_imp < It_{NI}$ and $cpu_time < T_{max}$ **do**
- 4 Randomly select two individuals Ind_1 and Ind_2 from Pop
- 5 Generate the routes of offspring C_1 and C_2 using ordered crossover on the routes of Ind_1 and Ind_2
- 6 Educate on both C_1 and C_2 by applying local search procedures to generate new routes and using the fitness value to select the best route
- 7 **if** C_i ($i = 1, 2$) is feasible **then**
- 8 Add C_i to the feasible subpopulation Pop_f
- 9 **else**
- 10 Add C_i to the infeasible subpopulation Pop_i and repair C_i with the probability P_{repair} using local search
- 11 **end if**
- 12 **if** the individuals become feasible after repairs **then**
- 13 The repaired individuals are added to the feasible subpopulation Pop_f
- 14 **else**

Algorithm 1: HGSADC-SBC

```
15   The repaired individuals are discarded, and the original infeasible solution is kept in the infeasible
16   end if
17   if the size of  $Pop_f$  or  $Pop_i$  reaches the maximum subpopulation size then
18     Select survivors from the corresponding subpopulation
19   end if
20   Set  $bs$  = the solution with the best fitness value from  $Pop_f$  and set  $bv$  = the fitness value of  $bs$ 
21   if  $bv \geq current\_best\_value$  then
22     Set  $iter\_no\_imp = iter\_no\_imp + 1$ 
23   Else
24     Set  $current\_best\_value = bv$ ,  $current\_best\_sol = bs$ , and  $iter\_no\_imp = 0$ 
25   end if
26   if  $iter\_no\_imp \geq It_{div}$  then
27     Diversify the population consisting of  $Pop_f$  and  $Pop_i$  and adjust the penalty parameter
28   end if
29   end while
30   return  $current\_best\_sol$ ,  $current\_best\_value$ 
```

The algorithm structure follows the one given by Vidal et al., 2012, Vidal et al., 2013, Vidal et al., 2014). The route representation, crossover operator, and education operator are based on the study by Li et al. (2016) with some modifications, mostly on extending their methods to address the scenario of multiple routes for multiple trucks and repairers from the scenario of a single route for only one truck. Moreover, in this algorithm, the crossover and local search operations in education and the repair process are applied to the routes of the solutions. Once new routes are generated through these operations, the SBC heuristic is subsequently applied to determine the loading, unloading, and repair quantities, completing each solution with both routes and associated quantities. The SBC heuristic is developed based on a novel station budget concept, and is detailed in Section 4.2.4.

4.2.3. Population initialization

To initialize a population, we generate solutions one by one until a specified population size is reached. The generation of each solution requires initializing routes for both repairers and trucks, followed by applying the SBC heuristic to determine the

associated (un)loading and repairing quantities. Each solution then undergoes a feasibility check (as detailed in Section 4.2.5) and is added to the feasible or infeasible subpopulation based on its feasibility.

To initialize a route, the depot is set as the first node. Then, new nodes are iteratively added to the route based on the last node of the current route, denoted as n_{last} . Specifically, we first find the set of nodes that allow the truck/repairer to reach there from n_{last} , conduct at least one loading/unloading/repairing there, and return to the depot within the time budget. The set is defined as the *reachable list* of n_{last} . Note that n_{last} itself is not included in the set. We randomly choose one node from the *reachable list* and append it to the end of the route. The procedure repeats until the reachable list of n_{last} is empty, and we finally append the depot at the end. Other routes are generated in the same manner. Note that the routes are generated one by one, in the order of repairers first and trucks second to make the best use of the repairers to repair broken bikes before truck arrivals. It should also be noted that any station that has already been visited by a repairer is not considered when generating routes for other repairers.

4.2.4. Station budget constrained (SBC) heuristic for quantity determination

For each route of a truck or repairer, we have to determine the quantities of bikes loaded, unloaded, and repaired at designated stations to form complete solutions and calculate their fitness values. To determine these quantities, an SBC heuristic is proposed. Some notations and terminologies are defined first to better describe the SBC heuristic:

- q_s represents the target usable inventory level at station s ($s \in S$), which is determined as the usable bike inventory at station s that corresponds to the lowest EUDF value. It together with the initial inventory of usable bikes p_s^0 can be used to define the station type. A station is categorized into a deficit station, a surplus station, and a balanced station if $p_s^0 < q_s$, $p_s^0 > q_s$, and $p_s^0 = q_s$ hold, respectively.
- p_s and b_s are used to track the inventory of usable and broken bikes at station s ($s \in S$), respectively. At the beginning of the SBC heuristic, they are assigned to the initial inventories before the repositioning starts. During the procedure of the heuristic, they are updated when bikes are loaded or unloaded from the trucks or when repairers conduct repairs.

- $\psi_{l,k}^u$ and $\psi_{l,k}^b$ are used to track the inventory of usable and broken bikes on truck k ($k \in K$) when it leaves the l -th visited node s'_{kl} ($l = 0, \dots, |r'_k| - 1$).
- $maxTime_{s,i}^r$ represents the station repairing budget assigned to station $s \in S$ for repairer $i \in R$.
- $maxTime_{s,k}^t$ represents the station (un)loading budget assigned to station $s \in S$ for truck $k \in K$.
- $maxTimeRemain_{s,k}^t$ represents the remaining station (un)loading budget assigned to station $s \in S$ for truck $k \in K$.
- $unsat_{s,k}^{load,u}$ represents the number of usable bikes not loaded onto truck k due to the limited station loading budget at station $s \in S$.
- $unsat_{s,k}^{unload,u}$ represents the number of usable bikes not unloaded to station $s \in S$ from truck k due to the limited station unloading budget.
- $unsat_{s,k}^{load,b}$ represents the number of broken bikes not loaded onto truck k due to the limited station loading budget at station $s \in S$.
- $maxTimeInit_{s,k}^t$ represents the initial station time budget assigned to station $s \in S$ for truck k before determining truck loading quantities.
- $maxTimeNew_{s,k}^t$ represents the updated station time budget assigned to station $s \in S$ for truck k after adjustment.

We point out that the variables presented above are global variables that are shared across all subsequent algorithms, which means any changes to these variables are effective in all algorithms. For the sake of brevity, we directly use and assign values to these variables within the algorithms without redefining them or including them as arguments in each algorithmic procedure. In addition, in the following algorithm description, we use a notation where a variable is enclosed in parentheses and subscripted by its index domains to represent a list of variables indexed by the subscripts. For example, $\left(maxTime_{s,i}^r\right)_{s \in S}$ represents a one-dimensional list that stores the station budget at each station in route r_i for repairer i .

The SBC heuristic procedure is formed by the following steps:

- **Step 1: Initialization.** Set initial values for station inventories.

- **Step 2: Assign repairing quantities for repairers.** For each repairer, conduct the following three subprocedures:
 - Assign a station repair budget to each visited station in the repairer's route.
 - Determine the number of bike repairs to be conducted at each station, constrained by the quantities allowed to be handled under the assigned station budget, the broken bike inventory at the station, and the deviation of the usable bike inventory to the station target.
 - Update the station's usable and broken bike inventories to reflect the completed repairs.
- **Step 3: First-round truck loading and unloading assignment.** For each truck, conduct the following three subprocedures:
 - Allocate a station (un)loading budget for each station in the truck's route.
 - Assign the initial quantities for loading and unloading usable and broken bikes at each station, based on station status (surplus or deficit) and station budget.
 - Update truck and station inventories accordingly.
- **Step 4: Adjust station budgets based on unused time.** Adjust the station (un)loading budgets to redistribute any unused time to stations where demand remains unmet.
- **Step 5: Second-round truck loading and unloading assignment.** Revert station inventories to their values before the first round of loading/unloading assignment, and reassign the quantities for the second round using the adjusted station budgets obtained from **Step 4**.
- **Step 6: Final amendments to routes and quantities.** Make final adjustments to the truck and repairer routes, along with their associated loading, unloading, and repairing quantities, to optimize the use of any time budget left.

For the detailed procedure of the SBC heuristic, readers can refer to **Algorithm 2 in Appendix B**.

In the following subsections, we delve into the details of each component of the algorithm, including station budget assignment in Steps 2.1 and 3.1 (Section 4.2.4), repairer repairing quantity assignment in Step 2.2 and truck (un)loading

quantity assignment in Step 3.2 (Section 4.2.4), station (un)loading budget and truck (un)loading quantity adjustments in Steps 4 and 5 respectively (Section 4.2.4), and the final amendments to the routes, repairing, and (un)loading quantities in Step 6 (Section 4.2.4).

Station budget allocation

The SBC heuristic allocates truck (un)loading time and repairer repairing time at stations by leveraging the EUDF to define a Benefit-Cost Ratio Function (BCRF). The EUDF captures user dissatisfaction as a function of station inventories, offering a convex framework for assessing the impact of inventory adjustments. The BCRF builds upon the EUDF by quantifying the effectiveness of resource allocation at each station to reduce user dissatisfaction per unit of time spent. Specifically, the BCRF for a station is computed as the change in EUDF value—reflecting the benefit of truck (un)loading or repairs—divided by the operational time required to achieve that change.

Given the usable bike inventory p , the broken bike inventory b , and the target usable bike inventory q at station s , the BCRF value for a truck at that station (denoted as $BCRF_s^t(p, b)$) is defined as

$$BCRF_s^t(p, b) = \begin{cases} 0, & p = q, b = 0; \\ \frac{F_s(p, b) - F_s(q, 0)}{t^{\text{load}} \cdot (|q - p| + b)}, & \text{Otherwise.} \end{cases} \quad (4.68)$$

For balanced stations without broken bikes, the BCRF value for a truck at that station is defined as zero, as indicated in the first condition of the piecewise function. In the second condition, the denominator is the time required to handle the usable and broken bikes at the station to achieve the target inventory and is considered to be a cost, assuming the value of time is fixed. The numerator is the penalty reduction and is considered to be a benefit. $F_s(q, 0)$ is the minimum EUDF value so the numerator is always non-negative.

Figure 4.4 shows an example illustrating the calculation of the values of the BCRF for trucks based on the EUDF. The three columns in this illustration represent Stations 1, 2, and 3, respectively.

In the first row, the bar charts display the current inventory of usable bikes, the target inventory for usable bikes, and the current inventory of broken bikes at

BCRF and EUDF illustration placeholder.

Figure 4.4: An illustration of calculating the BCRF values of three stations based on the EUDF values.

each station. Arrows indicate the deviations of the current inventories of usable bikes from the corresponding target inventory levels. The second row contains heat maps showing the EUDF values for various combinations of usable and broken bike inventories at each station. Darker colors correspond to higher dissatisfaction, as captured by the EUDF. The red dots represent the current inventory levels, and the green dots indicate the target inventory levels. The arrows between these points show how the current inventory can be adjusted to match the target, specifying how many usable and broken bikes need to be loaded and unloaded.

For Station 3, the change in color between the current and target inventory levels is subtle, suggesting that spending time at this station has little effect on reducing the EUDF value. In contrast, Station 2 shows a significant color shift, indicating a large potential for reducing dissatisfaction by adjusting the inventory level.

The observations in the last paragraph are further quantified in the BCRF values on the last row, which are calculated by dividing the reduction in EUDF value by the time required to adjust the inventory from the current to the target level. Station 2's BCRF value is nearly ten times greater than that of Station 3, emphasizing that more time should be spent on Station 2 to maximize the reduction in user dissatisfaction per unit of time.

Similarly, for each repairer, we also have $BCRF_s^r(p, b)$ defined as

$$BCRF_s^r(p, b) = \begin{cases} 0, & p \geq q, b = 0; \\ \frac{F_s(p, b) - F_s(p + \min\{b, \max\{q - p, 0\}\}, 0)}{t_{\text{repair}} \cdot \min\{b, \max\{q - p, 0\}\}}, & \text{Otherwise.} \end{cases} \quad (4.69)$$

For balanced and surplus stations without broken bikes, the BCRF value for a truck at that station is defined as zero, as indicated in the first condition of the piece-wise function. In the second condition, the denominator $\min\{b, \max\{q - p, 0\}\}$ is the maximum number of broken bikes repaired at the concerned station, which cannot be larger than the number of usable bikes required at that station because the repaired bikes become usable. That is why the repaired bikes are also included in the second term of the numerator.

The station repair budget is allocated based on the calculated $BCRF^r$ for each station, which is determined by prioritizing those with higher BCRF values that reflect a greater reduction in user dissatisfaction per unit of time. Specifically, the available repair time is distributed proportionally across stations according to the $BCRF^r$ of the station, ensuring a balanced allocation of resources while preventing excessive time spent at any single station. To make the best use of the time budget, any overly allocated time is redistributed to unsatisfied stations with unmet repair needs following the descending order of $BCRF^r$. The detailed algorithm for this is presented in **Algorithm 3** in Appendix C. The station (un)loading budget allocation algorithm for trucks is similar to the one for repairers, as presented in **Algorithm 4** in Appendix C, except that it considers the truck's loading and unloading operations instead of repair operations. To illustrate the main idea with a simple example, consider a route consisting of the above three stations 0–1–2–3–0, where 0 is the depot. After accounting for the route's travel time, the remaining time for bike loading and unloading is distributed proportionally across the stations based on their BCRF values. The results of this distribution are depicted in Figure 4.5, which are calculated as follows:

$$\begin{aligned}\text{Station 1: } & \frac{0.0103}{0.0103+0.0344+0.0032} = 21.5\%, \\ \text{Station 2: } & \frac{0.0344}{0.0103+0.0344+0.0032} = 71.8\%, \text{ and} \\ \text{Station 3: } & \frac{0.0032}{0.0103+0.0344+0.0032} = 6.7\%.\end{aligned}$$

BCRF-based time allocation placeholder.

Figure 4.5: Proportion of operational time allocated to each station based on their BCRF values.

Loading, unloading, and repairing quantity assignment

With the station (un)loading budgets determined, we proceed to assign the quantities of usable bikes to be loaded or unloaded by trucks, broken bikes to be loaded by trucks, and broken bikes to be repaired by repairers. To improve the clarity of the main text, the detailed algorithms for these assignments (from **Algorithm 5** to **Algorithm 9**) are provided in Appendix D. Different from the traditional greedy heuristic, which assigns quantities based solely on the truck's inventory and the station's demand, our heuristic incorporates the station budget as an additional factor.

The budget limits the maximum time a truck or repairer can spend at a station for operations. By considering this factor, we prevent excessive time consumption at early stations that may not necessarily justify such a large time investment, and ensure a more balanced and efficient allocation of resources across all stations.

Station (un)loading budget and truck (un)loading quantity adjustments

In **Algorithm 2** in Appendix B, after the assignment of loading and unloading quantities, it is observed that some stations may still have a loading budget unused while others encounter a shortfall. To make use of the extra time, an adjustment to the station (un)loading budget allocations for trucks is necessary. The adjustment aims to redistribute the unused station (un)loading budgets from stations where the allocated time budget exceeds the actual loading and unloading time requirements to those stations where the budget is insufficient. The algorithm is given in Appendix E.

Amendments to the routes and assigned quantities for trucks and repairers

When assigning repair, loading, and unloading quantities, a conservative rounding-down strategy is applied (e.g., Line 3 in Algorithm 6 in Appendix D). This can lead to the availability of unused time budgets, which can be utilized for more bike repairs and loading. Therefore, this section provides amendments to the routes and assigned quantities for trucks and repairers.

The amendment procedure begins by removing stations that are not assigned with any loading, unloading, and repairing quantities, then calculating the leftover time for each truck and repairer based on the outputs from **Algorithm 2**. It then focuses on the stations which are still not balanced. For each repairer at every station, the amendment procedure explores the possible broken bike repairing quantity that ranges from zero to the maximum allowed by the extra time. It selects the repairing quantity that results in the lowest $BCRF_s^r$. Meanwhile, each truck checks the surplus stations and redistributes as many surplus bikes as possible to deficit stations. The choice of the surplus and deficit stations prioritizes those with the highest $BCRF_s^t$. For each surplus station checked by the truck, the candidate destinations are set as the stations between the current station and the next depot along the truck's route, with surplus bikes unloaded to the next depot if unloading at any candidate station would violate station or truck capacity constraints. After checking all surplus stations, if

extra time remains, supplementary loadings of broken bikes are conducted similarly, with the distinction that broken bikes can only be unloaded at the depot.

4.2.5. Feasibility check and solution evaluation

After determining the loading, unloading, and repairing quantities, we can evaluate solution ϕ by

$$Z(\phi) = f(\phi) + \eta\varepsilon(\phi) \quad (4.70)$$

where $f(\phi)$ is the sum of the penalty costs of the total user dissatisfaction at all stations in the network and the total cost of carbon emissions during the rebalancing process. $\varepsilon(\phi)$ is related to the penalties for solution infeasibility and η is the penalty coefficient of $\varepsilon(\phi)$. If $\varepsilon(\phi)$ is 0, then solution ϕ is feasible.

The infeasibility we consider here originates from the fact that the heuristic proposed in Section 4.4 determines the quantities route by route separately, without considering the interdependencies between different routes and their effects on station inventories. This strategy is used to make the decision-making process more straightforward and easier to implement. It assumes that the arrival times at stations follow the visiting order of stations in the route, with all visits by trucks with lower indices occurring before any visits by trucks with higher indices. This assumption, however, can lead to invalid quantities being assigned.

Consider a problem where two trucks perform the repositioning task in a network with one depot. For simplicity, we assume there are no repairers in this example. The route for Truck 1 is (0–1–3–5–0), and the route for Truck 2 is (0–2–5–0). We notice that Station 5 is visited by both trucks. It is unclear which truck would arrive first at Station 5 when the arrival times at stations, which depend on quantity assignment and travel times, are not calculated. The method proposed in Section 4.2.4 presumed that the first truck arrived at Station 5 ahead of the second truck, updating the inventories and assigning the repositioning quantities accordingly, which can lead to an invalid assignment. For example, as shown in Figure 4.6 (a), suppose the capacity of the station is 20, with a usable bike inventory of 3, a broken bike inventory of 10, and a target usable bike inventory of 13. If Truck 1 arrives at Station 5 from Station 3 with a truckload of 0 usable bikes and 0 broken bikes, according to the strategy in Section 4.2.4, it collects 10 broken bikes (provided the remaining time is enough) at Station 5, and thus the broken bike

inventory of Station 5 is updated to 0. Then, for Truck 2, if it arrives at Station 5 with 12 usable bikes, it delivers 10 usable bikes to achieve the target usable bike inventory. However, if Truck 1 actually arrives at Station 5 later than Truck 2, the decision to deliver 10 usable bikes by Truck 2 results in a total inventory of usable and broken bikes of 23 ($3 + 10 + 10$), which exceeds the station capacity (see Figure 4.6 (b)). The scenario could become more complicated if the repairers also conduct repairs at the station. Therefore, we need to validate the feasibility of the solution by examining the inventories of the multiple visited stations after the operation of each visited truck or repairer.

Quantity-assignment infeasibility placeholder.

Figure 4.6: An example of solution infeasibility in the quantity assignment.

After the operations at a node, the usable or broken bike inventory may be below 0, or the total inventory of usable and broken bikes may be over the capacity of the station. We set $\varepsilon(\phi)$ to the total number of bikes that exceeds the capacity or is below 0, which is calculated following the procedure below:

- **Step 1:** Calculate the arrival time ar_s for every station $s \in S$ visited by each repairer and truck according to solution ϕ and the quantities determined in Section 4.2.4, and record the results as a list of tuples $(n, s, y_s^{b-}, y_s^{u+}, y_s^{u-}, g_s, ar_s)$, where n represents the index of the repairer or truck operating at station s , and the number of loaded broken bikes, loaded usable bikes, unloaded usable bikes, and repaired broken bikes are represented as y_s^{b-} , y_s^{u+} , y_s^{u-} , and g_s , respectively.
- **Step 2:** Sort the list of tuples according to the arrival time ar_s .
- **Step 3:** Select the nodes in the list one by one in the ascending order of arrival time and perform the loading/unloading/repairing to update the inventory of usable and broken bikes. Update $\varepsilon(\phi)$ if, after the operations at some node, the usable bike inventory p_s or the broken bike inventory b_s is below 0, or the total inventory of usable and broken bikes exceeds the capacity of the station C_s by

$$\varepsilon(\phi) \leftarrow \varepsilon(\phi) - (\min\{p_s, 0\} + \min\{b_s, 0\} + \min\{C_s - p_s - b_s, 0\}).$$

Next, we define the biased fitness value of a solution to consider the diversity in each subpopulation, which is calculated with

$$BF(\phi) = rank_{\text{fit}}(\phi) + \left(1 - \frac{N_{\text{elite}}}{N_{\text{indiv}}}\right) rank_{\text{sim}}(\phi) \quad (4.71)$$

where N_{elite} is the number of elite individuals that survive to the next generation, and N_{indiv} is the actual number of individuals in the subpopulation. $rank_{\text{fit}}(\phi)$ is the rank of solution ϕ in its subpopulation regarding $Z(\phi)$ sorted in the ascending order, and $rank_{\text{sim}}(\phi)$ is the rank of solution ϕ regarding the similarity with other solutions in the subpopulation.

In our problem, for each solution ϕ , we gather the arc set of all truck routes in ϕ to the set TA_ϕ , and the arc set of all repairer routes in ϕ to a set RA_ϕ . Let χ be the subpopulation that ϕ belongs to. Depending on the feasibility of ϕ , $\chi = Pop_f$ if ϕ is feasible, and $\chi = Pop_i$ otherwise. Then, the similarity of ϕ with respect to the subpopulation χ it belongs to is defined as

$$\left| TA_\phi \cap \left(\bigcup_{\phi' \in \chi \setminus \{\phi\}} TA_{\phi'} \right) \right| + \left| RA_\phi \cap \left(\bigcup_{\phi' \in \chi \setminus \{\phi\}} RA_{\phi'} \right) \right|. \quad (4.72)$$

which is the sum of the number of common arcs between the truck routes in the solution and the union of truck routes from every other solution in the corresponding subpopulation and the number of common arcs between repairer routes in the solution and the union of repairer routes from every other solution in the corresponding subpopulation.

4.2.6. Selection, crossover, education, and repairing

In each iteration, two individuals are selected using a binary tournament, which randomly picks two individuals from the population and reserves the one with a better-biased fitness value. The crossover and local search strategies for routes in education and repair are basically the same as those proposed by Li et al. (2016). Therefore, we omit the details here. The only difference is that in our solution, there can be multiple truck routes and multiple repairer routes. Hence, for each solution, the crossover, education, and repair are carried out route by route. For cases with more than one truck or repairer, we only consider the crossover of the routes with the same index. For example, the route of the first repairer in solution ϕ_1 only makes

a crossover with the route of the first repairer in solution ϕ_2 . The repair operation follows the method by Vidal (2022) except that the SBC heuristic is used after each new route is generated.

4.2.7. Population management and search guidance

The population management mechanism works with selection, crossover, and education operators to help propagate the characteristics of excellent solutions, increase population diversity, and provide guidance for a more efficient search. At the beginning of the algorithm, 4μ individuals are initialized (Li et al., 2016), where μ is the minimum subpopulation size. Each of them directly goes through an education process using local search procedures and the SBC heuristic, and then falls into a feasible or infeasible subpopulation based on the feasibility of the solution. The repair operation is activated for infeasible individuals with a probability of P_{repair} . The repair operation runs local search procedures and the SBC heuristic under a penalty factor of 10 times the original one, aiming to convert an infeasible solution into a feasible one (Vidal, 2022). If the repair is successful, the resulting individual is added to the feasible subpopulation. Otherwise, the infeasible one before repair is preserved in the infeasible subpopulation. The algorithm keeps generating new individuals under the process of selection, crossover, education, and repair until the time limit or the maximum number of iterations without improvement is reached.

During the above process, individuals generated through crossover, education, or repair are added to the corresponding subpopulation based on their feasibility. If the size of either subpopulation reaches $\mu + \lambda$, where λ is the number of offspring in a generation, survivor selection is conducted to remove the λ solutions with the highest biased fitness values to ensure that the size equals μ .

As in the study of Li et al. (2016), the penalty coefficient η in equation (4.70) is modified every 100 iterations to adjust the proportion of feasible solutions. Let ξ^{REF} be the target proportion of feasible individuals. If the proportion of feasible individuals with respect to $\varepsilon(\phi)$ is less than $\xi^{\text{REF}} - 5\%$ or greater than $\xi^{\text{REF}} + 5\%$, then we change η to 1.2η or 0.85η , respectively (Li et al., 2016).

The diversification operation is triggered if the best solution has not been improved for It_{div} iterations. In the diversification, the best $\frac{\mu}{3}$ solutions of each subpopulation are preserved (Li et al., 2016), and 4μ new individuals are generated, followed by the survivor selection to adjust the size of each subpopulation to μ .

4.3. COMPUTATIONAL EXPERIMENTS

The proposed model was implemented in Python 3.11 using Gurobi Python API (Gurobi Optimization, LLC, 2023). The hybrid algorithm HGSADC-SBC was implemented with C++. All numerical experiments were conducted on a computer equipped with an Intel Core i9-12900 processor (12 cores, 24 threads) and 64 GB of physical memory.

In Section 4.3.1, we introduce the parameter tuning and setting for HGSADC-SBC. In Section 4.3.2, we demonstrate the effectiveness of the proposed algorithm by presenting results and running times for small instances involving a single truck and repairer. In Section 4.3.3, we evaluate the algorithm’s performance on larger networks with multiple trucks and repairers. Section 4.3.4 focuses on the impact of varying repair times on the cost-effectiveness of including a repairer through experiments on different proportions of broken bikes.

The parameters regarding the network are from the dataset used by Ho and Szeto (2017), which is available at <https://www.dropbox.com/s/jbsf2wqssrr/hlms-set3.zip?dl=0>. The dataset contains the travel times between nodes, the capacity of each station, and the initial inventory of usable bikes at each station. The travel times used in the experiments are also from the dataset above, in which the asymmetric travel times (in seconds) between the stations were obtained from the Open Source Routing Machine project (<https://project-osrm.org/>) on the travel times of trucks between stations. However, since the project does not provide travel times for electric tricycles (the vehicle assumed to be used by repairers in this study), we assume the speed of the electric tricycles is proportional to that of the trucks. The proportionality constant is determined based on values from the existing literature that provide relevant speed information for both vehicles (see Table 4.3). As a result, the travel times for repairers between nodes can be calculated accordingly based on the constant.

Table 4.3: Parameter setting of the numerical experiment

Parameter		Value
ρ_0	The empty-load fuel consumption rate of each truck (liters/km)	0.252
ρ^*	The full-load fuel consumption rate of each truck (liters/km)	0.258
CER	The CO ₂ emission rate (kg/liter)	2.61

Parameter		Value
Q_k	The capacity of truck k (bikes)	25
t^{load}	Time for loading/unloading a bike (seconds)	60
t^{repair}	Time for repairing a bike on-site (seconds)	300
$\frac{v^t}{v^m}$	The ratio of speed between a truck and an electric tricycle	1.68
c^p	The penalty cost for a user unable to rent/return a bike (CNY)	2
c^e	The unit carbon trading cost (CNY/kg)	0.06
θ	The penalty coefficient used to eliminate unnecessary loading or unloading operations	1e-8

The EUDF value of each station was calculated based on the historical data of Citi Bike, which consists of the number of bike rentals and returns at each station, recorded every 15 min from 6:00 a.m. to 12:00p.m., over the course of a single day.

All instances used in this section were generated by randomly selecting a certain number of stations from the 518-station dataset. Specifically, in parameter tuning in Section 4.3.1, a 60-station network was generated. In the experiment in Section 4.3.2, five different instances were generated for each network size (6, 10, and 15 stations). In the experiment in Section 4.3.3, instances with network sizes ranging from 60 to 500 stations were created, with one instance generated for each size. In the experiment in Section 4.3.4, one of the 15-station networks generated in Section 4.3.2 was used. As the dataset provides no information on the initial inventory of broken bikes, we set the initial quantity of broken bikes b_i^0 at station i as follows: in the experiments in Sections 4.3.1, 4.3.2, and 4.3.3, b_i^0 was assigned a pre-generated random value within the range $[0, C_i - p_i^0]$, while in the experiment in Section 4.3.4, it was set proportional to the residual capacity $C_i - p_i^0$. Note that the time-indexed formulation for the MILP converts all travel times into integer intervals by dividing the actual travel time by τ and rounding up to the nearest integer, which can lead to an inflated time representation caused by such an approximation. Consequently, the lower bounds obtained from Gurobi in this section may not be the accurate bounds for the actual (continuous-time) problem.

The setting for other parameters of the experiments is listed in Table 4.3. Note that except for the experiment in Section 4.3.3, in which we vary the number of trucks and repairers on larger networks, we fixed the number of trucks and repairers to one. For the repositioning time budget, we uniformly set it to two hours in

the experiments in Sections 4.3.1, 4.3.2, and 4.3.4. In Section 4.3.3, for larger instances, experiments were conducted under both two-hour and three-hour time budgets. In the time discretization, we used 600 s as the length of each period τ for most experiments, except in Section 4.3.4, where repair times vary between 300 and 1200 s. To accommodate the largest repair time in these settings, in Section 4.3.4, we set the length of each period τ to 1200 s, ensuring that repair times do not exceed the period as defined in the parameter table in Table 4.1. The codes and the generated datasets used in the experiments are available at <https://github.com/rqhu1995/brpwr>.

4.3.1. Parameter tuning and setting for the algorithm

The parameter setting mostly follows that of Li et al. (2016), who referred to the research conducted by Vidal et al., 2012, Vidal et al., 2013, Vidal et al., 2014). The only parameter that differs from theirs and requires tuning is η , the one related to the penalty of infeasible solutions. η was set to take values from the set $\{10, 50, 100, 150, 200, 300, 400, 500\}$. We ran the algorithm, setting η to be each value from the candidate set. The result of the average objective value in 20 runs under different values of η is given in Figure 4.7, and 100 was chosen because it yields the best average result within a shorter CPU time. The detailed parameter setting is summarized in Table 4.4.

Parameter tuning plot placeholder.

Figure 4.7: Parameter tuning results for the penalty coefficient η ($|S| = 60$, $T = 2$ h, $|K| = |R| = 1$).

Table 4.4: Parameter setting for HGSADC-SBC.

Parameter	Value
The minimum subpopulation size μ	25
The number of offspring in a generation λ	40
The target proportion of feasible individuals ξ^{REF}	0.2
The maximum number of iterations without improvement It_{NI}	5000
The number of successive iterations that triggers the diversification operation It_{div}	200 ($0.4It_{\text{NI}}$)
The time limit for executing the algorithm T_{max}	7200 s
Probability for repairing an infeasible solution P_{repair}	0.5
The number of elite individuals that survive to the next generation N_{elite}	10

Parameter	Value
Penalty coefficient for solution infeasibility η	100

4.3.2. Comparison of the results between HGSADC-SBC and Gurobi

This section aims to compare the results obtained from HGSADC-SBC with the optimal solutions obtained by Gurobi for small network instances. Across all instances, the optimal solutions could be found using both the solver and the heuristic for all the tested instances, and when applied with different random seeds 20 times to a single instance, the heuristic consistently found the optimal solution. Therefore, we only report the CPU times using Gurobi and the average CPU times over 20 runs with HGSADC-SBC in Table 4.5.

Table 4.5: Comparison between the CPU times (s) of Gurobi and HGSADC-SBC on small networks (repositioning time budget $T = 2$ h, $|K| = |R| = 1$, $\tau = 600$ s).

$ S $	instance	Gurobi	HGSADC-SBC
6	1	13.41	6.29
	2	13.85	5.75
	3	66.97	6.22
	4	2.29	3.63
	5	22.29	4.84
10	1	928.50	3.87
	2	1409.89	5.27
	3	13768.72	3.02
	4	2404.03	5.16
	5	4172.75	4.17
15	1	2602.24	7.80
	2	4209.67	4.89
	3	6283.58	4.72
	4	3374.38	5.32
	5	2371.40	5.08

In most instances, HGSADC-SBC outperformed Gurobi in terms of CPU times,

producing optimal solutions in significantly less time. Noticeably, in one of the 10-station instances (instance 3), HGSADC-SBC completed the computations in as little as 3.02 s on average, whereas Gurobi required over 13,000 s (nearly four hours) for the same instance. However, in a smaller 6-station instance (instance 4), Gurobi required slightly less CPU time than HGSADC-SBC. This is likely because a significant portion of the computing time of the heuristic was spent on the time required to check the convergence. Despite this, the overall computational benefits of HGSADC-SBC are evident as the network size increases, making it a more efficient choice for large-scale problems. It is also worth noting that the time-index formulation can make the model computationally hard, even for instances comprising just a small number of stations, further showing the necessity of using heuristics to solve the problem efficiently.

4.3.3. Comparison of results from HGSADC-SBC and Gurobi on larger networks

In this section, we proceed to compare the results obtained from HGSADC-SBC with the best feasible solutions found by Gurobi for larger network instances with different numbers of trucks and repairers. Gurobi was given a two-hour time limit for computation in all instances in this section. In some cases, Gurobi terminated earlier due to running out of memory. Hence, no CPU times are reported for those instances (marked as N/A). The results are reported in Table 4.6.

Table 4.6: Comparison between the results of Gurobi and HGSADC-SBC on larger networks (repositioning time budget $T = 2$ h, $\tau = 600$ s).

S	K	R	Gurobi		CPU (s)	Empty Cell	HGSADC-SBC			
			UB	LB			¹ Obj. Avg.	² Obj. Best.	³ Obj. Std.	
60	1	1	2604.63	2517.12	7200.00		2581.29	2579.82	0.86	
	1	2	2533.44	2420.32	7200.00		2494.24	2486.32	4.09	
	2	1	3581.80	2284.28	7200.00		2400.65	2379.21	10.18	
	2	2	3517.71	2202.55	7200.00		2324.50	2302.98	11.76	
	1	1	4510.58	3893.04	7200.00		4038.28	4014.71	13.59	
90	1	2	4159.43	3782.32	7200.00		3941.78	3915.27	14.03	
	2	1	4510.58	3656.81	7200.00		3862.44	3829.09	19.77	
	2	2	4488.64	3557.92	7200.00		3768.58	3733.90	21.59	

S	K	R	Gurobi		CPU (s)	Empty Cell	HGSADC-SBC		
			UB	LB			¹ Obj. Avg.	² Obj. Best.	³ Obj. Std.
120	1	1	5113.72	4247.68	7200.00		4303.05	4293.75	3.58
	1	2	5103.87	4169.64	7200.00		4237.52	4226.39	4.17
	2	1	5113.72	4024.31	7200.00		4167.75	4129.41	16.19
	2	2	5103.87	3951.26	7200.00		4097.19	4071.74	11.87
200	1	1	8657.30	2919.70	7200.00		7821.12	7793.53	19.35
	1	2	8657.30	7563.69	7200.00		7719.58	7694.50	12.08
	2	1	8657.30	7378.11	7200.00		7641.82	7573.84	28.23
	2	2	8657.30	7286.86	7200.00		7561.16	7511.34	22.96
300	1	1	11717.13	10898.80	7200.00		11169.22	11134.10	15.16
	1	2	11716.99	10757.43	7200.00		11057.76	10990.90	19.38
	2	1	11717.13	10561.75	7200.00		10989.39	10953.90	19.84
	2	2	11716.99	10442.63	7200.00		10888.38	10848.40	20.63
400	1	1	15068.74	6026.88	7200.00		14280.67	14267.10	5.89
	1	2	15060.58	6026.88	7200.00		14178.63	14149.00	12.33
	2	1	15068.74	6026.59	7200.00		14169.14	14104.30	21.84
	2	2	15060.58	6026.59	7200.00		14059.18	14033.60	15.29
500	1	1	19793.68	7716.08	7200.00		18961.58	18927.40	18.77
	1	2	19785.07	7716.08	7200.00		18793.14	18734.00	22.68
	2	1	19793.68	7715.69	N/A		18752.12	18618.10	50.56
	2	2	19785.07	N/A	N/A		18612.62	18505.40	37.57

1 the average objective value for each instance in 20 runs.

2 the best objective value for each instance in 20 runs.

3 the standard deviation of the objective value for each instance in 20 runs.

4 average computational time for each instance in 20 runs in seconds.

5 $(UB - \text{Obj. Avg}) / UB$.

As shown in both Table 4.6 and Table 4.7, Gurobi was unable to find optimal solutions within the two-hour CPU time limit in any of the instances. For all instances where memory did not run out, Gurobi used the entire two-hour time limit to provide

the best feasible solution (UB). The difficulty in solving these larger instances again highlights the computational complexity of the problem.

Table 4.7: Comparison between the results of Gurobi and HGSADC-SBC on networks (repositioning time budget $T = 3$ h, $\tau = 600$ s).

S	K	R	Gurobi			Empty Cell	HGSADC-SBC		
			UB	LB	CPU (s)		¹ Obj. Avg.	² Obj. Best.	³ Obj. Std.
60	1	1	3553.62	2337.38	7200.00		2433.29	2412.09	3.14
	1	2	3507.20	2221.33	7200.00		2364.56	2352.33	5.67
	2	1	3581.80	2035.17	7200.00		2264.57	2248.72	8.76
	2	2	3507.20	1930.24	7200.00		2214.40	2196.06	6.44
90	1	1	4503.66	3705.60	7200.00		3903.04	3865.03	16.13
	1	2	4481.06	3563.25	7200.00		3778.54	3735.60	20.69
	2	1	4503.66	3398.88	7200.00		3705.04	3652.58	20.54
	2	2	4481.06	3264.80	7200.00		3606.92	3563.30	21.66
120	1	1	5113.72	4085.94	7200.00		4203.49	4180.03	9.81
	1	2	5103.87	3982.62	7200.00		4106.56	4087.78	8.83
	2	1	5113.72	3784.05	7200.00		4023.59	4007.56	10.96
	2	2	5103.87	3685.32	7200.00		3952.46	3909.70	17.55
200	1	1	8657.30	7457.23	7200.00		7695.07	7636.22	22.75
	1	2	8657.30	7322.35	7200.00		7575.25	7539.85	14.29
	2	1	8657.30	2916.37	7200.00		7471.22	7406.93	32.94
	2	2	8657.30	2916.37	7200.00		7393.52	7333.03	30.51
300	1	1	11715.76	4568.18	7200.00		11024.28	10973.30	21.87
	1	2	11715.62	10462.88	7200.00		10901.05	10858.60	16.88
	2	1	11715.76	4568.16	7200.00		10830.35	10756.00	23.79
	2	2	11715.62	4568.16	7200.00		10692.76	10638.50	25.19
400	1	1	15068.74	6026.59	7200.00		14137.72	14090.00	20.16
	1	2	15060.58	6026.59	7200.00		14009.58	13974.40	15.25
	2	1	15068.74	6026.58	N/A		13968.64	13926.70	21.06
	2	2	15060.58	N/A	N/A		13851.78	13822.50	12.96
500	1	1	19793.68	7715.71	7200.00		18818.29	18716.10	37.12
	1	2	19785.07	N/A	N/A		18596.33	18539.40	26.59
	2	1	19793.68	N/A	N/A		18572.48	18493.30	38.87
	2	2	19785.07	N/A	N/A		18407.06	18250.20	44.37

S	K	R	Gurobi		CPU (s)	Empty Cell	HGSADC-SBC		
			UB	LB			¹ Obj. Avg.	² Obj. Best.	³ Obj. Std.

- 1 the average objective value for each instance in 20 runs.
- 2 the best objective value for each instance in 20 runs.
- 3 the standard deviation of the objective value for each instance in 20 runs.
- 4 average computational time for each instance in 20 runs in seconds.
- 5 $(UB - \text{Obj. Avg}) / UB$.

On the other hand, HGSADC-SBC consistently outperformed Gurobi by finding better feasible solutions (as reflected by a positive gap) in significantly less time. As the repositioning time budget increased from two hours to three hours in the model, the performance gap between HGSADC-SBC and Gurobi widened, especially for instances comprising 60 stations. For example, in the 60-station instance with one truck and two repairers, the gap increased from 1.55 % under the two-hour budget to 32.58 % under the three-hour budget. This indicates that the increased complexity of the model with longer time budgets makes it more difficult for Gurobi to find better solutions, while HGSADC-SBC is still efficient in obtaining better solutions. Additionally, we notice that the computation for some very large instances ($|S| = 400$ and 500) used up the 64 GB memory of the computer before the two-hour time limit was reached. This suggests that as network size increases and resources become a limitation for Gurobi, HGSADC-SBC shows an additional advantage by requiring less memory and less CPU time, making it more suitable for real-world BRP with large networks.

4.3.4. Cost-effectiveness of introducing a repairer under varying repair times and broken bike inventory levels

In this section, we analyze the cost-effectiveness of involving a repairer under different repair times and broken bike quantities. The experiment was conducted on a fixed 15-station network, with the number of broken bikes at each station being proportional to the station's residual capacity with a proportional coefficient denoted by ℓ . The initial broken bike quantity at station i was set to $b_i^0 = \lfloor \ell \cdot (C_i - p_i^0) \rfloor$. ℓ ranges from 10 % to 100 % in an increment of 10 %. For each value of ℓ , the repair

time per bike varies from 300 s to 1200 s, with a step size of 100 s. The length of period τ is set to 1200 s in this section to guarantee $t^{\text{repair}} < \tau$.

To capture the cost-effectiveness of a repairer, an indicator ν is introduced, which is defined as the ratio of the reduction cost in user dissatisfaction (when a repairer is introduced) to the wage paid to the repairer, which is calculated as

$$\nu = \frac{c^p \sum_{i \in S} (\hat{F}_i - \hat{F}'_i)}{\varpi \cdot T / 3600} \quad (4.73)$$

where $\hat{F}_i$ and $\hat{F}'_i$ are the user dissatisfaction after the repositioning at station i with and without a repairer, respectively, c^p is the unit cost of the dissatisfaction, ϖ is the hourly wage of a repairer, and T is the time budget for the repositioning in seconds. The division by 3600 converts the time from seconds to hours, ensuring consistency with the wage unit in CNY/h. The numerator is the reduction in the dissatisfaction cost, while the denominator is the cost paid to the repairer. Note that the dissatisfaction after the repositioning with no repairers $\hat{F}'_i$ was calculated by setting $|R| = 0$ in the optimization model. The hourly wage was determined based on a piece of online job-hunting information and was set to 33.33 CNY/h.

For each setting of ℓ , experiments under varying repair times were conducted, and the results of the variations in ν over repair times are plotted in Figure 4.8. The results show a clear trend across all instances: As repair time increases, the cost-effectiveness ratio decreases consistently, meaning that on-site repairs become less cost-effective. This implies that a longer repair time leads to a smaller reduction in user dissatisfaction, as fewer repairs can be completed by the repairer within the available time. In the extreme case, when the repair time exceeds a certain threshold, introducing on-site repairs is cost-ineffective.

Repairer cost-effectiveness plot placeholder.

Figure 4.8: Variations in the cost-effectiveness ratio over repair times under different broken bike inventories.

One key observation is the existence of a critical point at which the cost-effectiveness ratio ν equals 1.0 at some level of broken bikes. For lower levels of broken bikes ($10\% \leq \ell \leq 20\%$), this ratio remains below 1.0 regardless of the repair times, indicating that the cost of employing a repairer outweighs the benefits. In such

cases, the system can still be effectively managed by trucks alone for repositioning, as the low number of broken bikes does not justify the additional cost of involving a repairer. However, for systems with higher levels of broken bikes ($\ell \geq 30\%$), a turning point emerges where the cost-effectiveness ratio shifts from above to below 1.0. This signifies that as the proportion of broken bikes increases, the benefit of reducing user dissatisfaction gradually begins to outweigh the cost of employing a repairer at lower ranges of repair times. Larger proportions of broken bikes shift the incorporation of a repairer from being cost-ineffective to cost-effective at lower ranges of repair times, reflecting the growing necessity for on-site repairs as the average damage level within the system rises. The changes in the cost-effectiveness ratio over the broken bike inventory levels suggest the necessity for a dynamic assignment of repairers based on the actual level of damage within the system, which includes varying the number of repairers in response to the dynamic change in the broken bike levels in the system to ensure the cost-effectiveness. Specifically, part-time repairer scheduling can serve as a flexible strategy to further enhance resource allocation, allowing systems to adjust the input for repairs dynamically.

Another important observation is the shift in the repair time at which the cost-effectiveness ratio ν equals 1.0. As the level of broken bikes increases, this critical point moves toward longer repair times. For example, when the broken bike inventory reaches 50 %, the point at which $\nu = 1.0$ occurs at approximately 492 s, whereas for the 90 % level, it occurs at around 580 s. This suggests that with a higher number of broken bikes, longer repair times can still yield a favorable cost-effectiveness ratio before the repairer's cost outweighs the benefit. However, once repair times exceed around 645 s, employing a repairer becomes less cost-effective across all instances, regardless of the level of broken bike inventory. Although our model assumes the same repair time for all bikes, this can be viewed as an average repair time, representing the overall damage level of the system. Hence, such a relatively long repair time suggests that the system is experiencing significant damage. Therefore, it may be more effective to shift resources from reactive repairs to preventive maintenance. This strategy can be adapted based on the system's changing damage degrees, helping to reduce the number of broken bikes and the average repair time over time and preventing the system from reaching a state where repairs become less cost-effective.

4.4. CONCLUSIONS

We investigate a new and practical static BRP with broken bikes and on-site repairers. The problem is to find the best routes and loading/unloading instructions of both usable and broken bikes for the repositioning trucks while determining the best routes for on-site repairers and the number of broken bikes repaired by each repairer at each station. The goal is to reduce the sum of the penalty costs of user dissatisfaction and the cost of total CO₂ emissions. The problem is formulated as a time-index Mixed Integer Linear Programming (MILP) model. A hybrid algorithm HGSADC-SBC is proposed to solve the problem efficiently for large instances. The algorithm relies on a novel concept of station budget, which utilizes the Benefit-Cost Ratio Function (BCRF) to determine (un)loading and repair quantities at stations, ensuring that time is spent effectively across the network.

Numerical experiments on small networks of up to 15 stations demonstrated that HGSADC-SBC consistently achieves optimal solutions, matching Gurobi's results while requiring significantly less computational time in most cases. For larger networks, ranging from 60 to 500 stations, HGSADC-SBC outperformed Gurobi in finding a feasible solution obtained under a two-hour time limit, demonstrating the algorithm's computational efficiency. Additionally, we analyzed the cost-effectiveness of introducing a repairer under varying repair times and broken bike inventory levels. The findings show that long repair times reduce cost-effectiveness. In the extreme case, when the repair time exceeds a certain level, introducing on-site repairs is cost-ineffective. Moreover, when the number of broken bikes in the system is small, introducing on-site repairs can become detrimental. The results also underscore the need for a dynamic repairer allocation strategy based on the system's actual damage level and suggest that preventive maintenance strategies can reduce the average number of broken bikes in the system and the average repair time over time.

It is worth noting that we conducted additional experiments comparing a full station set with a reduced station set, where stations with no broken bicycles and inventory levels that (nearly) minimize the EUDF were removed. The results showed no statistically significant difference in terms of objective values or CPU times between the two approaches. This indicates that the heuristic's natural allocation of negligible station budgets to such stations achieves a similar effect to explicitly removing them from the problem, supporting the efficiency and robustness of the current approach. Since the manuscript is already lengthy, we omit the detailed

experimental results in the manuscript.

While this study addresses key aspects of the problem, two limitations must be acknowledged to provide a comprehensive perspective on the findings. The time-indexed formulation, although necessary for the MILP model to handle multiple visits to stations by both trucks and repairers, introduces a discrepancy between the discrete and continuous-time problems by approximating travel times through integer conversion, limiting the accuracy of the model in reflecting the continuous-time problem. Additionally, the SBC heuristic assumes that bicycles are (un)loaded or repaired directly at their respective stations without intermediate stops or second-time transfers. This simplification streamlines implementation but restricts the heuristic from leveraging potential efficiency gains through temporary inventories, which could yield better solutions. Future research will develop methodologies to address these limitations.

CHAPTER 5

EXTENDED-USER-DISSATISFACTION-AWARE CLUSTER-FIRST, ROUTE-SECOND DECOMPOSITION FOR THE STATIC BIKE REPOSITIONING PROBLEM WITH BROKEN BIKES AND ON-SITE REPAIR

This chapter extends the cluster-first, route-second idea to the static bike repositioning problem with broken bikes and on-site repair. The problem setting combines the operational features introduced in Chapter 4 with the scalability motivation of Chapter 3. The resulting framework aims to partition a large-scale system into clusters while accounting for the expected change in user dissatisfaction caused by repositioning, repair, and station-level inventory changes.

The chapter first presents the EUDF-based formulation of the static bike repositioning problem with broken bikes and on-site repair. It then develops an EUDF-aware cluster-first, route-second framework, an Extended Activated Transfer Selection procedure for evaluating clustering decisions, and a Hybrid Genetic Search for solving the clustering problem. Cluster-level planning and computational experiments are then presented to assess the effectiveness and scalability of the proposed framework.

5.1. PROBLEM FORMULATION

5.1.1. Problem setting

This section presents the Bike Repositioning Problem with Broken Bikes and On-Site Repair (BRP-BR-OSR), which is used as the reference problem in the subsequent methodology. The problem is adapted from Hu, Szeto, and Ho [1]: it retains the operational setting and feasibility constraints of that model, removes the emission component, and evaluates a repositioning plan by the total user dissatisfaction at all stations on the following day.

The problem considers a BSS with one depot and a finite set of stations over an overnight rebalancing horizon. Each station has a fixed capacity, an initial usable-bike inventory, and an initial broken-bike inventory, all of which are assumed to be known at the beginning of the horizon. Usable bikes can be rented by users after

repositioning, whereas broken bikes cannot be rented and continue to occupy docks until they are collected or repaired.

A fleet of repositioning trucks and a set of repairers operate within the same time budget. Trucks depart from the depot, travel among stations, deliver or collect usable bikes, collect broken bikes, may return intermediately to the depot to load or unload bikes, and finally return to the depot. Repairers also depart from the depot, visit stations with broken bikes, repair broken bikes on site, and return to the depot. Each loading, unloading, or on-site repair activity has a fixed processing time per bike. Trucks may visit stations and the depot multiple times, while each station can be visited by at most one repairer.

The depot is assumed to have sufficient usable-bike inventory and sufficient storage capacity for usable and broken bikes. Repairers only perform repairs on-site at stations; the model does not decide how broken bikes returned to the depot are repaired afterward. During the rebalancing horizon, no rental or return demand occurs, and the inventories after repositioning become the starting inventories for the following day. Following Hu, Szeto, and Ho [1], we assume that repairers working at stations and any repair staff at the depot are paid fixed daily salaries. Therefore, total repair staffing cost is constant for the day and independent of the number of broken bikes repaired on site or sent back to the depot during the repositioning period; it is not included in the objective.

The objective of the BRP-BR-OSR is to determine truck routes, repairer routes, truck loading and unloading quantities, and on-site repair quantities that minimize total user dissatisfaction at all stations on the following day, subject to inventory, capacity, routing, and time-budget constraints.

5.1.2. Notation

The following notation is used throughout the formulation. The model keeps the time-indexed representation of truck movements, repairer movements, station inventories, truck loads, and station-level actions from Hu, Szeto, and Ho [1], but omits all emission-related parameters, variables, and objective terms. Table 5.1 lists the sets, parameters, and decision variables retained in the formulation.

Table 5.1

Notation for the BRP-BR-OSR formulation.

Sets and indices

Symbol	Description
$S = \{1, \dots, S \}$	The set of stations.
$S_0 = S \cup \{0\}$	The set of nodes, including depot 0.
K	The set of rebalancing trucks.
R	The set of repairers.
i, j	The node indices; $i, j \in S$ for station-only quantities and $i, j \in S_0$ for routing quantities.
k	The truck index, $k \in K$.
m	The repairer index, $m \in R$.
t, w	The time-period indices, $t, w = 1, \dots, T'$.

Parameters

Symbol	Description
p_i^0	The initial number of usable bikes at node i .
b_i^0	The initial number of broken bikes at node i .
C_i	The capacity of node i .
Q_k	The capacity of truck k .
T	The rebalancing time budget.
τ	The length of one discretized time period.
T'	The number of periods in the rebalancing horizon, $T' = \lfloor T/\tau \rfloor$.
t_{ij}^v	The travel time for a truck from node i to node j .
t_{ij}^r	The travel time for a repairer from node i to node j .
t'_{ij}	The number of periods required by a truck to travel from node i to node j .
t''_{ij}	The number of periods required by a repairer to travel from node i to node j .
t^{load}	The time required to load or unload one bike.
t^{repair}	The time required to repair one broken bike on site.
$F_i(p_i, b_i)$	The Extended User Dissatisfaction Function (EUDF) value at station i when its usable-bike and broken-bike inventories are p_i and b_i .
Θ_i	The set of feasible bivariate inventory states at station i .
$\alpha_{p_i, b_i}^i, \beta_{p_i, b_i}^i, \gamma_{p_i, b_i}^i$	The coefficients of the fitted plane associated with state (p_i, b_i) in the EUDF linearization for station i .

Decision variables

Symbol	Description
$x_{i,j,t,k}^v$	The binary variable equal to 1 if truck k departs from node i to node j in period t , and 0 otherwise.
$x_{i,j,t,m}^r$	The binary variable equal to 1 if repairer m departs from node i to node j in period t , and 0 otherwise.
$p_{i,t}$	The number of usable bikes at node i at the end of period t .
$b_{i,t}$	The number of broken bikes at node i at the end of period t .
$y_{i,t,k}^{u+}$	The number of usable bikes delivered to node i by truck k during period t .
$y_{i,t,k}^{u-}$	The number of usable bikes collected from node i by truck k during period t .
$y_{i,t,k}^{b+}$	The number of broken bikes delivered to node i by truck k during period t .
$y_{i,t,k}^{b-}$	The number of broken bikes collected from node i by truck k during period t .
$g_{i,t,m}$	The number of broken bikes repaired by repairer m at node i during period t .
$\psi_{i,j,t,k}^u$	The number of usable bikes on truck k when it departs from node i to node j during period t .
$\psi_{i,j,t,k}^b$	The number of broken bikes on truck k when it departs from node i to node j during period t .
∂_k^v	The period in which truck k returns to the depot and stays there until the end of the horizon.
∂_m^r	The period in which repairer m returns to the depot and stays there until the end of the horizon.
$s_{t,k}^v$	The binary variable equal to 1 if truck k stays at the depot from period t to the end of the horizon, and 0 otherwise.
$s_{t,m}^r$	The binary variable equal to 1 if repairer m stays at the depot from period t to the end of the horizon, and 0 otherwise.
$\hat{F}_i$	The auxiliary variable representing the linearized EUDF value at station i .

Travel times and the rebalancing budget T are measured in the same units. To use them in the time-indexed formulation, we divide the horizon into periods of equal length τ and convert truck and repairer travel times into discrete period counts

as follows:

$$t'_{ij} = \begin{cases} \lceil t_{ij}^v / \tau \rceil, & i \neq j, \\ 1, & i = j, \end{cases} \quad \forall i, j \in S_0, \quad (5.1)$$

$$t''_{ij} = \begin{cases} \lceil t_{ij}^r / \tau \rceil, & i \neq j, \\ 1, & i = j, \end{cases} \quad \forall i, j \in S_0, \quad (5.2)$$

$$T' = \lfloor T / \tau \rfloor. \quad (5.3)$$

The binary route variables use a departure-period convention. If $x_{i,j,t,k}^v = 1$, truck k leaves node i in period t and can next depart from node j only after t'_{ij} periods; repairer movements use the same convention with t''_{ij} . These period counts maintain the discrete route sequence. Original travel times and processing times are checked separately by the cumulative time constraints in (5.20). The ceiling operation in (5.1)–(5.2) can overstate travel time within the time-expanded route; the remaining period slack can absorb loading, unloading, and repair activities subject to the cumulative time bounds. This time-indexed representation is used because trucks may visit the same station or the depot multiple times, so the formulation must track resource locations, station inventories, and truck loads over the discretized horizon.

5.1.3. EUDF-based objective

Following Kaspi, Raviv, and Tzur [2], the EUDF value $F_i(p_i, b_i)$ represents the weighted expected user dissatisfaction at station i over a day, given that p_i usable bikes and b_i broken bikes are present at the station at the beginning of that day. In the BRP-BR-OSR, the overnight repositioning plan determines the inventories after repositioning, which are used as the starting inventories for the following day.

The feasible bivariate inventory states at station i are

$$\Theta_i = \{(p_i, b_i) \in \mathbb{Z}_+^2 : p_i + b_i \leq C_i\}. \quad (5.4)$$

The EUDF values are precomputed for all states in Θ_i and are treated as model inputs. For the detailed calculation of these values, readers can refer to Kaspi, Raviv, and Tzur [2].

Kaspi, Raviv, and Tzur [2] also proposed a method for determining a convex

polyhedral function that closely approximates the EUDF values, enabling the integration of the EUDF into the subsequent linear optimization model. Specifically, for each $(p_i, b_i) \in \Theta_i$, a plane $\alpha_{p_i, b_i}^i p_i + \beta_{p_i, b_i}^i b_i + \gamma_{p_i, b_i}^i$ can be fitted as close as possible to $F_i(p_i, b_i)$, where α_{p_i, b_i}^i , β_{p_i, b_i}^i , and γ_{p_i, b_i}^i are the coefficients of the fitted plane for state (p_i, b_i) . These fitted-plane coefficients are precomputed and used in the time-indexed MILP below.

5.1.4. Time-indexed mixed-integer formulation

The time-indexed MILP for the BRP-BR-OSR is given as follows:

$$\min \sum_{i \in S} \hat{F}_i \quad (5.5)$$

$$\text{s.t. } \hat{F}_i \geq \alpha_{p_i, b_i}^i p_{i, T'} + \beta_{p_i, b_i}^i b_{i, T'} + \gamma_{p_i, b_i}^i, \quad \forall (p_i, b_i) \in \Theta_i, i \in S. \quad (5.6)$$

$$p_{i,1} = p_i^0 - \sum_{k \in K} y_{i,1,k}^{u-} + \sum_{k \in K} y_{i,1,k}^{u+} + \sum_{m \in R} g_{i,1,m}, \quad \forall i \in S_0, \quad (5.7a)$$

$$p_{i,t} = p_{i,t-1} - \sum_{k \in K} y_{i,t,k}^{u-} + \sum_{k \in K} y_{i,t,k}^{u+} + \sum_{m \in R} g_{i,t,m}, \quad \forall i \in S_0, t = 2, \dots, T'. \quad (5.7b)$$

$$b_{i,1} = b_i^0 - \sum_{k \in K} y_{i,1,k}^{b-} + \sum_{k \in K} y_{i,1,k}^{b+} - \sum_{m \in R} g_{i,1,m}, \quad \forall i \in S_0, \quad (5.8a)$$

$$b_{i,t} = b_{i,t-1} - \sum_{k \in K} y_{i,t,k}^{b-} + \sum_{k \in K} y_{i,t,k}^{b+} - \sum_{m \in R} g_{i,t,m}, \quad \forall i \in S_0, t = 2, \dots, T'. \quad (5.8b)$$

$$p_{i,t} + b_{i,t} \leq C_i, \quad \forall i \in S, t = 1, \dots, T'. \quad (5.9)$$

$$\sum_{j \in S_0} \psi_{0,j,1,k}^u = y_{0,1,k}^{u-}, \quad \forall k \in K, \quad (5.10a)$$

$$\sum_{j \in S_0} \psi_{i,j,t,k}^u = \sum_{\substack{j \in S_0: \\ t > t'_{j,i}}} \psi_{j,i,t-t'_{j,i},k}^u - y_{i,t,k}^{u+} + y_{i,t,k}^{u-}, \quad \forall i \in S_0, \quad t = 2, \dots, T', \quad k \in K, \quad (5.10b)$$

$$\psi_{i,j,T',k}^u = 0, \quad \forall i, j \in S_0, \quad k \in K, \quad (5.10c)$$

$$\psi_{i,j,t,k}^u \leq Q_k x_{i,j,t,k}^v, \quad \forall i, j \in S_0, \quad t = 1, \dots, T', \quad k \in K. \quad (5.10d)$$

$$\psi_{i,j,1,k}^b = 0, \quad \forall i, j \in S_0, \quad k \in K, \quad (5.11a)$$

$$\sum_{j \in S_0} \psi_{i,j,t,k}^b = \sum_{\substack{j \in S_0: \\ t > t'_{j,i}}} \psi_{j,i,t-t'_{j,i},k}^b - y_{i,t,k}^{b+} + y_{i,t,k}^{b-}, \quad \forall i \in S_0, \quad t = 2, \dots, T', \quad k \in K, \quad (5.11b)$$

$$\psi_{i,j,T',k}^b = 0, \quad \forall i, j \in S_0, \quad k \in K, \quad (5.11c)$$

$$\psi_{i,j,t,k}^b \leq Q_k x_{i,j,t,k}^v, \quad \forall i, j \in S_0, \quad t = 1, \dots, T', \quad k \in K. \quad (5.11d)$$

$$\psi_{i,j,t,k}^u + \psi_{i,j,t,k}^b \leq Q_k, \quad \forall i, j \in S_0, \quad t = 1, \dots, T', \quad k \in K. \quad (5.12)$$

$$y_{i,t,k}^{u+} \leq \min\{C_i, Q_k\} \sum_{j \in S_0} x_{i,j,t,k}^v, \quad \forall i \in S_0, \quad t = 1, \dots, T', \quad k \in K, \quad (5.13a)$$

$$y_{i,t,k}^{u-} \leq \min\{C_i, Q_k\} \sum_{j \in S_0} x_{i,j,t,k}^v, \quad \forall i \in S_0, \quad t = 1, \dots, T', \quad k \in K. \quad (5.13b)$$

$$y_{0,t,k}^{b^+} \leq Q_k \sum_{j \in S_0} x_{0,j,t,k}^v, \quad \forall t = 1, \dots, T', k \in K, \quad (5.14a)$$

$$y_{i,t,k}^{b^+} = 0, \quad \forall i \in S, t = 1, \dots, T', k \in K, \quad (5.14b)$$

$$y_{0,t,k}^{b^-} = 0, \quad \forall t = 1, \dots, T', k \in K, \quad (5.14c)$$

$$y_{i,t,k}^{b^-} \leq \min\{b_i^0, Q_k\} \sum_{j \in S_0} x_{i,j,t,k}^v, \quad \forall i \in S, t = 1, \dots, T', k \in K. \quad (5.14d)$$

$$g_{0,t,m} = 0, \quad \forall t = 1, \dots, T', m \in R, \quad (5.15a)$$

$$g_{i,t,m} \leq b_i^0 \sum_{j \in S_0} x_{i,j,t,m}^r, \quad \forall i \in S, t = 1, \dots, T', m \in R. \quad (5.15b)$$

$$\sum_{t=1}^{T'} \sum_{k \in K} y_{i,t,k}^{b^-} + \sum_{t=1}^{T'} \sum_{m \in R} g_{i,t,m} \leq b_i^0, \quad \forall i \in S. \quad (5.16)$$

$$\sum_{j \in S_0} x_{0,j,1,k}^v = 1, \quad \forall k \in K, \quad (5.17a)$$

$$\sum_{j \in S} x_{0,j,1,m}^r = 1, \quad \forall m \in R, \quad (5.17b)$$

$$\sum_{i \in S_0} x_{i,0,T'-t'_{i,0},k}^v = 1, \quad \forall k \in K, \quad (5.17c)$$

$$\sum_{i \in S_0} x_{i,0,T'-t''_{i,0},m}^r = 1, \quad \forall m \in R. \quad (5.17d)$$

$$\sum_{\substack{j \in S_0: \\ t > t'_{j,i}}} x_{j,i,t-t'_{j,i},k}^v = \sum_{u \in S_0} x_{i,u,t,k}^v, \quad \forall i \in S_0, t = 2, \dots, T', k \in K, \quad (5.18a)$$

$$\sum_{\substack{j \in S_0: \\ t > t''_{j,i}}} x_{j,i,t-t''_{j,i},m}^r = \sum_{u \in S_0} x_{i,u,t,m}^r, \quad \forall i \in S_0, t = 2, \dots, T', m \in R. \quad (5.18b)$$

$$s_{t,k}^v \geq (t - \partial_k^v + 1)/T', \quad \forall t = 1, \dots, T', k \in K, \quad (5.19a)$$

$$s_{t,k}^v \leq (t - \partial_k^v + 0.5)/T' + 1, \quad \forall t = 1, \dots, T', k \in K, \quad (5.19b)$$

$$s_{t,m}^r \geq (t - \partial_m^r + 1)/T', \quad \forall t = 1, \dots, T', m \in R, \quad (5.19c)$$

$$s_{t,m}^r \leq (t - \partial_m^r + 0.5)/T' + 1, \quad \forall t = 1, \dots, T', m \in R. \quad (5.19d)$$

$$\sum_{i,j \in S_0} \sum_{\substack{t \leq w: \\ t > t'_{ij}}} t_{ij}^v x_{i,j,t-t'_{ij},k}^v + \sum_{i \in S_0} \sum_{t \leq w} t^{\text{load}} \left(y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-} + y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-} \right) \leq w\tau, \quad \forall w = 1, \dots, T', k \in K, \quad (5.20a)$$

$$\sum_{i,j \in S_0} \sum_{\substack{t \leq w: \\ t > 0}} t_{ij}^v x_{i,j,t,k}^v + \sum_{i \in S_0} \sum_{t \leq w} t^{\text{load}} \left(y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-} + y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-} \right) \geq (w-2)\tau - s_{w,k}^v T, \quad \forall w = 1, \dots, T', k \in K, \quad (5.20b)$$

$$\sum_{i,j \in S_0} \sum_{\substack{t \leq w: \\ t > t'_{ij}}} t_{ij}^r x_{i,j,t-t'_{ij},m}^r + \sum_{i \in S} \sum_{t \leq w} t^{\text{repair}} g_{i,t,m} \leq w\tau, \quad \forall w = 1, \dots, T', m \in R, \quad (5.20c)$$

$$\sum_{i,j \in S_0} \sum_{\substack{t \leq w: \\ t > 0}} t_{ij}^r x_{i,j,t,m}^r + \sum_{i \in S} \sum_{t \leq w} t^{\text{repair}} g_{i,t,m} \geq (w-2)\tau - s_{w,m}^r T, \quad \forall w = 1, \dots, T', m \in R. \quad (5.20d)$$

$$x_{0,0,t,k}^v \leq 1 - \left(s_{t+1,k}^v - s_{t,k}^v \right), \quad \forall t = 1, \dots, T' - 1, k \in K, \quad (5.21a)$$

$$x_{0,0,t,m}^r \leq 1 - \left(s_{t+1,m}^r - s_{t,m}^r \right), \quad \forall t = 1, \dots, T' - 1, m \in R, \quad (5.21b)$$

$$x_{i,i,t,k}^v \leq y_{i,t,k}^{u^+} + y_{i,t,k}^{u^-} + y_{i,t,k}^{b^+} + y_{i,t,k}^{b^-} + \lfloor \tau/t^{\text{load}} \rfloor s_{t,k}^v, \quad \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (5.21c)$$

$$x_{i,i,t,m}^r \leq g_{i,t,m} + \lfloor \tau/t^{\text{repair}} \rfloor s_{t,m}^r, \quad \forall i \in S_0, t = 1, \dots, T', m \in R. \quad (5.21d)$$

$$\sum_{i \in S_0} \sum_{j \in S_0} x_{i,j,t,k}^v \leq 1, \quad \forall t = 1, \dots, T', k \in K, \quad (5.22a)$$

$$\sum_{i \in S_0} \sum_{j \in S_0} x_{i,j,t,m}^r \leq 1, \quad \forall t = 1, \dots, T', m \in R, \quad (5.22b)$$

$$\sum_{\substack{j \in S_0: \\ j \neq i}} \sum_{t=t''_{j,i}+1}^{T'} \sum_{m \in R} x_{j,i,t-t''_{j,i},m}^r \leq 1, \quad \forall i \in S_0. \quad (5.22c)$$

$$x_{i,j,t,k}^v \in \{0, 1\}, \quad \forall i, j \in S_0, t = 1, \dots, T', k \in K, \quad (5.23a)$$

$$x_{i,j,t,m}^r \in \{0, 1\}, \quad \forall i, j \in S_0, t = 1, \dots, T', m \in R, \quad (5.23b)$$

$$s_{t,k}^v \in \{0, 1\}, \quad \forall t = 1, \dots, T', k \in K, \quad (5.23c)$$

$$s_{t,m}^r \in \{0, 1\}, \quad \forall t = 1, \dots, T', m \in R, \quad (5.23d)$$

$$p_{i,t}, b_{i,t} \geq 0, \quad \forall i \in S_0, t = 1, \dots, T', \quad (5.23e)$$

$$y_{i,t,k}^{u+}, y_{i,t,k}^{u-}, y_{i,t,k}^{b+}, y_{i,t,k}^{b-} \in \mathbb{Z}_+, \quad \forall i \in S_0, t = 1, \dots, T', k \in K, \quad (5.23f)$$

$$g_{i,t,m} \in \mathbb{Z}_+, \quad \forall i \in S_0, t = 1, \dots, T', m \in R, \quad (5.23g)$$

$$\partial_k^v \in \mathbb{Z}_+, \quad 0 \leq \partial_k^v \leq T', \quad \forall k \in K, \quad (5.23h)$$

$$\partial_m^r \in \mathbb{Z}_+, \quad 0 \leq \partial_m^r \leq T', \quad \forall m \in R, \quad (5.23i)$$

$$\psi_{i,j,t,k}^u, \psi_{i,j,t,k}^b \geq 0, \quad \forall i, j \in S_0, t = 1, \dots, T', k \in K, \quad (5.23j)$$

$$\hat{F}_i \geq 0, \quad \forall i \in S. \quad (5.23k)$$

The objective in (5.5) minimizes total user dissatisfaction at all stations on the following day. Constraints (5.6) introduce the auxiliary variable $\hat{F}_i$ for each station i and ensure that it is no smaller than every fitted plane used to approximate the EUDF value at the inventories after repositioning.

Constraints (5.7a) and (5.7b) keep track of the number of usable bikes at each node at the end of each period. Usable-bike inventory decreases when trucks collect usable bikes, increases when trucks deliver usable bikes, and also increases when broken bikes are repaired on site. Constraints (5.8a) and (5.8b) keep track of the number of broken bikes at each node at the end of each period. Broken-bike inventory decreases when trucks collect broken bikes or repairers repair broken bikes on site,

and it increases when trucks unload broken bikes. Constraints (5.9) stipulate that the total number of usable and broken bikes at each station is no more than its capacity.

Constraints (5.10a) specify the initial load of usable bikes on each truck during the first period. Constraints (5.10b) update the load of usable bikes on each truck at each node during each period. Constraints (5.10c) require that each truck carries no usable bikes at the end of the last period. Constraints (5.10d) ensure that usable bikes can be carried only on arcs traveled by trucks and that the usable-bike load does not exceed the truck capacity. Similar constraints for broken bikes are provided in constraints (5.11a)–(5.11d). Constraints (5.12) ensure that the total number of usable and broken bikes on each truck during each period is no more than its capacity.

Constraints (5.13a) and (5.13b) make sure that the quantity of usable bikes delivered to or collected from a visited node in a period is no more than the minimum of the node capacity and the truck capacity. Constraints (5.14a) limit the number of broken bikes that can be deposited at the depot in a period by a truck. Constraints (5.14b) ensure that no broken bikes are delivered to stations. Constraints (5.14c) enforce that no broken bikes are collected from the depot. Constraints (5.14d) make sure that the number of broken bikes collected from a visited station in a period is no more than the minimum of that station's initial broken-bike inventory and the truck capacity.

Constraints (5.15a) guarantee that no broken bikes are repaired by repairers at the depot. Constraints (5.15b) limit the number of broken bikes repaired at a station by a repairer in a period and link on-site repair quantities to repairer visits. Constraints (5.16) ensure that the total number of broken bikes collected by trucks and repaired by repairers at each station does not exceed that station's initial broken-bike inventory.

Constraints (5.17a) and (5.17b) require all trucks and repairers to depart from the depot initially. Constraints (5.17c) and (5.17d) require all trucks and repairers to return to the depot by the end of the rebalancing horizon. Constraints (5.18a) and (5.18b) are the flow conservation conditions for trucks and repairers, respectively, and account for the discrete travel duration on each arc. Constraints (5.19a)–(5.19d) relate $\varsigma_{t,k}^v$ and $\varsigma_{t,m}^r$ to ∂_k^v and ∂_m^r , respectively, thereby identifying the periods after a truck or repairer has made its final return to the depot and remains there.

Constraints (5.20a) and (5.20b) restrict the upper and lower bounds of the cumulative operational time for each truck. Specifically, constraints (5.20a) imply that, up to each period, the sum of truck travel time and loading or unloading time cannot exceed the time elapsed from the start of repositioning to the end of that period. Constraints (5.20b) imply that, up to each period before the truck enters its final depot-stay portion, the same cumulative time cannot be lower than the time elapsed to the end of two periods before that period. Similar constraints are imposed on repairers in constraints (5.20c) and (5.20d), where cumulative operational time includes repairer travel time and on-site repair time.

Constraints (5.21a) and (5.21b) ensure that a truck or repairer stays at the depot after its final return. Constraints (5.21c) and (5.21d) require that, before the final return to the depot, a truck or repairer that stays at the same node for another period must conduct at least one loading, unloading, or repair activity there. Constraints (5.22a) enforce that each truck can travel between at most one pair of nodes in a period. Similar constraints for repairers are given in constraints (5.22b). Constraints (5.22c) ensure that each station can be visited at most once by at most one repairer during the whole rebalancing horizon.

Constraints (5.23a) and (5.23b) define the binary route variables for trucks and repairers. Constraints (5.23c) and (5.23d) define the binary depot-stay variables. Constraints (5.23e) define the nonnegative inventory variables. Constraints (5.23f) and (5.23g) define the nonnegative integer loading, unloading, collection, delivery, and repair variables. Constraints (5.23h) and (5.23i) define the return-period variables and their bounds. Constraints (5.23j) define the nonnegative truck-load variables, and constraints (5.23k) define the nonnegative auxiliary EUDF variables.

The formulation specifies the BRP-BR-OSR feasible set and the EUDF-based objective on inventories after repositioning used as the reference problem in the remainder of this chapter.

5.2. EUDF-AWARE CLUSTER-FIRST, ROUTE-SECOND FRAMEWORK

This section presents the cluster-first route-second (CFRS) solution method for large-scale BRP-BR-OSR instances. CFRS decomposes the original network into two connected layers. In the clustering phase, the station set is partitioned into clusters whose internal inventory-adjustment potential is high. In the cluster-level

planning phase, each selected cluster is converted into a cluster-level BRP-BR-OSR instance and solved with the HGSADC-SBC algorithm of Hu, Szeto, and Ho [1].

The partition is the interface between the two layers. The partition search evaluates candidate partitions by Extended Activated Transfer Selection (EATS), a fast surrogate that estimates the EUDF reduction obtainable inside each cluster. The cluster-level planning phase receives the best-found partition and produces the cluster-level on-site repair and repositioning routes. The framework is specified through the station partition representation, the EATS cluster evaluator, the HGS-EUC partition search, and the cluster-level planning interface.

5.2.1. Station partition representation

Let S be the set of stations and let node 0 denote the depot. HGS-EUC represents a solution as a partition

$$\Pi = \{B_1, \dots, B_M\}, \quad B_m \subseteq S, \quad B_m \cap B_{m'} = \emptyset \ (m \neq m'), \quad \bigcup_{m=1}^M B_m = S. \quad (5.24)$$

Each block B_m is a station cluster. The solution stores the cluster membership of every station and one representative station, called the medoid, for each non-empty cluster. The medoid is an auxiliary representative of the cluster, not an additional decision variable of the original BRP-BR-OSR model. The initial bivariate station states (p_i^0, b_i^0) are fixed instance data and are not part of the solution representation.

The partition must satisfy cluster-size bounds

$$L \leq |B_m| \leq U, \quad m = 1, \dots, M, \quad (5.25)$$

where L and U are the lower and upper bounds on the number of stations in a cluster.

5.3. EXTENDED ACTIVATED TRANSFER SELECTION

For a partition $\Pi = \{B_1, \dots, B_M\}$, the search uses the score

$$S_{\text{part}}(\Pi) = \sum_{m=1}^M G(B_m), \quad (5.26)$$

where $G(B_m)$ is the EATS value of cluster B_m . A larger $S_{\text{part}}(\Pi)$ indicates a partition with larger estimated EUDF reduction under the cluster-level approximation.

EATS evaluates one cluster without constructing detailed vehicle or repairer routes. Its core idea is to greedily activate local inventory-changing operations that provide positive EUDF reduction per unit of proxy time, while maintaining inventory feasibility and a cluster time budget. The accepted operations define aggregate bivariate state changes, and the cluster value is the resulting EUDF reduction.

The operation records considered by EATS have three types: a usable-bike transfer from station i to station j , a broken-bike pickup at station i by a truck, and an on-site repair at station i by a repairer. An accepted record contains the operation type and an accepted quantity q . These records imply aggregate state changes (δ_i^p, δ_i^b) . A usable-bike transfer from i to j decreases δ_i^p by q and increases δ_j^p by q . A broken-bike pickup at i decreases δ_i^b by q . An on-site repair at i increases δ_i^p by q and decreases δ_i^b by q . After the accepted records are fixed, the cluster value is

$$G(B) = \sum_{i \in B} [F_i(p_i^0, b_i^0) - F_i(p_i^0 + \delta_i^p, b_i^0 + \delta_i^b)]. \quad (5.27)$$

The single-unit scores measure marginal EUDF reduction per unit of proxy operation time. At a working state (p_i, b_i) , the marginal EUDF reduction from removing one usable bike from station i is

$$\Delta_i^-(p_i, b_i) = F_i(p_i, b_i) - F_i(p_i - 1, b_i), \quad (5.28)$$

and the marginal EUDF reduction from adding one usable bike to station i is

$$\Delta_i^+(p_i, b_i) = F_i(p_i, b_i) - F_i(p_i + 1, b_i). \quad (5.29)$$

For a usable-bike transfer from i to j , the single-unit score is

$$\sigma_{ij}^U = \frac{\Delta_i^-(p_i, b_i) + \Delta_j^+(p_j, b_j)}{2t^{\text{load}}}. \quad (5.30)$$

For a broken-bike pickup at station i , the single-unit score is

$$\sigma_i^B = \frac{F_i(p_i, b_i) - F_i(p_i, b_i - 1)}{2t^{\text{load}}}. \quad (5.31)$$

For an on-site repair at station i , the single-unit score is

$$\sigma_i^R = \frac{F_i(p_i, b_i) - F_i(p_i + 1, b_i - 1)}{t^{\text{repair}}}. \quad (5.32)$$

Candidate operations are generated only when the resulting bivariate state is feasible and the corresponding score is positive.

The time proxy separates truck work and repairer work into two parallel streams. Let A_T be the set of stations activated by accepted usable-bike transfers or broken-bike pickups, and let A_R be the set of stations activated by accepted on-site repairs. Let Q_U , Q_B , and Q_R be the accepted quantities of usable-bike transfers, broken-bike pickups, and on-site repairs, respectively. The cluster time proxy is

$$\widehat{T}(B) = \max\{\widehat{T}_T(B), \widehat{T}_R(B)\}, \quad (5.33)$$

where

$$\widehat{T}_T(B) = \text{MSS}(A_T) + 2 \text{Depot}(A_T) + 2t^{\text{load}}(Q_U + Q_B), \quad (5.34)$$

and

$$\widehat{T}_R(B) = \rho [\text{MSS}(A_R) + 2 \text{Depot}(A_R)] + t^{\text{repair}} Q_R. \quad (5.35)$$

For any active set $A \subseteq B$, define

$$\bar{t}_{ij} = \max\{t_{ij}^v, t_{ji}^v\}, \quad i, j \in S_0, \quad (5.36)$$

and

$$\text{MSS}(A) = \max_{h \in A} \sum_{i \in A \setminus \{h\}} \bar{t}_{hi}, \quad (5.37)$$

$$\text{Depot}(A) = \max_{i \in A} \max\{t_{0i}^v, t_{i0}^v\}. \quad (5.38)$$

For $A = \emptyset$, both terms are set to zero. A candidate batch is admissible only if the updated active sets and accepted quantities satisfy

$$\widehat{T}(B) \leq T. \quad (5.39)$$

Algorithm 3 gives the complete EATS evaluation routine.

During EATS evaluation, station-level consistency is enforced throughout the

Algorithm 3: EATS evaluation of a cluster

Input: Cluster B , initial states (p_i^0, b_i^0) , EUDF functions $F_i(\cdot)$, travel-time data, time budget T , loading time t^{load} , repair time t^{repair} , and repairer travel-time multiplier ρ

Output: Cluster value $G(B)$ and accepted operation records $\mathcal{R}_B$

```
1 Step 0. Initialization;
2 Set  $(p_i, b_i) \leftarrow (p_i^0, b_i^0)$  and  $(\delta_i^p, \delta_i^b) \leftarrow (0, 0)$  for all  $i \in B$ ;
3 Set  $\mathcal{R}_B \leftarrow \emptyset$ ,  $A_T \leftarrow \emptyset$ ,  $A_R \leftarrow \emptyset$ , and  $Q_U, Q_B, Q_R \leftarrow 0$ ;
4 Initialize station-role flags and residual inventory counters;
5 Step 1. Candidate generation and scoring;
6 Generate feasible usable-bike transfer, broken-bike pickup, and on-site repair candidates
   under the current working state;
7 Compute their single-unit scores by (5.30)–(5.32);
8 Insert positive-score candidates into a unified priority queue;
9 Step 2. Greedy activation;
10 while the priority queue is non-empty do
11     Pop the candidate with the largest stored score;
12     Recheck feasibility under the current working state and station-role flags;
13     Recompute its single-unit score if the working state has changed;
14     if the recomputed score is positive then
15         Find the largest admissible batch quantity  $q$  that respects residual inventory,
           station-role consistency, and (5.39);
16         if  $q > 0$  then
17             Accept the batch, append the operation record to  $\mathcal{R}_B$ , and update  $(p_i, b_i)$  and
                $(\delta_i^p, \delta_i^b)$  at affected stations;
18             Update  $A_T, A_R, Q_U, Q_B, Q_R$ , residual inventory counters, and affected
               candidate scores;
19         end
20     end
21 end
22 Step 3. Residual improvement;
23 while a positive-score usable-bike transfer or on-site repair is feasible do
24     Select the best remaining candidate after lazy re-evaluation;
25     Accept the largest admissible batch quantity and update all affected states, records, active
       sets, quantities, and candidate scores;
26 end
27 Step 4. Cluster valuation;
28 Compute  $G(B)$  by (5.27);
29 return  $G(B)$  and  $\mathcal{R}_B$ ;
```

two selection phases. Usable-bike export and import roles are controlled by the p -direction state of a station. Broken-bike pickup and on-site repair draw from the same station-level broken-bike inventory, so their accepted quantities at a station are jointly bounded by the available broken bikes. This keeps (5.27) consistent with the aggregate bivariate state changes returned by Algorithm 3.

5.4. HYBRID GENETIC SEARCH FOR EXTENDED-UDF CLUSTERING

HGS-EUC is a hybrid genetic search for EUDF-guided clustering. It maintains a population of partition solutions, generates offspring through cluster-based crossover, restores cluster-size feasibility, improves solutions through local search education, and manages the population according to objective quality and partition diversity. Algorithm 4 summarizes the procedure. The station partition representation and EATS evaluation routine are defined in Sections 5.2 and 5.3, while the construction method, search operators, and population management rules are defined in the following subsections.

Algorithm 4: HGS-EUC partition search

Input: Station set S , number of clusters M , construction dissimilarity d^{con} , EATS value $G(\cdot)$, and HGS parameters
Output: Best-found partition Π^*

```

1  $\mathcal{P} \leftarrow \emptyset$ ;  $\Pi^* \leftarrow \emptyset$ ;  $it_{\text{noimp}} \leftarrow 0$ ;
2 while  $|\mathcal{P}| < 4\mu$  do
3   | Construct a k-medoids solution  $\Pi$  using  $d^{\text{con}}$ ;
4   | Repair cluster sizes, educate  $\Pi$ , evaluate  $S_{\text{part}}(\Pi)$ , and insert  $\Pi$  into  $\mathcal{P}$ ;
5   | Update  $\Pi^*$ ;
6 end
7 while the stopping criteria are unsatisfied do
8   | Select two parents from  $\mathcal{P}$  by tournament on biased fitness;
9   | Generate two offspring by cluster-based crossover with swapped parent order;
10  foreach offspring  $\Omega$  do
11    | Repair cluster sizes, educate  $\Omega$ , evaluate  $S_{\text{part}}(\Omega)$ , and insert  $\Omega$  into  $\mathcal{P}$ ;
12  end
13  if  $|\mathcal{P}| > \mu + \lambda$  then
14    | Apply survivor selection based on biased fitness;
15  end
16  Update  $\Pi^*$  and  $it_{\text{noimp}}$ ;
17  if the diversification interval is reached then
18    | Retain elite solutions and regenerate additional solutions;
19  end
20 end
21 return  $\Pi^*$ ;

```

5.4.1. Solution construction

Initial and regenerated solutions are produced by a k-medoids-style construction. The construction uses an auxiliary station-to-station dissimilarity matrix. For two stations $i, j \in S$, let $\hat{t}_{ij}$ be the normalized symmetric truck travel time and let $\hat{P}_{ij}$ be the normalized bilateral usable-transfer potential computed from the initial bivariate states. The construction dissimilarity is

$$d_{ij}^{\text{con}} = \alpha \hat{t}_{ij} + (1 - \alpha)(1 - \hat{P}_{ij}), \quad i, j \in S, \quad (5.40)$$

where $0 \leq \alpha \leq 1$. A larger α gives more weight to travel-time proximity, whereas a smaller α gives more weight to usable-transfer potential.

The first medoid is selected randomly. Subsequent medoids are sampled with probability increasing in each station's dissimilarity to its nearest current medoid. After medoids are initialized, each station is assigned to the nearest medoid, medoids are recomputed from the assigned clusters, and the assignment and medoid-update steps are repeated until the medoids stabilize or an iteration cap is reached. The constructed solution is then processed according to the initialization and diversification steps in Algorithm 4.

5.4.2. Cluster-based crossover

Given two parents $\Pi^{(1)} = \{B_1^{(1)}, \dots, B_M^{(1)}\}$ and $\Pi^{(2)} = \{B_1^{(2)}, \dots, B_M^{(2)}\}$, crossover copies a randomly selected half of the clusters from $\Pi^{(1)}$ into the offspring. For each station that remains unassigned, the operator reads its cluster assignment in $\Pi^{(2)}$. If that cluster is not already affected by the inherited clusters from $\Pi^{(1)}$, the station follows its assignment in $\Pi^{(2)}$. Otherwise, the station is assigned to the inherited cluster with the nearest medoid under d^{con} . Empty clusters are reseeded and medoids are recomputed at the end of the operator. Each mating event is applied with both parent orders, which produces two offspring.

5.4.3. Cluster-size repair and education

Cluster-size repair is applied before education in both population initialization and the evolutionary loop. Oversized clusters are reduced by relocating stations to clusters with available capacity, and undersized clusters are filled by pulling stations from clusters above the lower bound. Medoids are updated after repair.

Education improves a repaired solution through local search. The education module contains six operators: relocating one station, swapping two stations, merge-splitting two clusters, relocating a pair of stations, swapping a pair for a single station, and swapping one pair of stations for another pair. The operators compare candidate moves by recomputing the EATS scores of the affected clusters. In first-improvement mode, the operator order is randomized and the procedure stops after the first improving move. In exhaustive mode, the operator sequence is repeated until no operator yields further improvement.

5.4.4. Population management and diversification

Population management balances objective value and partition diversity. Since the search maximizes $S_{\text{part}}(\Pi)$, objective ranks are assigned in descending order of this score. Diversity is measured by pairwise same-cluster disagreement. For two solutions Π and Π' , define

$$D(\Pi, \Pi') = \frac{2}{|S|(|S| - 1)} \sum_{\{i,j\} \subseteq S} |\chi_{ij}(\Pi) - \chi_{ij}(\Pi')|, \quad (5.41)$$

where

$$\chi_{ij}(\Pi) = \begin{cases} 1, & \text{if stations } i \text{ and } j \text{ are in the same cluster in } \Pi, \\ 0, & \text{otherwise.} \end{cases} \quad (5.42)$$

The diversity rank is computed from the average disagreement to the n_{close} closest solutions in the population. The biased fitness of a solution is

$$BF(\Pi) = \text{rank}_{\text{obj}}(\Pi) + \left(1 - \frac{n_{\text{elite}}}{|\mathcal{P}|}\right) \text{rank}_{\text{div}}(\Pi). \quad (5.43)$$

Lower biased fitness is preferred in parent selection and survivor selection. When the population size exceeds $\mu + \lambda$, solutions with the worst biased fitness values are removed. Diversification is triggered after a prescribed number of consecutive non-improving iterations: an elite subset is retained, and additional solutions are generated by the construction procedure to restore population diversity. HGS-EUC terminates when the no-improvement limit, wall-clock limit, or maximum iteration limit is reached.

5.5. CLUSTER-LEVEL PLANNING

The cluster-level planning phase receives the best-found partition $\Pi^\star = \{B_1^\star, \dots, B_M^\star\}$. For each selected cluster $B_m^\star$, the adapter constructs the cluster-level node set

$$V_m^0 = B_m^\star \cup \{0\}, \quad (5.44)$$

where node 0 is the shared depot of the original network. The adapter reindexes the stations in $B_m^\star$ as local station nodes, extracts the corresponding travel-time, station-state, capacity, and EUDF data from the full instance, and forms an independent cluster-level BRP-BR-OSR instance.

Each cluster-level instance is solved with HGSADC-SBC following Hu, Szeto, and Ho [1]. Let $Z^{\text{dissat}}(B_m^\star)$ be the total user dissatisfaction returned for cluster $B_m^\star$. The CFRS cluster-level planning dissatisfaction is

$$Z_{\text{CFRS}}^{\text{dissat}} = \sum_{m=1}^M Z^{\text{dissat}}(B_m^\star). \quad (5.45)$$

This value reports the outcome of the complete CFRS solution after the cluster-level planning instances have been solved.

5.6. COMPUTATIONAL EXPERIMENTS

The proposed CFRS method was implemented in C++20 and controlled by a Python orchestrator. The experiments were carried out on two Linux workstations: a 13th Gen Intel Core i9-13900 workstation with 32 logical cores and 62 GiB RAM, and a 12th Gen Intel Core i9-12900 workstation with 24 logical cores and 62 GiB RAM. Method A uses Gurobi 13.0 to solve the time-indexed formulation in Section 5.1.4; Methods B and C use the HGSADC-SBC implementation of Hu, Szeto, and Ho [1], with Method C applying the CFRS decomposition proposed in Section 5.2 before solving the cluster-level planning instances.

The test instances were prepared as follows. For network sizes up to 500 stations, we use the instances of Hu, Szeto, and Ho [1], which are derived from the Citi Bike instances of Ho and Szeto [3]. For the 800-, 1000-, and 2000-station instances, we generate additional instances by sampling stations from a 2117-station Citi Bike pool covering Manhattan and Brooklyn and by recomputing asymmetric

station-to-station travel times with OSRM driving routes. In all instances, the station capacities, initial usable-bike inventories, initial broken-bike inventories, truck travel times, and EUDF values are treated as input data. The EUDF values are computed from historical rental and return rates following Kaspi, Raviv, and Tzur [2].

The computational study uses one 60-station calibration instance and three instance groups. The calibration instance is used only for parameter tuning. Set 2 contains five instances at each of the 60-, 200-, and 300-station scales and is used for the three-method benchmark. Set 3 contains five instances at each of the 400-, 500-, 800-, 1000-, and 2000-station scales and is used for the large-scale comparison. Set 4 contains additional configurations generated from the Set 3 instances by varying one input parameter at a time: truck capacity, operational time budget, target cluster size, and broken-bike rate.

Unless otherwise stated, the truck capacity is $Q = 25$ bikes, the loading or unloading time is $t^{\text{load}} = 60$ s per bike, the on-site repair time is $t^{\text{repair}} = 300$ s per bike, and the operational time budget is $T = 7200$ s. All method runs are given a 600 s computational limit. Each heuristic cell is replicated with 20 independent random seeds. For Method A, we report Gurobi’s lower bound, upper bound, and MIP gap. For Methods B and C, we report the average and best objective values over the 20 seeds. All objectives are total user dissatisfaction values and are minimized. CPU time is therefore not included as a comparison column.

The gap columns compare feasible objective values. Their definitions are given in the table notes. A positive gap means that Method C obtains a lower average objective than the reference method.

5.6.1. Parameter tuning

The HGS-EUC parameters in the clustering phase were tuned on the 60-station calibration instance. We used a sequential fixed-budget tuning protocol. In each phase, one parameter was varied while the previously selected parameters were held fixed; each candidate was evaluated over 20 independent seeds. The selection rule was mean objective first, mean paired rank second, and mean wall time third. Figure 5.1 shows the tuning results, and Table 5.5 lists the parameter values used in all subsequent experiments.

Figure placeholder for the sequential tuning results of the HGS-EUC parameters

Figure 5.1: Sequential tuning of the HGS-EUC parameters on the 60-station calibration instance. The selected value in each panel is marked by a red circle.

Table 5.5: Selected HGS-EUC parameters used in the computational experiments.

Parameter	Meaning	Value
T_A	Clustering-phase wall-clock limit (s)	120
μ	Population size	30
λ	Number of offspring per generation	30
n_{elite}	Number of elite solutions retained	8
n_{close}	Number of close solutions used for diversity ranking	7
α	Weight of travel-time proximity in the construction dissimilarity	0.6

The selected values are $T_A = 120$, $\mu = 30$, $\lambda = 30$, $n_{\text{elite}} = 8$, $n_{\text{close}} = 7$, and $\alpha = 0.6$. These values are fixed in the remainder of the computational study.

5.6.2. Benchmark comparison on small instances

This subsection compares three methods on Set 2. Method A solves the time-indexed MILP with Gurobi. Method B applies the HGSADC-SBC heuristic of Hu, Szeto, and Ho [1] directly to the full instance. Method C is the proposed CFRS method: HGS-EUC first partitions the station set, and HGSADC-SBC then solves the cluster-level planning instances.

Table 5.6: Small-scale benchmark comparison. Method A is Gurobi applied to the time-indexed MILP; Methods B and C are the monolithic HGSADC-SBC and the proposed CFRS method, respectively.

Instance	Method A (Gurobi)			Method B		Method C (CFRS)			
	LB ^a	UB ^b	MIP gap ^c (%)	Obj. Avg ^d	Obj. Best ^e	Obj. Avg ^d	Obj. Best ^e	Gap _A ^f (%)	Gap _B ^g (%)
60_1	10.53	218.00	95.17	96.59	90.07	83.15	79.30	61.86	13.91
60_2	37.63	367.55	89.76	225.80	218.10	214.68	204.66	41.59	4.93
60_3	28.68	397.09	92.78	204.85	192.00	180.98	175.88	54.42	11.65
60_4	68.40	434.88	84.27	293.52	276.18	268.27	260.67	38.31	8.60
60_5	19.68	212.69	90.75	125.02	116.26	113.27	110.45	46.74	9.40
200_1	48.52	1066.71	95.45	861.26	836.34	817.15	795.57	23.39	5.12
200_2	143.13	1407.38	89.83	1060.66	1046.52	1011.61	994.17	28.12	4.62
200_3	70.14	1195.02	94.13	840.61	816.88	800.38	783.98	33.02	4.79
200_4	113.95	1074.83	89.40	823.57	807.91	788.57	774.96	26.63	4.25
200_5	103.35	1013.39	89.80	787.52	775.05	742.45	728.31	26.74	5.72
300_1	82.39	1393.50	94.09	1144.48	1120.99	1054.79	1018.28	24.31	7.84
300_2	175.24	2040.34	91.41	1753.99	1733.60	1642.37	1622.58	19.51	6.36
300_3	99.65	1178.50	91.54	1029.25	1010.58	950.99	935.87	19.31	7.60
300_4	171.65	1960.64	91.25	1601.16	1572.52	1489.48	1463.77	24.03	6.97
300_5	152.90	1743.50	91.23	1384.85	1353.40	1286.88	1266.94	26.19	7.07

^a LB is Gurobi's final lower bound.

^b UB is Gurobi's final upper bound.

^c MIP gap is Gurobi's final MIP gap.

^d Obj. Avg is the average objective value over 20 independent runs.

^e Obj. Best is the best objective value over 20 independent runs.

^f $\text{Gap}_A = (\text{UB}_A - \bar{Z}_C) / \text{UB}_A$.

^g $\text{Gap}_B = (\bar{Z}_B - \bar{Z}_C) / \bar{Z}_B$. Positive gaps indicate lower average objectives for Method C.

Table 5.6 shows that Gurobi does not close the MIP gap within the 600 s limit on these instances. The average MIP gap is 91.39%. Method C obtains a lower average feasible objective than both the Gurobi upper bound and Method B in all 15 instances. The average improvement of Method C over the Gurobi upper bound is 32.94%, and the average improvement over Method B is 7.26%.

5.6.3. Comparison on large instances

For larger networks, the time-indexed MILP is no longer used as a computational benchmark. The comparison therefore focuses on Methods B and C, both of which

are run on Set 3 with the same computational budget and the same 20 random seeds.

Table 5.7: Large-scale comparison between Method B and Method C.

Instance	Method B		Method C (CFRS)		Gap _B ^a (%)
	Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
400_1	2022.80	1983.87	1870.30	1846.87	7.54
400_2	1713.69	1705.29	1575.77	1561.58	8.05
400_3	2090.49	2053.35	1973.52	1961.06	5.60
400_4	2120.33	2099.38	1966.92	1956.64	7.24
400_5	1918.34	1902.06	1776.22	1766.06	7.41
500_1	2409.88	2365.99	2191.53	2178.43	9.06
500_2	2798.26	2790.33	2561.41	2535.51	8.46
500_3	2076.70	2031.93	1914.42	1891.98	7.81
500_4	2415.63	2391.20	2180.17	2153.61	9.75
500_5	2553.49	2518.15	2375.26	2350.38	6.98
800_1	3460.09	3449.64	3188.80	3156.45	7.84
800_2	4032.11	3990.51	3625.44	3596.43	10.09
800_3	4013.71	3992.52	3656.68	3619.33	8.90
800_4	3820.65	3744.37	3544.29	3476.61	7.23
800_5	3528.83	3514.34	3215.21	3195.54	8.89
1000_1	4328.55	4298.80	3930.07	3906.01	9.21
1000_2	5211.50	5186.77	4787.90	4715.01	8.13
1000_3	4805.44	4783.74	4364.95	4334.91	9.17
1000_4	4622.22	4588.71	4264.53	4237.72	7.74
1000_5	4920.69	4906.12	4592.36	4556.58	6.67
2000_1	9659.61	9551.43	8741.95	8694.27	9.50
2000_2	10597.14	10542.07	9323.29	9139.02	12.02
2000_3	10247.37	10143.42	8890.51	8844.98	13.24
2000_4	10276.37	10267.57	9025.16	8968.67	12.18
2000_5	10742.77	10634.74	9495.00	9400.45	11.61

^a Gap_B = $(\bar{Z}_B - \bar{Z}_C)/\bar{Z}_B$. Positive values indicate lower average objectives for Method C.

Table 5.7 shows that Method C obtains a lower average objective than Method B in every large instance. The average improvement over Method B is 8.81% across all 25 instances and 11.71% across the five 2000-station instances. These results indicate that the cluster-first decomposition improves the feasible solutions obtained by the routing heuristic at the larger scales tested here.

5.6.4. Comparison under varied instance configurations

The following experiments are additional instance-configuration tests rather than a sensitivity analysis intended to derive operational prescriptions. Each test varies one input parameter while keeping the same comparison structure: Method B is applied to the full instance, Method C applies CFRS, and both methods are evaluated using the same computational budget. The purpose is to examine whether the advantage of Method C persists across different configurations of the benchmark instances.

Vehicle-capacity configurations

Truck capacity is varied over $Q \in \{15, 20, 25, 30, 35\}$ bikes. The remaining parameters are kept at their default values.

Table 5.8: Comparison under vehicle-capacity configurations.

Instance	Q	Method B		Method C (CFRS)		Gap_B^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
400_1	15	2050.52	2035.29	1940.12	1918.93	5.38
	20	2027.33	1999.33	1896.80	1861.56	6.44
	25	2022.80	1983.87	1870.30	1846.87	7.54
	30	2000.07	1976.68	1870.30	1820.26	6.49
	35	1978.66	1956.82	1859.69	1820.26	6.01
400_2	15	1736.56	1726.75	1611.73	1586.85	7.19
	20	1719.42	1708.60	1588.61	1563.74	7.61
	25	1713.69	1705.29	1575.77	1561.58	8.05
	30	1707.80	1696.48	1564.24	1550.97	8.41
	35	1696.29	1688.29	1554.54	1544.74	8.36
400_3	15	2145.10	2124.06	1999.58	1968.71	6.78
	20	2115.40	2112.41	1985.18	1968.71	6.16
	25	2090.49	2053.35	1973.52	1961.06	5.60
	30	2068.76	2032.23	1961.95	1953.18	5.16
	35	2068.76	2032.23	1951.61	1931.54	5.66
400_4	15	2151.88	2124.61	2002.57	1989.61	6.94
	20	2138.02	2124.61	1983.96	1963.67	7.21
	25	2120.33	2099.38	1966.92	1956.64	7.24
	30	2120.33	2088.97	1952.94	1936.35	7.89
	35	2108.38	2088.88	1943.24	1919.65	7.83
400_5	15	1941.67	1926.85	1817.96	1804.00	6.37

Continued on next page

Table 5.8 (continued)

Instance	Q	Method B		Method C (CFRS)		Gap_B^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	20	1935.43	1902.06	1794.02	1778.25	7.31
	25	1918.34	1902.06	1776.22	1766.06	7.41
	30	1916.99	1897.45	1775.29	1762.34	7.39
	35	1900.08	1881.05	1775.20	1762.34	6.57
500_1	15	2452.88	2431.00	2267.25	2250.17	7.57
	20	2434.52	2408.26	2224.77	2213.42	8.62
	25	2409.88	2365.99	2191.53	2178.43	9.06
	30	2409.88	2365.99	2191.53	2178.43	9.06
	35	2409.88	2365.99	2189.98	2178.43	9.12
500_2	15	2818.51	2797.67	2604.74	2578.16	7.58
	20	2811.89	2797.67	2604.74	2578.16	7.37
	25	2798.26	2790.33	2561.41	2535.51	8.46
	30	2772.85	2751.99	2545.39	2523.18	8.20
	35	2772.85	2751.99	2545.39	2520.97	8.20
500_3	15	2129.27	2106.20	1967.43	1947.29	7.60
	20	2112.51	2104.23	1945.45	1928.97	7.91
	25	2076.70	2031.93	1914.42	1891.98	7.81
	30	2074.35	2031.93	1911.15	1891.98	7.87
	35	2074.35	2031.93	1903.82	1884.88	8.22
500_4	15	2464.13	2450.83	2270.99	2247.42	7.84
	20	2420.85	2405.24	2214.63	2196.32	8.52
	25	2415.63	2391.20	2180.17	2153.61	9.75
	30	2402.50	2378.46	2180.17	2153.61	9.25
	35	2382.60	2355.12	2163.75	2148.69	9.19
500_5	15	2607.35	2595.00	2445.96	2425.57	6.19
	20	2588.18	2570.74	2425.16	2369.22	6.30
	25	2553.49	2518.15	2375.26	2350.38	6.98
	30	2553.49	2518.15	2375.26	2350.38	6.98
	35	2553.49	2518.15	2375.26	2350.38	6.98
800_1	15	3492.04	3472.37	3253.01	3244.30	6.84
	20	3469.23	3449.64	3231.18	3200.12	6.86
	25	3460.09	3449.64	3188.80	3156.45	7.84
	30	3458.97	3442.31	3188.80	3156.45	7.81
	35	3444.26	3433.92	3170.20	3131.65	7.96
800_2	15	4109.32	4078.84	3743.69	3704.46	8.90
	20	4078.20	4038.43	3696.98	3670.77	9.35
	25	4032.11	3990.51	3625.44	3596.43	10.09

Continued on next page

Table 5.8 (continued)

Instance	Q	Method B		Method C (CFRS)		Gap_B^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	30	4032.11	3990.51	3625.44	3596.43	10.09
	35	4032.11	3990.51	3613.25	3580.40	10.39
800_3	15	4073.09	4066.69	3742.33	3716.65	8.12
	20	4045.57	4033.86	3668.18	3627.45	9.33
	25	4013.71	3992.52	3656.68	3619.33	8.90
	30	4011.14	3967.06	3628.45	3582.50	9.54
	35	4006.22	3967.06	3628.45	3567.36	9.43
800_4	15	3866.17	3827.27	3591.97	3545.69	7.09
	20	3847.96	3799.04	3591.97	3545.69	6.65
	25	3820.65	3744.37	3544.29	3476.61	7.23
	30	3820.65	3744.37	3533.05	3476.61	7.53
	35	3820.65	3744.37	3510.43	3476.61	8.12
800_5	15	3583.42	3570.71	3267.76	3246.57	8.81
	20	3548.84	3529.98	3267.76	3231.20	7.92
	25	3528.83	3514.34	3215.21	3195.54	8.89
	30	3516.66	3509.76	3210.93	3186.22	8.69
	35	3502.38	3482.74	3182.75	3148.97	9.13
1000_1	15	4425.99	4405.27	4010.42	3981.73	9.39
	20	4373.76	4364.55	3969.82	3945.95	9.24
	25	4328.55	4298.80	3930.07	3906.01	9.21
	30	4328.55	4298.80	3918.82	3864.00	9.47
	35	4328.55	4298.80	3916.46	3864.00	9.52
1000_2	15	5277.42	5255.57	4949.27	4897.99	6.22
	20	5227.69	5209.61	4929.98	4897.99	5.69
	25	5211.50	5186.77	4787.90	4715.01	8.13
	30	5209.22	5174.49	4787.90	4715.01	8.09
	35	5180.54	5119.82	4787.90	4715.01	7.58
1000_3	15	4841.94	4822.34	4504.46	4451.68	6.97
	20	4833.95	4818.08	4441.06	4424.52	8.13
	25	4805.44	4783.74	4364.95	4334.91	9.17
	30	4762.84	4710.22	4360.69	4278.97	8.44
	35	4745.25	4683.10	4329.85	4278.97	8.75
1000_4	15	4718.10	4708.23	4388.04	4371.89	7.00
	20	4670.84	4589.52	4353.27	4288.04	6.80
	25	4622.22	4588.71	4264.53	4237.72	7.74
	30	4618.56	4588.71	4258.20	4237.58	7.80

Continued on next page

Table 5.8 (continued)

Instance	Q	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
1000_5	35	4603.15	4561.84	4258.20	4206.97	7.49
	15	4984.93	4941.86	4671.52	4631.41	6.29
	20	4946.44	4916.01	4629.44	4613.82	6.41
	25	4920.69	4906.12	4592.36	4556.58	6.67
	30	4895.01	4854.54	4550.16	4495.00	7.04
2000_1	35	4892.22	4854.54	4550.16	4495.00	6.99
	15	10054.25	9890.36	8933.20	8893.66	11.15
	20	9955.45	9823.04	8757.81	8694.27	12.03
	25	9659.61	9551.43	8741.95	8694.27	9.50
	30	9659.61	9551.43	8667.99	8640.72	10.27
2000_2	35	9659.61	9551.43	8667.99	8615.80	10.27
	15	10961.22	10766.11	9473.78	9429.74	13.57
	20	10727.63	10565.64	9357.71	9268.38	12.77
	25	10597.14	10542.07	9323.29	9139.02	12.02
	30	10596.89	10538.27	9252.03	9139.02	12.69
2000_3	35	10596.89	10536.82	9192.77	9128.02	13.25
	15	10423.65	10268.34	9137.37	9086.59	12.34
	20	10247.37	10247.37	9004.63	8935.21	12.13
	25	10247.37	10143.42	8890.51	8844.98	13.24
	30	10232.00	10143.42	8886.76	8844.98	13.15
2000_4	35	10229.71	10143.42	8853.63	8746.22	13.45
	15	10508.61	10318.40	9096.25	9025.45	13.44
	20	10372.33	10267.57	9035.34	8968.67	12.89
	25	10276.37	10267.57	9025.16	8968.67	12.18
	30	10146.97	9952.14	8963.83	8873.62	11.66
2000_5	35	10146.97	9952.14	8922.17	8815.42	12.07
	15	11089.73	10933.37	9645.85	9501.20	13.02
	20	10845.13	10634.74	9556.73	9473.16	11.88
	25	10742.77	10634.74	9495.00	9400.45	11.61
	30	10742.77	10634.74	9495.00	9400.45	11.61
2000_5	35	10742.77	10634.74	9454.01	9319.16	12.00

^a Gap_B = $(\bar{Z}_B - \bar{Z}_C)/\bar{Z}_B$. Positive values indicate lower average objectives for Method C.

Table 5.8 shows that Method C obtains a lower average objective than Method B for every vehicle-capacity configuration. The average Gap_B is 8.67% over all rows of the table, and the average gap remains between 8.34% and 8.90% at each tested

value of Q .

Time-budget configurations

The operational time budget is varied over $T \in \{3600, 5400, 7200, 9000, 10800\}$ s. The remaining parameters are kept at their default values.

Table 5.9: Comparison under time-budget configurations.

Instance	T (s)	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
400_1	3600	2135.89	2113.57	2088.25	2073.37	2.23
	5400	2055.10	2028.70	1947.32	1926.27	5.24
	7200	2022.80	1983.87	1870.30	1846.87	7.54
	9000	1969.97	1944.50	1812.78	1791.60	7.98
	10800	1958.52	1944.50	1743.51	1710.14	10.98
400_2	3600	1803.03	1792.58	1744.26	1730.41	3.26
	5400	1746.81	1731.66	1641.34	1625.43	6.04
	7200	1713.69	1705.29	1575.77	1561.58	8.05
	9000	1678.92	1675.83	1520.60	1514.28	9.43
	10800	1638.51	1618.89	1483.44	1480.92	9.46
400_3	3600	2257.68	2238.08	2194.85	2182.75	2.78
	5400	2155.52	2128.97	2047.08	2024.85	5.03
	7200	2090.49	2053.35	1973.52	1961.06	5.60
	9000	2066.76	2053.35	1887.76	1872.97	8.66
	10800	2011.15	1991.58	1840.55	1828.50	8.48
400_4	3600	2226.03	2216.40	2160.20	2135.11	2.96
	5400	2164.43	2157.78	2039.31	2019.69	5.78
	7200	2120.33	2099.38	1966.92	1956.64	7.24
	9000	2097.95	2082.52	1898.39	1886.77	9.51
	10800	2054.83	2034.69	1803.37	1759.65	12.24
400_5	3600	2017.28	2006.29	1967.52	1947.23	2.47
	5400	1957.63	1944.47	1864.52	1852.01	4.76
	7200	1918.34	1902.06	1776.22	1766.06	7.41
	9000	1893.21	1872.77	1723.99	1705.78	8.94
	10800	1847.76	1824.11	1651.90	1612.03	10.60
500_1	3600	2557.72	2540.33	2464.80	2442.30	3.63
	5400	2483.97	2466.06	2305.71	2293.40	7.18
	7200	2409.88	2365.99	2191.53	2178.43	9.06
	9000	2389.16	2365.99	2130.21	2097.87	10.84

Continued on next page

Table 5.9 (continued)

Instance	T (s)	Method B		Method C (CFRS)		Gap_B^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
500_2	10800	2320.00	2279.03	2050.34	2033.00	11.62
	3600	2957.35	2924.98	2861.51	2851.86	3.24
	5400	2859.37	2841.85	2694.11	2685.88	5.78
	7200	2798.26	2790.33	2561.41	2535.51	8.46
	9000	2741.41	2726.88	2467.84	2454.63	9.98
	10800	2659.08	2630.20	2392.08	2376.47	10.04
500_3	3600	2223.89	2214.53	2128.61	2109.12	4.28
	5400	2132.26	2123.10	1988.88	1971.66	6.72
	7200	2076.70	2031.93	1914.42	1891.98	7.81
	9000	2049.66	2016.75	1837.09	1820.55	10.37
	10800	2007.66	1968.89	1771.13	1758.44	11.78
500_4	3600	2528.21	2511.37	2433.00	2416.63	3.77
	5400	2443.96	2421.80	2280.61	2267.75	6.68
	7200	2415.63	2391.20	2180.17	2153.61	9.75
	9000	2346.04	2297.77	2111.44	2094.94	10.00
	10800	2318.94	2278.65	2029.88	2005.42	12.47
500_5	3600	2697.00	2691.78	2621.52	2596.47	2.80
	5400	2607.90	2595.86	2457.67	2438.14	5.76
	7200	2553.49	2518.15	2375.26	2350.38	6.98
	9000	2547.63	2515.98	2306.11	2288.41	9.48
	10800	2501.98	2480.47	2224.39	2209.62	11.09
800_1	3600	3622.09	3604.93	3489.16	3459.52	3.67
	5400	3525.30	3503.02	3313.53	3272.31	6.01
	7200	3460.09	3449.64	3188.80	3156.45	7.84
	9000	3384.86	3339.37	3084.92	3049.90	8.86
	10800	3329.90	3289.60	2975.31	2923.61	10.65
800_2	3600	4233.43	4217.27	4044.47	4032.77	4.46
	5400	4134.07	4112.83	3817.36	3782.04	7.66
	7200	4032.11	3990.51	3625.44	3596.43	10.09
	9000	3995.22	3986.96	3534.33	3488.31	11.54
	10800	3922.09	3880.94	3435.27	3402.85	12.41
800_3	3600	4187.22	4132.05	4059.29	4035.62	3.06
	5400	4082.67	4040.74	3810.00	3787.65	6.68
	7200	4013.71	3992.52	3656.68	3619.33	8.90
	9000	3972.79	3938.23	3510.91	3474.31	11.63
	10800	3861.50	3845.57	3389.02	3343.05	12.24

Continued on next page

Table 5.9 (continued)

Instance	T (s)	Method B		Method C (CFRS)		Gap_B^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
800_4	3600	4054.61	4045.89	3896.88	3872.37	3.89
	5400	3926.90	3867.29	3712.69	3672.46	5.45
	7200	3820.65	3744.37	3544.29	3476.61	7.23
	9000	3753.96	3727.15	3439.14	3395.75	8.39
	10800	3678.48	3649.48	3311.39	3293.25	9.98
800_5	3600	3684.86	3647.53	3544.39	3524.32	3.81
	5400	3581.32	3531.78	3339.87	3319.27	6.74
	7200	3528.83	3514.34	3215.21	3195.54	8.89
	9000	3493.35	3482.81	3122.69	3090.75	10.61
	10800	3398.40	3380.45	3015.27	2978.82	11.27
1000_1	3600	4581.99	4567.86	4334.05	4307.33	5.41
	5400	4402.96	4381.09	4070.45	4045.66	7.55
	7200	4328.55	4298.80	3930.07	3906.01	9.21
	9000	4248.63	4216.09	3715.60	3667.60	12.55
	10800	4141.95	4075.56	3601.45	3553.40	13.05
1000_2	3600	5472.11	5451.57	5287.26	5248.41	3.38
	5400	5327.43	5293.58	5000.94	4979.61	6.13
	7200	5211.50	5186.77	4787.90	4715.01	8.13
	9000	5110.56	5076.78	4663.04	4644.27	8.76
	10800	5027.02	5011.78	4542.38	4500.55	9.64
1000_3	3600	5056.03	5038.13	4810.55	4790.36	4.86
	5400	4897.56	4852.39	4545.50	4521.16	7.19
	7200	4805.44	4783.74	4364.95	4334.91	9.17
	9000	4700.38	4667.38	4235.45	4219.90	9.89
	10800	4575.79	4557.58	4104.73	4031.45	10.29
1000_4	3600	4874.38	4859.73	4702.11	4680.79	3.53
	5400	4732.15	4702.24	4430.36	4401.94	6.38
	7200	4622.22	4588.71	4264.53	4237.72	7.74
	9000	4570.92	4539.94	4088.87	4009.57	10.55
	10800	4460.78	4409.42	3972.46	3929.86	10.95
1000_5	3600	5222.83	5189.28	4997.04	4967.14	4.32
	5400	5050.09	5030.73	4742.41	4713.79	6.09
	7200	4920.69	4906.12	4592.36	4556.58	6.67
	9000	4834.43	4822.52	4409.06	4372.05	8.80
	10800	4728.96	4706.61	4286.71	4237.62	9.35
2000_1	3600	10231.42	10121.58	9552.02	9485.73	6.64
	5400	10129.33	9992.59	9061.70	8999.43	10.54

Continued on next page

Table 5.9 (continued)

Instance	T (s)	Method B		Method C (CFRS)		Gap_B^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	7200	9659.61	9551.43	8741.95	8694.27	9.50
	9000	9659.61	9551.43	8419.51	8385.01	12.84
	10800	9659.61	9551.43	8205.63	8111.02	15.05
2000_2	3600	10901.06	10874.86	10128.89	10099.85	7.08
	5400	10679.85	10679.85	9622.80	9589.41	9.90
	7200	10597.14	10542.07	9323.29	9139.02	12.02
	9000	10521.20	10462.33	9041.62	8921.74	14.06
	10800	10377.16	10269.67	8732.92	8630.75	15.84
2000_3	3600	10501.65	10501.65	9742.18	9722.08	7.23
	5400	10295.80	10143.42	9236.36	9180.79	10.29
	7200	10247.37	10143.42	8890.51	8844.98	13.24
	9000	10145.08	10091.75	8649.49	8572.69	14.74
	10800	10000.27	9894.68	8335.96	8297.65	16.64
2000_4	3600	10497.62	10389.15	9789.32	9762.83	6.75
	5400	10442.54	10318.27	9262.53	9181.11	11.30
	7200	10276.37	10267.57	9025.16	8968.67	12.18
	9000	10181.32	10166.95	8679.15	8499.59	14.75
	10800	10120.12	10061.47	8439.24	8373.58	16.61
2000_5	3600	11037.30	10969.20	10322.48	10281.23	6.48
	5400	10800.54	10684.97	9866.29	9808.57	8.65
	7200	10742.77	10634.74	9495.00	9400.45	11.61
	9000	10699.24	10575.86	9145.91	9091.90	14.52
	10800	10573.62	10455.51	8939.68	8825.90	15.45

^a $\text{Gap}_B = (\bar{Z}_B - \bar{Z}_C) / \bar{Z}_B$. Positive values indicate lower average objectives for Method C.

Table 5.9 shows that Method C obtains a lower average objective than Method B for every time-budget configuration. The average Gap_B increases from 4.24% at $T = 3600$ s to 11.93% at $T = 10800$ s, showing that the relative advantage of the CFRS decomposition is maintained throughout the tested time budgets.

Target-cluster-size configurations

The target cluster size is varied over $\{50, 75, 100, 125, 150\}$ stations. This parameter belongs only to Method C, because Method B solves the full instance without a clustering phase.

Table 5.10: Comparison under target-cluster-size configurations. Method B has no cluster-size parameter; therefore its objective values are shown once within each instance block.

Instance	Target size	Method B		Method C (CFRS)		
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	Gap _B ^a (%)
400_1	50	2022.80	1983.87	1007.91	999.32	50.17
	75			1139.68	1127.81	43.66
	100			1870.30	1846.87	7.54
	125			1297.38	1289.11	35.86
	150			1370.05	1344.09	32.27
400_2	50	1713.69	1705.29	1380.45	1366.01	19.45
	75			1515.48	1491.33	11.57
	100			1575.77	1561.58	8.05
	125			1690.78	1665.67	1.34
	150			1706.48	1698.31	0.42
400_3	50	2090.49	2053.35	1672.66	1616.70	19.99
	75			1834.90	1809.01	12.23
	100			1973.52	1961.06	5.60
	125			2016.84	1985.68	3.52
	150			2039.63	2030.08	2.43
400_4	50	2120.33	2099.38	1574.16	1534.39	25.76
	75			1713.63	1704.52	19.18
	100			1966.92	1956.64	7.24
	125			1895.87	1862.23	10.59
	150			1993.54	1966.72	5.98
400_5	50	1918.34	1902.06	1283.04	1263.78	33.12
	75			1418.22	1401.82	26.07
	100			1776.22	1766.06	7.41
	125			1599.26	1591.16	16.63
	150			1670.09	1651.69	12.94
500_1	50	2409.88	2365.99	1449.19	1416.08	39.86
	75			1665.64	1643.90	30.88
	100			2191.53	2178.43	9.06
	125			1923.10	1912.73	20.20
	150			1921.45	1907.27	20.27
500_2	50	2798.26	2790.33	1636.47	1617.14	41.52
	75			1826.32	1798.02	34.73
	100			2561.41	2535.51	8.46
	125			2069.93	2056.14	26.03

Continued on next page

Table 5.10 (continued)

Instance	Target size	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
500_3	150			2075.03	2039.77	25.85
	50	2076.70	2031.93	1803.60	1773.67	13.15
	75			1989.47	1969.61	4.20
	100			1914.42	1891.98	7.81
	125			2027.16	2003.31	2.39
	150			2051.95	2025.66	1.19
500_4	50	2415.63	2391.20	1923.71	1891.82	20.36
	75			2159.06	2129.77	10.62
	100			2180.17	2153.61	9.75
	125			2244.92	2228.54	7.07
	150			2259.54	2242.93	6.46
500_5	50	2553.49	2518.15	1565.91	1548.62	38.68
	75			1765.11	1735.64	30.87
	100			2375.26	2350.38	6.98
	125			2012.35	2000.84	21.19
	150			2017.83	1998.54	20.98
800_1	50	3460.09	3449.64	2817.92	2740.43	18.56
	75			3230.09	3182.40	6.65
	100			3188.80	3156.45	7.84
	125			3344.32	3321.12	3.35
	150			3389.65	3325.31	2.04
800_2	50	4032.11	3990.51	2695.49	2670.32	33.15
	75			2986.28	2965.62	25.94
	100			3625.44	3596.43	10.09
	125			3264.81	3238.21	19.03
	150			3372.68	3349.80	16.35
800_3	50	4013.71	3992.52	3064.21	3034.94	23.66
	75			3444.57	3416.41	14.18
	100			3656.68	3619.33	8.90
	125			3753.87	3727.24	6.47
	150			3858.12	3826.07	3.88
800_4	50	3820.65	3744.37	3063.71	3017.19	19.81
	75			3394.11	3331.82	11.16
	100			3544.29	3476.61	7.23
	125			3765.07	3716.98	1.45
	150			3801.39	3744.62	0.50

Continued on next page

Table 5.10 (continued)

Instance	Target size	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
800_5	50	3528.83	3514.34	3005.16	2955.83	14.84
	75			3347.69	3314.77	5.13
	100			3215.21	3195.54	8.89
	125			3474.97	3427.12	1.53
	150			3449.16	3438.88	2.26
1000_1	50	4328.55	4298.80	3147.26	3071.89	27.29
	75			3589.50	3554.51	17.07
	100			3930.07	3906.01	9.21
	125			4142.95	4057.49	4.29
	150			4168.62	4118.20	3.69
1000_2	50	5211.50	5186.77	3441.82	3410.79	33.96
	75			3904.20	3786.69	25.08
	100			4787.90	4715.01	8.13
	125			4327.05	4275.41	16.97
	150			4445.31	4386.41	14.70
1000_3	50	4805.44	4783.74	4047.16	4022.68	15.78
	75			4573.96	4511.39	4.82
	100			4364.95	4334.91	9.17
	125			4709.19	4618.60	2.00
	150			4799.57	4733.08	0.12
1000_4	50	4622.22	4588.71	3718.44	3696.03	19.55
	75			4122.92	4083.54	10.80
	100			4264.53	4237.72	7.74
	125			4460.99	4383.11	3.49
	150			4425.38	4290.54	4.26
1000_5	50	4920.69	4906.12	3646.94	3590.14	25.89
	75			4003.04	3939.36	18.65
	100			4592.36	4556.58	6.67
	125			4541.63	4508.07	7.70
	150			4631.29	4595.34	5.88
2000_1	50	9659.61	9551.43	7443.86	7250.44	22.94
	75			8191.41	8115.42	15.20
	100			8741.95	8694.27	9.50
	125			9087.45	9051.66	5.92
	150			9273.45	9238.11	4.00
2000_2	50	10597.14	10542.07	7902.37	7795.12	25.43
	75			8688.31	8621.67	18.01

Continued on next page

Table 5.10 (continued)

Instance	Target size	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	100			9323.29	9139.02	12.02
	125			9748.27	9692.06	8.01
	150			9901.57	9824.65	6.56
2000_3	50	10247.37	10143.42	7513.66	7445.21	26.68
	75			8387.31	8220.24	18.15
	100			8890.51	8844.98	13.24
	125			9306.14	9205.07	9.19
	150			9426.57	9306.30	8.01
2000_4	50	10276.37	10267.57	7748.81	7650.43	24.60
	75			8505.06	8425.14	17.24
	100			9025.16	8968.67	12.18
	125			9326.73	9282.82	9.24
	150			9478.20	9339.21	7.77
2000_5	50	10742.77	10634.74	8078.27	7880.65	24.80
	75			8928.28	8854.37	16.89
	100			9495.00	9400.45	11.61
	125			9849.64	9788.57	8.31
	150			9977.00	9917.27	7.13

^a Gap_B = $(\bar{Z}_B - \bar{Z}_C)/\bar{Z}_B$. Positive values indicate lower average objectives for Method C.

Table 5.10 shows that the target cluster size directly affects the performance of Method C. Method C has the largest average advantage when the target cluster size is 50 stations, where the average Gap_B is 26.36%. The average Gap_B decreases to 17.96% at 75 stations and 8.81% at 100 stations; it remains positive at 10.07% and 8.64% for target sizes 125 and 150, respectively. Thus, smaller cluster sizes generally perform better in these tests, but Method C maintains a positive average advantage over Method B for all tested target sizes. This is consistent with the role of the target cluster size in CFRS: it determines the sizes of the cluster-level planning subproblems passed to HGSADC-SBC.

Broken-bike-rate configurations

The broken-bike rate is varied over {0.00, 0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00}. The remaining parameters are kept at their default values.

Table 5.11: Comparison under broken-bike-rate configurations.

Instance	Broken-bike rate	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
400_1	0.00	692.01	676.13	600.15	586.16	13.27
	0.25	647.56	641.64	536.98	528.98	17.08
	0.50	649.65	646.29	561.51	554.35	13.57
	0.75	675.39	663.29	601.44	587.08	10.95
	1.00	706.54	684.20	634.21	630.70	10.24
	1.25	773.35	748.11	704.63	694.24	8.89
	1.50	890.86	870.11	822.87	814.29	7.63
	1.75	1173.46	1153.25	1070.37	1045.80	8.79
	2.00	1606.63	1561.56	1378.15	1356.37	14.22
400_2	0.00	1010.86	1005.92	921.86	917.20	8.80
	0.25	941.08	931.31	866.62	855.97	7.91
	0.50	952.63	943.20	880.89	871.01	7.53
	0.75	1000.53	995.47	932.79	924.35	6.77
	1.00	1064.49	1056.12	997.52	993.79	6.29
	1.25	1148.45	1135.65	1099.80	1090.84	4.24
	1.50	1356.59	1333.18	1294.55	1275.18	4.57
	1.75	1855.25	1818.82	1674.51	1658.93	9.74
	2.00	2491.75	2462.31	2209.84	2185.77	11.31
400_3	0.00	1106.91	1092.90	983.10	973.17	11.19
	0.25	1053.46	1044.37	931.56	919.52	11.57
	0.50	1053.92	1038.07	947.96	933.72	10.05
	0.75	1094.41	1086.10	993.67	986.05	9.21
	1.00	1145.09	1132.11	1049.36	1033.42	8.36
	1.25	1230.97	1224.26	1129.35	1119.21	8.25
	1.50	1400.39	1387.56	1297.07	1265.05	7.38
	1.75	1759.68	1735.34	1622.23	1583.05	7.81
	2.00	2307.40	2255.47	2131.36	2072.77	7.63
400_4	0.00	988.26	967.63	878.91	863.97	11.06
	0.25	923.63	918.42	831.61	819.89	9.96
	0.50	939.39	931.02	862.89	853.35	8.14
	0.75	975.84	972.05	916.70	908.34	6.06
	1.00	1023.59	1010.54	955.37	940.52	6.67
	1.25	1089.45	1072.95	1038.49	1028.15	4.68
	1.50	1239.62	1226.40	1178.67	1163.59	4.92
	1.75	1571.78	1560.81	1466.44	1426.79	6.70
	2.00	2119.05	2073.95	1939.81	1921.21	8.46

Continued on next page

Table 5.11 (continued)

Instance	Broken-bike rate	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
400_5	0.00	862.42	854.09	768.80	759.48	10.86
	0.25	804.29	795.42	710.35	698.12	11.68
	0.50	819.72	806.25	733.85	722.79	10.48
	0.75	864.77	846.57	777.70	770.85	10.07
	1.00	901.77	883.83	811.20	801.76	10.04
	1.25	980.31	966.83	890.21	883.67	9.19
	1.50	1144.59	1120.66	1040.69	1019.99	9.08
	1.75	1481.30	1430.28	1338.15	1303.97	9.66
	2.00	2070.01	2016.33	1776.64	1743.36	14.17
500_1	0.00	1067.63	1049.61	918.92	913.95	13.93
	0.25	974.36	961.00	839.94	827.27	13.80
	0.50	981.00	969.67	863.98	856.18	11.93
	0.75	1015.75	996.49	909.31	902.58	10.48
	1.00	1064.32	1048.43	973.94	963.18	8.49
	1.25	1140.51	1119.02	1045.29	1028.97	8.35
	1.50	1362.53	1296.69	1246.67	1223.81	8.50
	1.75	1878.53	1853.95	1676.37	1624.87	10.76
	2.00	2606.81	2491.64	2217.78	2141.65	14.92
500_2	0.00	1137.36	1118.45	1020.84	1003.32	10.24
	0.25	1076.63	1068.27	970.11	955.95	9.89
	0.50	1085.91	1076.22	989.42	980.15	8.89
	0.75	1114.40	1085.40	1037.48	1028.72	6.90
	1.00	1175.26	1166.80	1086.23	1074.60	7.57
	1.25	1258.70	1236.60	1156.31	1143.39	8.13
	1.50	1412.74	1398.04	1313.88	1310.12	7.00
	1.75	1727.76	1683.51	1606.66	1589.04	7.01
	2.00	2249.77	2210.15	2050.54	2009.48	8.86
500_3	0.00	1195.65	1185.44	1035.97	1024.83	13.35
	0.25	1117.78	1096.82	967.93	960.46	13.41
	0.50	1135.12	1127.09	999.33	992.10	11.96
	0.75	1171.31	1157.65	1046.03	1034.36	10.70
	1.00	1215.62	1204.70	1109.32	1091.79	8.74
	1.25	1314.14	1297.73	1194.63	1170.19	9.09
	1.50	1498.16	1486.32	1366.95	1356.58	8.76
	1.75	1925.71	1887.99	1699.10	1691.86	11.77
	2.00	2437.37	2408.37	2156.67	2147.46	11.52
500_4	0.00	1236.14	1229.51	1079.68	1067.06	12.66

Continued on next page

Table 5.11 (continued)

Instance	Broken-bike rate	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	0.25	1167.07	1151.23	1024.82	1013.99	12.19
	0.50	1172.29	1164.65	1055.55	1035.71	9.96
	0.75	1189.09	1153.31	1099.16	1092.79	7.56
	1.00	1263.56	1252.78	1179.11	1159.14	6.68
	1.25	1370.35	1355.40	1250.81	1237.49	8.72
	1.50	1511.71	1473.05	1424.32	1399.18	5.78
	1.75	1919.47	1885.05	1769.36	1728.34	7.82
	2.00	2565.07	2524.15	2286.86	2266.89	10.85
500_5	0.00	1048.96	1044.55	921.04	904.45	12.20
	0.25	965.78	943.73	832.58	816.60	13.79
	0.50	969.51	952.77	863.76	855.84	10.91
	0.75	998.33	974.65	909.46	900.96	8.90
	1.00	1058.62	1040.42	967.59	949.63	8.60
	1.25	1134.67	1092.63	1045.83	1036.46	7.83
	1.50	1332.10	1304.30	1225.23	1210.71	8.02
	1.75	1761.28	1714.49	1568.29	1550.42	10.96
	2.00	2270.34	2237.06	2002.63	1983.15	11.79
800_1	0.00	1629.51	1617.56	1362.63	1335.38	16.38
	0.25	1481.58	1464.16	1253.35	1234.88	15.40
	0.50	1479.30	1466.90	1288.00	1264.42	12.93
	0.75	1528.86	1513.97	1380.00	1369.55	9.74
	1.00	1622.06	1601.61	1474.98	1446.84	9.07
	1.25	1746.93	1717.39	1600.10	1581.88	8.40
	1.50	2037.14	1986.48	1845.49	1813.52	9.41
	1.75	2635.03	2598.26	2354.19	2304.40	10.66
	2.00	3575.40	3517.88	3003.78	2965.33	15.99
800_2	0.00	1847.04	1839.08	1583.46	1572.12	14.27
	0.25	1691.64	1675.17	1475.98	1464.04	12.75
	0.50	1721.59	1691.96	1524.46	1514.09	11.45
	0.75	1773.71	1755.82	1584.41	1568.54	10.67
	1.00	1856.81	1848.81	1695.86	1678.79	8.67
	1.25	1981.01	1973.41	1803.99	1790.64	8.94
	1.50	2223.58	2193.55	2026.28	1997.92	8.87
	1.75	2764.09	2738.46	2520.97	2499.29	8.80
	2.00	3717.08	3650.97	3211.52	3166.07	13.60
800_3	0.00	2125.71	2112.55	1812.46	1798.61	14.74
	0.25	1963.54	1947.57	1721.85	1697.27	12.31

Continued on next page

Table 5.11 (continued)

Instance	Broken-bike rate	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	0.50	1976.59	1956.01	1791.15	1772.25	9.38
	0.75	2045.18	2015.40	1875.11	1853.33	8.32
	1.00	2160.26	2132.02	1966.58	1930.98	8.97
	1.25	2309.83	2276.46	2133.07	2114.50	7.65
	1.50	2655.98	2622.47	2515.45	2470.32	5.29
	1.75	3379.46	3323.52	3065.43	3030.16	9.29
	2.00	4340.94	4223.64	3855.29	3838.26	11.19
800_4	0.00	2070.19	2022.56	1771.63	1740.92	14.42
	0.25	1886.97	1875.43	1647.44	1637.04	12.69
	0.50	1904.22	1897.74	1723.77	1708.53	9.48
	0.75	2000.95	1987.97	1816.93	1799.06	9.20
	1.00	2106.90	2065.98	1917.13	1907.02	9.01
	1.25	2274.95	2236.98	2077.89	2055.49	8.66
	1.50	2618.37	2595.38	2405.59	2373.23	8.13
	1.75	3348.50	3313.07	3090.13	3075.52	7.72
	2.00	4544.51	4467.46	3999.49	3928.45	11.99
800_5	0.00	2043.55	2034.58	1764.98	1759.99	13.63
	0.25	1858.46	1847.46	1644.39	1616.07	11.52
	0.50	1881.47	1865.62	1704.51	1688.39	9.41
	0.75	1954.39	1931.69	1809.81	1796.45	7.40
	1.00	2060.26	2027.74	1907.18	1894.97	7.43
	1.25	2211.05	2188.66	2072.02	2058.46	6.29
	1.50	2595.15	2565.01	2448.12	2426.16	5.67
	1.75	3447.31	3418.57	3118.34	3066.18	9.54
	2.00	4683.08	4613.77	4143.13	4081.36	11.53
1000_1	0.00	2331.78	2306.41	1953.46	1906.28	16.22
	0.25	2120.95	2108.25	1830.08	1822.19	13.71
	0.50	2129.36	2120.39	1870.25	1826.45	12.17
	0.75	2223.88	2203.77	1962.36	1945.91	11.76
	1.00	2313.62	2291.42	2104.03	2081.05	9.06
	1.25	2488.17	2472.00	2270.79	2250.47	8.74
	1.50	2926.11	2859.25	2653.95	2601.69	9.30
	1.75	3880.86	3811.41	3415.02	3387.24	12.00
	2.00	5285.49	5114.75	4439.23	4383.81	16.01
1000_2	0.00	2499.78	2491.56	2144.86	2123.06	14.20
	0.25	2276.77	2262.73	1988.71	1974.85	12.65
	0.50	2291.92	2270.43	2052.48	2024.22	10.45

Continued on next page

Table 5.11 (continued)

Instance	Broken-bike rate	Method B		Method C (CFRS)		
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	Gap _B ^a (%)
	0.75	2399.40	2371.12	2155.57	2133.53	10.16
	1.00	2519.41	2500.90	2324.96	2284.38	7.72
	1.25	2708.33	2688.19	2504.92	2479.98	7.51
	1.50	3150.31	3095.20	2977.57	2925.75	5.48
	1.75	4074.75	4032.93	3696.45	3659.04	9.28
	2.00	5441.59	5260.69	4797.06	4699.46	11.84
1000_3	0.00	2884.38	2877.54	2478.87	2450.24	14.06
	0.25	2648.95	2629.11	2345.59	2301.98	11.45
	0.50	2699.49	2665.07	2395.80	2366.22	11.25
	0.75	2806.40	2800.08	2534.55	2516.27	9.69
	1.00	2931.29	2887.41	2693.54	2675.99	8.11
	1.25	3186.10	3151.65	2913.72	2897.53	8.55
	1.50	3760.28	3733.38	3434.07	3354.27	8.68
	1.75	4802.99	4708.91	4345.25	4307.82	9.53
	2.00	6195.23	6082.96	5444.79	5386.25	12.11
1000_4	0.00	2554.14	2551.20	2159.99	2150.72	15.43
	0.25	2368.17	2361.46	2046.43	2001.36	13.59
	0.50	2377.79	2366.13	2108.29	2095.39	11.33
	0.75	2467.00	2435.60	2216.14	2193.65	10.17
	1.00	2566.22	2535.48	2364.37	2337.27	7.87
	1.25	2727.94	2672.47	2575.08	2538.15	5.60
	1.50	3091.31	3056.61	2883.31	2853.38	6.73
	1.75	3935.80	3905.66	3526.74	3510.51	10.39
	2.00	5074.16	4973.89	4466.93	4427.19	11.97
1000_5	0.00	2633.78	2564.83	2181.31	2156.94	17.18
	0.25	2402.49	2385.55	2071.43	2044.69	13.78
	0.50	2418.81	2388.55	2149.31	2093.14	11.14
	0.75	2512.79	2491.62	2239.90	2201.91	10.86
	1.00	2623.01	2593.90	2363.52	2337.36	9.89
	1.25	2799.35	2792.18	2556.49	2531.33	8.68
	1.50	3289.37	3271.48	3018.21	2949.09	8.24
	1.75	4195.22	4118.38	3786.66	3727.17	9.74
	2.00	5692.91	5576.82	4939.20	4899.86	13.24
2000_1	0.00	5083.35	5055.55	4132.11	4116.52	18.71
	0.25	4834.07	4681.22	3922.18	3876.06	18.86
	0.50	4894.39	4762.76	4016.83	3942.64	17.93
	0.75	5096.52	5013.49	4249.77	4221.37	16.61

Continued on next page

Table 5.11 (continued)

Instance	Broken-bike rate	Method B		Method C (CFRS)		Gap _B ^a (%)
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	
	1.00	5305.84	5277.84	4560.48	4511.25	14.05
	1.25	5818.97	5783.23	5016.99	4974.75	13.78
	1.50	6997.48	6842.58	5888.97	5850.29	15.84
	1.75	8980.51	8807.12	7419.30	7374.32	17.38
	2.00	11363.16	11224.53	9391.65	9311.69	17.35
2000_2	0.00	5505.27	5458.02	4545.47	4503.65	17.43
	0.25	5245.50	5116.10	4285.65	4264.73	18.30
	0.50	5309.18	5159.93	4404.73	4364.09	17.04
	0.75	5567.91	5529.40	4683.90	4653.75	15.88
	1.00	5794.53	5675.85	4976.86	4950.36	14.11
	1.25	6344.17	6270.50	5487.57	5460.43	13.50
	1.50	7666.30	7577.89	6395.32	6352.77	16.58
	1.75	10284.16	10095.96	8169.74	8082.39	20.56
	2.00	14309.97	14198.60	10499.37	10269.13	26.63
2000_3	0.00	5108.21	5026.63	4119.53	4098.52	19.35
	0.25	4769.59	4673.51	3863.54	3810.04	19.00
	0.50	4953.44	4919.71	3997.04	3970.84	19.31
	0.75	5059.43	4975.39	4202.60	4175.54	16.94
	1.00	5294.46	5229.26	4511.70	4450.19	14.78
	1.25	5731.57	5653.44	4922.83	4884.72	14.11
	1.50	6637.68	6544.52	5640.54	5586.19	15.02
	1.75	8372.46	8279.53	7004.40	6926.96	16.34
	2.00	10920.81	10731.88	9032.60	8912.98	17.29
2000_4	0.00	5116.89	5060.75	4177.00	4145.10	18.37
	0.25	4841.44	4693.09	3941.47	3918.73	18.59
	0.50	4932.45	4893.61	4079.22	4032.13	17.30
	0.75	5077.64	5004.31	4332.91	4282.86	14.67
	1.00	5284.89	5206.45	4586.41	4550.86	13.22
	1.25	5732.80	5674.35	5034.27	4995.65	12.18
	1.50	6753.00	6671.95	5823.93	5783.38	13.76
	1.75	8547.36	8323.60	7096.71	6994.24	16.97
	2.00	10430.03	10317.39	9033.45	8873.78	13.39
2000_5	0.00	5664.75	5625.34	4677.39	4654.86	17.43
	0.25	5397.70	5252.52	4434.76	4416.04	17.84
	0.50	5541.82	5513.89	4551.93	4527.96	17.86
	0.75	5666.34	5565.73	4807.66	4736.91	15.15
	1.00	5931.91	5872.64	5135.48	5096.11	13.43

Continued on next page

Table 5.11 (continued)

Instance	Broken-bike rate	Method B		Method C (CFRS)		
		Obj. Avg	Obj. Best	Obj. Avg	Obj. Best	Gap _B ^a (%)
	1.25	6519.05	6488.38	5620.10	5561.61	13.79
	1.50	7694.57	7613.75	6637.71	6592.03	13.74
	1.75	10131.48	9972.53	8354.15	8231.81	17.54
	2.00	13476.75	12623.03	10640.71	10574.31	21.04

^a $\text{Gap}_B = (\bar{Z}_B - \bar{Z}_C) / \bar{Z}_B$. Positive values indicate lower average objectives for Method C.

Table 5.11 shows that Method C obtains a lower average objective than Method B for every broken-bike-rate configuration. The average Gap_B is 11.42% over all rows, and the gap remains positive at every tested broken-bike rate.

5.7. CONCLUSIONS

This chapter proposes an EUDF-aware cluster-first route-second heuristic to solve the static bike repositioning problem with broken bikes and on-site repair. In this problem, trucks move usable bikes and collect broken bikes, while repairers visit stations to repair broken bikes on site. The objective is to minimize the total user dissatisfaction on the following day, evaluated by the Extended User Dissatisfaction Function. The proposed heuristic first applies a hybrid genetic search to partition the station network into clusters. An Extended Activated Transfer Selection procedure is used to estimate the user-dissatisfaction reduction within each cluster, and the resulting cluster-level planning problems are solved by the HGSADC-SBC heuristic. To illustrate the proposed heuristic's performance, it was tested on Citi Bike-based instances with up to 2000 stations. On the small benchmark instances, Gurobi does not close the MIP gap within the 600 s limit, and the proposed heuristic improves the Gurobi upper bounds by 32.94% on average. The average improvement over the monolithic HGSADC-SBC heuristic is 7.26% on the small instances and 8.81% on the large instances. For the five 2000-station instances, the average improvement is 11.71%. The computational results also show that the proposed heuristic maintains a positive advantage under different vehicle capacities, time budgets, and broken-bike rates. However, the heuristic fixes the station partition before detailed cluster-level planning, so it may miss useful operations involving stations in different clusters.

Therefore, a future research direction is to develop adaptive clustering and cross-cluster coordination mechanisms for large-scale bike-repositioning problems with broken bikes and on-site repair.

CHAPTER 6

CONCLUSIONS AND FUTURE RESEARCH DIRECTIONS

6.1. SUMMARY OF MAIN FINDINGS

This thesis studies static bike repositioning problems with user-dissatisfaction-based service measures, with particular attention to scalable decomposition methods and the operational treatment of broken bikes. Chapters 3–5 develop the main models and solution methods.

Chapter 3 considers the multi-vehicle static bike repositioning problem with the User Dissatisfaction Function (UDF) as the service-quality measure. The chapter formulates a UDF-aware cluster-first, route-second decomposition framework in which the clustering phase accounts for both user dissatisfaction reduction and routing-time feasibility. The proposed Activated Transfer Selection procedure provides an efficient approximation for evaluating clusters, and the Hybrid Genetic Search for UDF-aware Clustering (HGS-UC) makes the clustering model tractable for large station networks. The computational results show that the proposed decomposition can produce competitive solutions on small and medium instances and outperform several baseline clustering methods on larger instances.

Chapter 4 extends the static bike repositioning problem by incorporating broken bikes and on-site repair. The chapter formulates the problem as a time-indexed mixed-integer programming model involving truck routing, repairer routing, and station-level decisions for repairing, loading, and unloading bikes. It then develops the HGSADC-SBC heuristic, which combines Hybrid Genetic Search with Adaptive Diversity Control and a station-budget-constrained quantity assignment procedure. The computational results indicate that the heuristic can obtain high-quality feasible solutions more efficiently than direct optimization on larger instances. The experiments also show that the value of introducing on-site repair depends on the repair time and the amount of broken bikes in the system.

Chapter 5 combines the scalability motivation of Chapter 3 with the broken-bike and on-site-repair setting of Chapter 4. The chapter develops an EUDF-aware cluster-first, route-second framework for the static bike repositioning problem with broken bikes and on-site repair. The Extended Activated Transfer Selection procedure evaluates the expected benefit of clustering decisions, while the Hybrid

Genetic Search for Extended-UDF Clustering (HGS-EUC) searches for station partitions. Cluster-level planning is then performed using the Chapter 4 heuristic. The computational results show that the proposed framework improves upon monolithic planning heuristics and maintains its advantage across different instance sizes, vehicle capacities, time budgets, and broken-bike rates.

6.2. CONTRIBUTIONS OF THE THESIS

The thesis makes three main contributions. First, it develops user-dissatisfaction-aware decomposition models for static bike repositioning. Instead of treating clustering as a purely geographic or demand-balancing preprocessing step, the proposed methods incorporate service-quality effects directly into the clustering phase. This provides a closer link between network partitioning and the final repositioning objective.

Second, the thesis extends static bike repositioning models to account for broken bikes and on-site repair operations. By jointly considering usable-bike repositioning, broken-bike collection, and repairer routing, the formulation captures operational interactions that are important in systems where damaged bikes occupy docks and reduce service availability.

Third, the thesis designs scalable heuristic solution frameworks for large bike-sharing networks. The ATS and EATS procedures provide tractable cluster-evaluation mechanisms, while HGS-UC, HGSADC-SBC, and HGS-EUC address different layers of the resulting planning problems. Together, these methods show how user-dissatisfaction-based models can be made more computationally practical for large-scale static repositioning.

6.3. FUTURE RESEARCH DIRECTIONS

Several directions can naturally extend this work. First, future research may study dynamic and stochastic repositioning settings. The models in this thesis focus on static overnight planning with known initial inventories and planning inputs. Incorporating uncertain demand, uncertain damage discovery, and real-time inventory updates would allow the methods to support rolling-horizon or intra-day decision making.

Second, repair operations can be modeled in greater operational detail. Future extensions may consider heterogeneous repair types, uncertain repair durations, repairer shift scheduling, preventive maintenance, and coordination between on-site repair and depot-based repair. These extensions would be especially relevant when the damage level of a system changes over time.

Third, future work may strengthen coordination across clusters. The decomposition methods in Chapters 3 and 5 improve scalability by fixing station partitions before detailed routing or cluster-level planning. Adaptive repartitioning, boundary-station exchanges, or limited cross-cluster transfers may recover benefits that are missed by a fixed partition.

Finally, broader service-quality measures could be incorporated into the proposed frameworks. The UDF and EUDF capture expected user dissatisfaction associated with bike and dock availability, but operators may also consider fairness across stations, reliability over multiple periods, emissions, labor utilization, and robustness to disruptions. Extending the decomposition methods to handle such multi-criteria service measures would further improve their practical relevance.

APPENDIX A

Linearization of the EUDF

The Extended User Dissatisfaction Function (EUDF) is a bivariate function that represents a weighted sum of the expected shortages of usable bikes and the expected shortages of lockers at a bike-sharing station in a day. It is calculated based on the quantities of both usable and unusable bikes at the station. The calculation of the EUDF values follows a Markov process, which captures the dynamics of bicycle inventory levels over time. For detailed steps for how the EUDF values are calculated, readers can refer to the approach outlined in the paper of Kaspi et al. (2017).

Let $F_i(p'_i, b'_i)$ be the EUDF, where p'_i and b'_i are the usable and broken inventory levels at the station, respectively. The domain of EUDF at each station i is $\Theta_i = \{(p'_i, b'_i) \mid p'_i \geq 0, b'_i \geq 0, p'_i + b'_i \leq C_i\}$, where C_i is the station capacity. For each point $(p'_i, b'_i) \in \Theta_i$, a plane $\alpha_{p_i, b_i}^i \cdot p'_i + \beta_{p_i, b_i}^i \cdot b'_i + \gamma_{p_i, b_i}^i$ that passes as close as possible to $F_i(p'_i, b'_i)$ can be fitted, where α_{p_i, b_i}^i , β_{p_i, b_i}^i , and γ_{p_i, b_i}^i are the coefficients of the plane that can be computed for every $(p_i, b_i) \in \Theta_i$. These coefficients are obtained by solving a linear programming (LP) model which ensures that the polyhedral function $\tilde{F}_i(p'_i, b'_i) = \max_{(p_i, b_i) \in \theta} (\alpha_{p_i, b_i}^i \cdot p'_i + \beta_{p_i, b_i}^i \cdot b'_i + \gamma_{p_i, b_i}^i)$ remains below or equal to the EUDF values at all points except for $(p_i, b_i) \in \theta_i$, with the objective of minimizing the gap between the fitted function and the actual EUDF at each point. The LP model is given as follows:

Decision variables:

$\alpha_{p_i, b_i}^i, \beta_{p_i, b_i}^i, \gamma_{p_i, b_i}^i$ Coefficients of the fitted plane

s Gap at the point (p_i, b_i) between the fitted plane and the EUDF value

The LP formulation:

Minimize s (A.74)

s.t.

$$\alpha_{p_i, b_i}^i \cdot p'_i + \beta_{p_i, b_i}^i \cdot b'_i + \gamma_{p_i, b_i}^i \leq F(p'_i, b'_i) \quad \forall (p'_i, b'_i) \in \theta \setminus \{(p_i, b_i)\} \quad (\text{A.75})$$

$$\alpha_{p_i, b_i}^i \cdot p_i + \beta_{p_i, b_i}^i \cdot b_i + \gamma_{p_i, b_i}^i = F(p_i, b_i) - s \quad (\text{A.76})$$

$$\alpha_{p_i, b_i}^i, \beta_{p_i, b_i}^i, \gamma_{p_i, b_i}^i \quad free, \quad (A.77)$$

$$s \geq 0 \quad (A.78)$$

The objective (A.74) is to minimize the gap between the fitted plane and the EUDF value. Constraints (A.75) ensure that the fitted plane passes under the EUDF at all integer points except for (p_i, b_i) . Constraint (A.76) defines the gap at the point (p_i, b_i) between the fitted plane and the EUDF value. Constraints (A.77) and (A.78) are the domain constraints for the decision variables.

After the polyhedral function $\tilde{F}_i(p_i, b_i)$ is formed at each point $(p_i, b_i) \in \Theta_i$, we replace the EUDF $F_i(p_{i,T'}, b_{i,T'})$ in the objective function with an auxiliary variable $\hat{F}_i$ and add the following constraints to the model:

$$\hat{F}_i \geq \alpha_{p_i, b_i}^i \cdot p_{i,T'} + \beta_{p_i, b_i}^i \cdot b_{i,T'} + \gamma_{p_i, b_i}^i, \forall (p_i, b_i) \in \{(p_i, b_i) \in \mathbb{Z}^2 \mid p_i \geq 0, b_i \geq 0, p_i + b_i \leq C_i\}, \forall i \in S. \quad (A.79)$$

APPENDIX B

SBC Heuristic for the Quantity Assignment

Algorithm 2: *SBCAssignment* (S_R, S_K)

```

1   $p_s = p_s^0, b_s = b_s^0, \forall s \in S$  // initialization
   // lines 2 to 9: repairing quantity assignment
2  for  $i = 1$  to  $|R|$  do
3    // assign the repairing budget for repairer  $i$  at each visited station (Algorithm 3)
       $\left( \maxTime_{s,i}^r \right)_{s \in S} = allocateRepairBudget(r_i)$ 
      // determine the repairing quantity for repairer  $i$  at each visited station in route  $r_i$  (Algorithm 5)
4     $\left( g_{s,i} \right)_{s \in S} = assignRepair\left(r_i, \left( \maxTime_{s,i}^r \right)_{s \in S}\right)$ 
8    /* lines 5 to 8: station inventory update (create a duplicate of the usable and broken bike inventory)
5    for  $s = 1$  to  $|S|$  do
6       $p_s = p_s + g_{s,i}, p'_s = p_s$ 
7       $b_s = b_s - g_{s,i}, b'_s = b_s$ 
8    end for
9  end for
   // lines 10 to 36: truck (un)loading quantity assignment (the first round)
10 for  $k = 1$  to  $|K|$  do
11    $unsat_{s,k}^{load,u} = unsat_{s,k}^{unload,u} = unsat_{s,k}^{load,b} = 0, \forall s \in S$  // initialization
      /* assign the station (un)loading budget for truck  $k$  at each station (Algorithm 4) and record it in
12    $\left( \maxTimeInit_{s,k}^t \right)_{s \in S} = allocateLoadingBudget(r'_k)$ 
13    $\left( \maxTimeRemain_{s,k}^t \right)_{s \in S} = \left( \maxTimeInit_{s,k}^t \right)_{s \in S}$  // initialization
      /* at the first departure from the depot, we load no usable bikes by default, and the adjustment on
14    $y_{0,k}^u = y_{0,k}^u = y_{0,k}^{b+} = y_{0,k}^{b-} = 0, \psi_{0,k}^u = \psi_{0,k}^b = 0$ 
      // lines 15 to 33: assign loading and unloading quantities for each node in the route
15   for  $l = 1$  to  $|r'_k| - 1$  do
16      $\psi_{l,k}^u = \psi_{l,k}^b = 0$ 
17      $s' = s'_{kl}$ 
18      $y_{l,k}^u = y_{l,k}^u = y_{l,k}^{b+} = y_{l,k}^{b-} = 0$  // all (un)loading quantities are initialized with 0
19     if  $s' = 0$  then
       /* at intermediate visits to the depot, we load no usable bikes by default, and the adjustment on
20      $y_{l,k}^u = \psi_{l-1,k}^u, y_{l,k}^{b+} = \psi_{l-1,k}^b$ 
21   else
```

Algorithm 2: *SBCAssignment* (S_R, S_K)

```

22   if  $p_{s'} > q_{s'}$  then // for surplus stations, load usable bikes (Algorithm 6).
       $\left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,u} \right)_{s \in S}, y_{l,k}^{u-} =$ 
23    $assignLoadU \left( r'_k, l, \left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,u} \right)_{s \in S} \right)$ 
24   else
25   if  $p_{s'} < q_{s'}$  then // for deficit stations, unload usable bikes (Algorithms 7–8)
       $\left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,u} \right)_{s \in S}, \left( unsat_{s,k}^{unload,u} \right)_{s \in S}, y_{l,k}^{u+} =$ 
26    $assignUnloadU \left( r'_k, l, \left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,u} \right)_{s \in S}, \right.$ 
       $\left. \left( unsat_{s,k}^{unload,u} \right)_{s \in S} \right)$ 
27   end if
28   end if
      // station and truck inventory update
29    $p_{s'} = p_{s'} - y_{l,k}^{u-} + y_{l,k}^{u+}, \psi_{l,k}^u = \psi_{l,k}^u + y_{l,k}^{u-} - y_{l,k}^{u+}$ 
      // for both types of stations, load broken bikes (Algorithm 9)
       $\left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,b} \right)_{s \in S}, y_{l,k}^{b-} =$ 
30    $assignLoadB \left( r'_k, l, \left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{unload,b} \right)_{s \in S} \right)$ 
31    $b_{s'} = b_{s'} - y_{l,k}^{b-}, \psi_{l,k}^b = \psi_{l,k}^b + y_{l,k}^{b-}$  // station and truck inventory update
32   end if
33   end for
      //  $BCRF_s^t$  is updated here in preparation for the reassignment of the station (un)loading budget
34   Update  $BCRF_s^t$  for each station based on the current station inventories
      /* adjust the station un(loading) budget of truck  $k$  by distributing the unused time to unbalanced
35    $adjustLoadingBudget \left( \left( \maxTimeNew_{s,k}^t \right)_{s \in S} = \right.$ 
       $\left. \left( \maxTimeInit_{s,k}^t \right)_{s \in S}, \left( \maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,u} \right)_{s \in S}, \left( unsat_{s,k}^{unload,u} \right)_{s \in S}, \left( unsat_{s,k}^{load,b} \right)_{s \in S} \right)$ 
36   end for
      /* restore the station inventories to the inventories before the first-round truck (un)loading quantity
37    $p_s = p'_s, b_s = b'_s$ 
38   for  $k = 1$  to  $|K|$  do
      /* assign the quantities for bike (un)loading for the second round using the updated station (un)loading
39   Set  $\left( \maxTimeRemain_{s,k}^t \right)_{s \in S} = \left( \maxTimeNew_{s,k}^t \right)_{s \in S}$  and repeat the procedure from Line 15 to Line 28
40   end for

```

Algorithm 2: *SBCAssignment* (S_R, S_K)

41 Conduct amendments to both trucks and repairers regarding the routes and their associated repairing
42 **return** G_R, Y_K

The algorithm begins by assigning quantities of repairs for each repairer and updating the station inventories (Line 1 to Line 9). Following this, the algorithm iteratively traverses the route of each truck, determining the quantities of loading/unloading based on the status of each station (Line 10 to Line 40). A key feature of this SBC heuristic is setting a maximum time for bike loading/unloading for every truck and a maximum time for bike repairing for every repairer, referred to as the *station budget*. Specifically, we refer to the budget at each station as the *station (un)loading budget* for trucks and the *station repairing budget* for repairers. The introduction of the station budget prevents the truck/repairer from excessively spending time at the initial stations, which can lead to insufficient time for the latter stations. For each truck, the first round of (un)loading quantity assignments is conducted (Line 15 to Line 33), followed by an adjustment on the station (un)loading budgets based on unsatisfied demand and unused time budgets in the first round (Lines 34 and 35). Afterward, the algorithm restores the station inventories to the values before the truck (un)loading quantity assignment (Line 37) and conducts a second-round assignment to adjust the truck (un)loading quantities (Line 38 to Line 40). Next, amendments to both trucks and repairers regarding the routes and their associated repairing or (un)loading quantities are made to utilize the unused time budget resulting from the rounding-down strategies applied during quantity assignments (Line 41). Finally, the algorithm returns the assigned repairing and (un)loading quantities for usable and broken bikes (Line 42).

APPENDIX C

Assignment Algorithms for Station Repairing Budget and Station (Un)loading Budget

Algorithm 3: *allocateRepairBudget* (r_i)

```

1  Initialization:  $\maxTime_{s,i}^r = 0, \forall s \in S$ 
2   $T^{\text{op}} = T - \sum_{j=0}^{|r_i|-1} t_{s_{i_j}, s_{i_{j+1}}}^r$  // total time budget left for repairing bikes
3  while  $T^{\text{op}} < t^{\text{repair}}$  do
    // remove the station with the lowest benefit-cost ratio function value from the route
4  Remove station  $s' = \arg \min_{s \in S} BCRF_s^r$  from  $r_i$ 
5   $T^{\text{op}} = T - \sum_{j=0}^{|r_i|-1} t_{s_{i_j}, s_{i_{j+1}}}^r$ 
6  end while
    /* the set unsat stores the stations where broken bikes cannot be all repaired due to insufficient station repairing budget
7   $unsat = \emptyset$ 
    // record the overly assigned station repairing budget
8   $extraTime = 0$ 
    // lines 9 to 18: assign the maximum repairing quantity at each station ( $1 \leq j \leq |r_i| - 1$ )
9  for  $j$  in 1 to  $|r_i| - 2$  do
    // distribute the total station repairing budget using the ratio of the BCRF value of each station
10  $\maxTime_{s_{i_j}, i}^r = \frac{T^{\text{op}} \times BCRF_{s_{i_j}}^r (p_{s_{i_j}}, b_{s_{i_j}})}{\sum_{j'=1}^{|r_i|-2} BCRF_{s_{i_{j'}}}^r (p_{s_{i_{j'}}}, b_{s_{i_{j'}}})}$ 
11  $repair_{s_{i_j}}^{\text{max}} = \min \left\{ \max \left\{ q_{s_{i_j}} - p_{s_{i_j}}, 0 \right\}, b_{s_{i_j}} \right\}$  // maximum repairing quantity
12 if  $\maxTime_{s_{i_j}}^r < repair_{s_{i_j}}^{\text{max}} \cdot t^{\text{repair}}$  then
    // record stations with insufficient repairing budgets to repair all broken bikes
13  $unsat = unsat \cup \{s_{i_j}\}$ 
14 else
    // gather the excess repairing budget and adjust the repairing budget
15  $extraTime = extraTime + \maxTime_{s_{i_j}, i}^r - repair_{s_{i_j}}^{\text{max}} \cdot t^{\text{repair}}$ 
16  $\maxTime_{s_{i_j}}^r = repair_{s_{i_j}}^{\text{max}} \cdot t^{\text{repair}}$ 
17 end if
18 end for
    // lines 19 to 25: reassign the total excess time to nodes with insufficient budgets
19 while  $extraTime > t^{\text{repair}}$  and  $unsat \neq \emptyset$  do

```

Algorithm 3: *allocateRepairBudget* (r_i)

20 $\widehat{s}$ = The node in *unsat* with the highest $BCRF_s^r$ (break ties by choosing the smallest $\widehat{s}$)
21 $added = \min \left\{ extraTime, repair_{\widehat{s}}^{\max} \cdot t^{\text{repair}} - maxTime_{\widehat{s},i}^r \right\}$
22 $maxTime_{\widehat{s},i}^r = maxTime_{\widehat{s},i}^r + added$
23 $extraTime = extraTime - added$
24 $unsat = unsat \setminus \{\widehat{s}\}$
25 **end while**
26 **return** $\left(maxTime_{s,i}^r \right)_{s \in S}$

The algorithm begins by computing the total time available for repairs T^{op} by deducting travel time between stations from the total time budget T (Line 2). It then conducts a feasibility check and route pruning to ensure that T^{op} is sufficient for at least one repair (Line 3 to Line 6). Then, the algorithm assigns the total time available to each station based on $BCRF_s^r$, and determines the maximum possible number of repairs within the allocated time. Stations where the time is insufficient are added to *unsat*, while the excess time budget is added to *extraTime* (Line 7 to Line 18). If there is extra time left (i.e., $extraTime > 0$) and *unsat* is not empty, the algorithm redistributes the extra time to stations in *unsat*, prioritizing those with the highest $BCRF_s^r$ until all *extraTime* is allocated or *unsat* is empty (Line 19 to Line 25). Finally, the algorithm returns the station repairing budget for each station.

Algorithm 4: *allocateLoadingBudget* (r'_k)

1 Initialization: $maxTime_{s,k}^t = 0, \forall s \in S$
2 $T^{\text{op}} = T - \sum_{l=0}^{|r'_k|-1} t_{s'_{k_l}, s'_{k_{l+1}}}^t$ // The total time budget left for truck k to load and unload bikes
3 **while** $T^{\text{op}} < 2t^{\text{load}}$ **do**
 // remove the station with the lowest benefit-cost ratio function value from the route
4 Remove station $s' = \arg \min_{s \in S} BCRF_s^t$ from r'_k
5 $T^{\text{op}} = T - \sum_{l=0}^{|r'_k|-1} t_{s'_{k_l}, s'_{k_{l+1}}}^t$
6 **end while**
7 $unsat = \emptyset$
8 $extraTime = 0$
9 NS = The set of stations in r'_k

Algorithm 4: *allocateLoadingBudget* (r'_k)

```
10 for  $s$  in  $NS$  do
11    $maxTime_{s,k}^t = \frac{T^{op} \times BCRF_s^t(p_s, b_s)}{\sum_{s' \in NS} BCRF_{s'}^t(p_{s'}, b_{s'})}$ 
12    $load_s^{\max} = |p_s - q_s| + b_s$ 
13   if  $maxTime_{s,k}^t < 2t^{\text{load}} \cdot load_s^{\max}$  then
14      $unsat = unsat \cup \{s\}$ 
15   else
16      $extraTime = extraTime + maxTime_{s,k}^t - 2t^{\text{load}} \cdot load_s^{\max}$ 
17      $maxTime_{s,k}^t = 2t^{\text{load}} \cdot load_s^{\max}$ 
18   end if
19 end for
20 while  $extraTime > 2t^{\text{load}}$  and  $unsat \neq \emptyset$  do
21    $\hat{s}$  = The node in  $unsat$  with the highest  $BCRF_s^t$ 
22    $added = \min \left\{ extraTime, 2t^{\text{load}} \cdot load_{\hat{s}}^{\max} - maxTime_{\hat{s},k}^t \right\}$ 
23    $maxTime_{\hat{s},k}^t = maxTime_{\hat{s},k}^t + added$ 
24    $extraTime = extraTime - added$ 
25    $unsat = unsat \setminus \{\hat{s}\}$ 
26 end while
27 return  $\left( maxTime_{s,k}^t \right)_{s \in S}$ 
```

Note that for a visited surplus station, we must consider that each bike loaded onto the truck also needs to be unloaded at another node (either the depot or a deficit station). Consequently, we set the station (un)loading budget as twice the loading time for each bike at this station. For a visited deficit station, at the current stage, we assume that all bikes being unloaded are additionally loaded from the depot. Therefore, we also set the station (un)loading budget as twice the loading time for each bike at this station. It is obvious that this approach leads to an overestimation, as it double-counts the time for bikes that are simply being transferred from surplus to deficit stations. To correct this, we make appropriate adjustments to the station (un)loading budgets during the detailed station-by-station assignment process, as outlined in **Algorithms 6, 7, and 8** in subsequent sections.

APPENDIX D

Repairing, Loading, and Unloading Quantity Assignment Algorithms

We start by introducing the repairing quantity assignment. The algorithm takes the route of repairman i and the station repairing budget as the input and returns the number of broken bikes repaired by repairman i at each station. For surplus stations, no bike is repaired at the current stage. For deficit stations, the repairing quantity is constrained by the deviation of usable bike inventory from the target level and the station repairing budget.

Algorithm 5: $assignRepair \left(r_i, \left(maxTime_{s,i}^r \right)_{s \in S} \right)$

```

1  Initialization:  $g_{s,i} = 0, \forall s \in S$ 
2  for  $j = 1$  to  $|r_i| - 2$  do
3       $g_{s_{ij},i} = \min \left\{ \max \{ q_{s_{ij}} - p_{s_{ij}}, 0 \}, b_{s_{ij}}, \left\lfloor maxTime_{s_{ij},i}^r / t^{repair} \right\rfloor \right\}$ 
4  end for
5  return  $(g_{s,i})_{s \in S}$ 

```

Next, we present the algorithms for (un)loading quantity assignment of trucks, including usable bike loading (**Algorithm 6**), usable bike unloading (**Algorithms 7 and 8**), and broken bike loading (**Algorithm 9**). Note that we use l to represent the order index for defining when the concerned node is visited by truck and assigned a loading quantity in these four algorithms.

Algorithm 6: $assignLoadU \left(r'_k, l, \left(maxTimeRemain_{s,k}^t \right)_{s \in S}, \left(unsat_{s,k}^{load,u} \right)_{s \in S} \right)$

```

1   $s = s'_{k_l}$ 
2   $n = \min \left\{ Q_k - \psi_{l-1,k}^u - \psi_{l-1,k}^b, p_s - q_s \right\}$  // the quantity without the station loading budget
3   $y_{l,k}^{u-} = \min \left\{ n, \left\lfloor maxTimeRemain_{s,k}^t / 2t^{load} \right\rfloor \right\}$  // the quantity with the station loading budget
   // Record the number of bikes not loaded due to the limited station loading budget
4   $unsat_{s,k}^{load,u} = unsat_{s,k}^{load,u} + (n - y_{l,k}^{u-})$ 
5   $maxTimeRemain_{s,k}^t = maxTimeRemain_{s,k}^t - 2t^{load} \cdot y_{l,k}^{u-}$ 
6  return  $\left( maxTimeRemain_{s,k}^t \right)_{s \in S}, \left( unsat_{s,k}^{load,u} \right)_{s \in S}, y_{l,k}^{u-}$ 

```

Algorithm 6 calculates the loading quantity of usable bikes for truck k at a given node. The number of bikes loaded is constrained by the truck's capacity, the station's surplus inventory, and the station loading budget (Line 3). The algorithm records the unsatisfied loading quantity due to the limited station loading budget (Line 4) and updates the station loading budget (Line 5). Finally, it returns the updated remaining station (un)loading budget, unsatisfied loading quantity due to the limited station loading budget, and the number of bikes loaded onto truck k (Line 6).

Algorithm 7: $assignUnloadU \left(r'_k, l, \left(maxTimeRemain_{s,k}^t \right)_{s \in S}, \left(unsat_{s,k}^{load,u} \right)_{s \in S}, \left(unsat_{s,k}^{unload,u} \right)_{s \in S} \right)$

```

1   $s = s'_{k_l}$ 
2   $y_{l,k}^{u+} = \min \left\{ \psi_{l-1,k}^u, C_s - p_s - b_s, q_s - p_s \right\}$ 
   /* add back the double-counted loading time as these unloaded bikes are not from the depot but from
3   $maxTimeRemain_{s,k}^t = maxTimeRemain_{s,k}^t + 2t^{load} \cdot y_{l,k}^{u+}$ 
   // target inventory not reached due to insufficient bike supply on the truck
4  if  $y_{l,k}^{u+} < q_s - p_s$  and  $y_{l,k}^{u+} \neq C_s - p_s - b_s$  then
   /* line 5 to 20: load more bikes from the visited but still surplus stations and unload them to the
5  while  $\exists \hat{s} \in S : unsat_{\hat{s},k}^{load,u} > 0$  and  $maxTimeRemain_{s,k}^t \geq 2t^{load}$  do
6   $s' =$  the station with the highest  $BCRF_{\hat{s}}^t$  and  $unsat_{s',k}^{load,u} > 0$  (break ties by choosing the smallest
7   $l' =$  the index of station  $s'$  in the route  $r'_k$  (break ties by choosing the one with the largest order
   // minimum truck residual capacity from the chosen node to the current node
8   $rc_k = \min_{l''=l', l'+1, \dots, l-1} Q_k - \psi_{l'',k}^u - \psi_{l'',k}^b$ 
   // the extra loaded quantity with no station loading budget
9   $n = \min \left( q_s - p_s - y_{l,k}^{u+}, rc_k, unsat_{s',k}^{load,u} \right)$ 
   // the extra loaded quantity with the remaining station loading budget
10  $\Lambda^{load,u} = \min \left\{ n, \left\lfloor maxTimeRemain_{s,k}^t / 2t^{load} \right\rfloor \right\}$ 
11  $unsat_{s,k}^{unload,u} = unsat_{s,k}^{unload,u} + (n - \Lambda^{load,u})$  // update the unsatisfied unloading quantity
   // update the remaining station loading budget of the current station
12  $maxTimeRemain_{s,k}^t = maxTimeRemain_{s,k}^t - 2t^{load} \cdot \Lambda^{load,u}$ 
13  $unsat_{s',k}^{load,u} = unsat_{s',k}^{load,u} - \Lambda^{load,u}$  // update the unsatisfied loading quantity
14  $p_{s'} = p_{s'} - \Lambda^{load,u}$  // update the station inventory after extra loading of bikes
15 for  $l''$  in  $l'$  to  $l - 1$  do // update the truck inventories

```

Algorithm 7: $assignUnloadU \left(r'_k, l, \left(maxTimeRemain^t_{s,k} \right)_{s \in S}, \left(unsat^{load,u}_{s,k} \right)_{s \in S}, \left(unsat^{unload,u}_{s,k} \right)_{s \in S} \right)$

```

16    $\psi_{l'',k}^u = \psi_{l'',k}^u + \Lambda^{load,u}$ 
17   end for
18    $y_{l',k}^u = y_{l',k}^u + \Lambda^{load,u}$  // update the loading quantity
19    $y_{l,k}^u = y_{l,k}^u + \Lambda^{load,u}$  // update the unloading quantity
20   end while
21   if  $y_{l,k}^u < q_s - p_s$  then // the target inventory is still not reached
22      $dv = \max \{ dv \mid dv < l, s'_{k_{dv}} = 0 \}$  // order index of the last visited depot
23     // adjust the loading at the previously visited depot (Algorithm 8)
24      $h, maxTimeRemain^t_{s,k}, unsat^{unload,u}_{s,k} = adjustDepotLoading \left( dv, l, maxTimeRemain^t_{s,k}, unsat^{unload,u}_{s,k} \right)$ 
25      $y_{l,k}^u = y_{l,k}^u + h$ 
26   end if
27 end if
28 return  $\left( maxTimeRemain^t_{s,k} \right)_{s \in S}, \left( unsat^{load,u}_{s,k} \right)_{s \in S}, \left( unsat^{unload,u}_{s,k} \right)_{s \in S}, y_{l,k}^u$ 

```

Algorithm 7 determines the unloading quantity by considering the truck's usable bike inventory, the station's residual capacity, and the deviation of the inventory from the target inventory levels (Line 2). If the usable bike shortage cannot be satisfied due to insufficient usable bikes on the truck, the algorithm utilizes the unsatisfied loading quantities from previously visited stations that are still in a surplus state and tries to load more bikes at those stations to satisfy the shortage at the current station (Lines 5 to Line 20). Note that during this process, we also capture the number of usable bikes unable to be loaded from other nodes by truck k due to the insufficient station loading budget (Line 9 to Line 11). If the station's demand remains unmet, a final adjustment is made on the previous depot visit (Line 21 to Line 26) by reducing the number of usable bikes unloaded or loading more usable bikes there. The detailed procedure for adjustment at the depot is presented in **Algorithm 8**. Note that we use dv to denote the order index for the last visited depot in r'_k that requires (un)loading adjustment, h to denote the increased loading of usable bikes at the station that requires (un)loading adjustment, and s is the station that requires (un)loading adjustment from the previous depot.

Algorithm 8: $adjustDepotLoading \left(dv, l, maxTimeRemain_{s,k}^t, unsat_{s,k}^{unload,u} \right)$

```

1   $rc_k = \min_{l'=dv, dv+1, \dots, l-1} Q_k - \psi_{l',k}^u - \psi_{l',k}^b$ 
2   $n = \min \left\{ rc_k, q_s - p_s - y_{l,k}^{u+} \right\}$ 
3   $h = \min \left\{ n, \left\lfloor maxTimeRemain_{s,k}^t / 2t^{load} \right\rfloor \right\}$ 
4   $unsat_{s'_{kl},k}^{unload,u} = unsat_{s'_{kl},k}^{unload,u} + (n - h)$ 
5  if  $y_{dv,k}^{u+} > 0$  then // the original operation at the depot is unloading
6     $y_{dv,k}^{u+} = \max \left\{ y_{dv,k}^{u+} - h, 0 \right\}$  // reduce the unloading of usable bikes at the depot
    // if the reduction exceeds the unloading quantity, the final decision at the depot becomes loading
7     $y_{dv,k}^{u-} = \max \left\{ h - y_{dv,k}^{u+}, 0 \right\}$ 
8     $maxTimeRemain_{s'_{kl},k}^t = maxTimeRemain_{s'_{kl},k}^t - 2t^{load} \cdot y_{dv,k}^{u-}$ 
9  end if
10 if  $y_{dv,k}^{u-} > 0$  then // the original operation at the depot is loading
11    $y_{dv,k}^{u-} = y_{dv,k}^{u-} + h$  // increase the loading quantity
12    $maxTimeRemain_{s,k}^t = maxTimeRemain_{s,k}^t - 2t^{load} \cdot h$ 
13 end if
14 for  $l' = dv$  to  $l - 1$  do
15    $\psi_{l',k}^u = \psi_{l',k}^u + h$ 
16 end for
17 return  $h, maxTimeRemain_{s,k}^t, unsat_{s,k}^{unload,u}$ 

```

Algorithm 8 initiates by calculating the minimum residual capacity of the truck from the last visited depot to the upcoming station $s = s'_{kl}$ (Line 1). It then computes the additional number of bikes that need to be loaded at the depot, which is the lowest value among the truck's minimum residual capacity, the shortage of usable bikes at the upcoming station, and the number of additional bikes that can be loaded using the station loading budget of the upcoming deficit station (Lines 2–3). If the initial operation at the depot is to unload bikes, the algorithm decreases the unloading quantity by the newly calculated amount. If the adjustment leads to a surplus, the operation at the depot is switched from unloading to loading. Conversely, if the depot was initially set to load bikes, the algorithm simply increases the loading quantity by the computed adjustment (Line 5 to Line 13). The algorithm terminates by updating the truck inventory along the route from the depot to the upcoming deficit station and

returns the adjustment value. During the process, the unsatisfied unloading demand is limited by station unloading budgets and the budgets are updated accordingly (Line 4 and Line 12).

After usable bikes are loaded or unloaded, we continue to assign the quantities of broken bikes to be loaded using **Algorithm 9**. The assignment follows a similar procedure as **Algorithm 6**, with the loading quantities of broken bikes constrained by the residual capacity of the truck, the broken bike inventory at the station, and the station loading budget.

Algorithm 9: $assignLoadB \left(r'_k, l, \left(maxTimeRemain^t_{s,k} \right)_{s \in S}, \left(unsat^{load,b}_{s,k} \right)_{s \in S} \right)$

- 1 $s = s'_{k_l}$
- 2 $n = \min \left\{ b_s, Q_k - \psi_{l-1,k}^u - \psi_{l-1,k}^b \right\}$
- 3 $y_{l,k}^{b^-} = \min \left\{ n, \left\lfloor maxTimeRemain^t_{s,k} / 2t^{load} \right\rfloor \right\}$
- 4 // The number of broken bikes not loaded due to the limited station loading budget
 $unsat^{load,b}_{s,k} = unsat^{load,b}_{s,k} + \left(n - y_{l,k}^{b^-} \right)$
- 5 $maxTimeRemain^t_{s,k} = maxTimeRemain^t_{s,k} - 2t^{load} \cdot y_{l,k}^{b^-}$
- 6 **return** $\left(maxTimeRemain^t_{s,k} \right)_{s \in S}, \left(unsat^{load,b}_{s,k} \right)_{s \in S}, y_{l,k}^{b^-}$

APPENDIX E

(Un)loading Budget Adjustment Algorithm for Trucks

Algorithm 10 shows the procedure of the (un)loading budget adjustment.

Algorithm 10: $adjustLoadingBudget \left(\left(\maxTimeInit_{s,k}^t \right)_{s \in S}, \left(\maxTimeRemain_{s,k}^t \right)_{s \in S}, \left(unsat_{s,k}^{load,u} \right)_{s \in S}, \left(unsat_{s,k}^{unload,u} \right)_{s \in S}, \left(unsat_{s,k}^{load,b} \right)_{s \in S} \right)$

```

1   $unusedTime_k = 0$  // the variable that stores the time budget that is assigned but not used
   /* lines 2 to 6: add up all the bikes not loaded/unloaded at each station due to insufficient station (u)
2  for  $\hat{s}$  in  $S$  do
3       $unsat_{\hat{s},k} = unsat_{\hat{s},k}^{load,u} + unsat_{\hat{s},k}^{load,b} + unsat_{\hat{s},k}^{unload,u}$ 
4       $maxTimeNew_{\hat{s},k}^t = maxTimeInit_{\hat{s},k}^t - maxTimeRemain_{\hat{s},k}^t$ 
5       $unusedTime_k = unusedTime_k + maxTimeRemain_{\hat{s},k}^t$ 
6  end for
7  while  $unusedTime_k \geq 2t^{load}$  and  $\exists \hat{s} \in S : unsat_{\hat{s},k} > 0$  do
8       $s' =$  The station that satisfies  $unsat_{s',k} > 0$  with the highest  $BCRF_{s'}^t$  (break ties by choosing the sm
9       $\Delta = \min \{ unusedTime_k, 2t^{load} \cdot unsat_{s',k} \}$ 
10      $maxTimeNew_{s',k}^t = maxTimeNew_{s',k}^t + \Delta$ 
11      $unusedTime_k = unusedTime_k - \Delta$ 
12      $unsat_{s',k} = unsat_{s',k} - \lfloor \Delta / 2t^{load} \rfloor$ 
13 end while
14 return  $\left( maxTimeNew_{s,k}^t \right)_{s \in S}$ 

```

Algorithm 10 begins with a traversal of each station in the network, counting the total number of bikes not (un)loaded at each station due to insufficient (un)loading budgets, reducing the overly assigned budget from the initial station (un)loading budget, and sum up these unused times (Line 1 to Line 6). The algorithm then enters a loop where it redistributes unused time budget to stations with unsatisfied loading or unloading needs, i.e., the bikes that could not be loaded or unloaded due to limited station (un)loading budgets (Line 7 to Line 13). Within each iteration, the algorithm selects the station with the highest $BCRF_{s'}^t$ and allocates additional time from the unused budget to make up for this insufficiency (Line 8 to Line 12). This process is repeated until there is either no more unused time or all the demands have been met. Finally, the updated station (un)loading budgets are returned (Line 14).

Bibliography

- [1] R. Hu, W. Y. Szeto, and S. C. Ho. “Repositioning in bike sharing systems with broken bikes considering on-site repairs”. In: *Transportation Research Part E: Logistics and Transportation Review* 201 (2025), p. 104155.
- [2] M. Kaspi, T. Raviv, and M. Tzur. “Bike-sharing systems: User dissatisfaction in the presence of unusable bicycles”. In: *IIE Transactions* 49.2 (2017), pp. 144–158. DOI: 10.1080/0740817X.2016.1224960.
- [3] S. C. Ho and W. Y. Szeto. “A hybrid large neighborhood search for the static multi-vehicle bike-repositioning problem”. In: *Transportation Research Part B: Methodological* 95 (2017), pp. 340–363. DOI: 10.1016/j.trb.2016.11.003.